Karlsruhe University of Applied Sciences

Department of Computer Science and Business Information Systems

Business Information Systems

BACHELOR´S THESIS

Efficiency vs. Performance in Green AI: An Empirical Investigation of Energy Consumption, Carbon Footprint, and Predictive Accuracy of Classification Algorithms at Varying Dataset Sizes.

Author	Eron Qorri
Matriculation number	85383
Workplace	Hochschule Karlsruhe
First Advisor	Prof. Dr. rer. nat. Christine Preisach
Second Advisor	Prof. Dr. Reimar Hofmann
Closing Date	31 July, 2026

31 July, 2026

Chairman of the Examination Board

Contents

List of Abbreviations

ACC Accuracy.

Adam Adaptive Moment Estimation.

AI Artificial Intelligence.

AMP Automatic Mixed Precision.

API Application Programming Interface.

BERT Bidirectional Encoder Representations from Transformers.

BO Bayesian Optimization.

CPU Central Processing Unit.

CUDA Compute Unified Device Architecture.

CV Cross-Validation.

DL Deep Learning.

DRAM Dynamic Random-Access Memory.

EPM External Power Meter.

EWF1 Efficiency-Weighted F1.

FLOP Floating-Point Operation.

FN False Negative.

FP False Positive.

FP32 32-bit single-precision floating-point format.

GBDT Gradient Boosted Decision Trees.

gCO_2eq grams of CO_2-equivalent.

GP Gaussian Process.

GPU Graphics Processing Unit.

HPO Hyperparameter Optimization.

JIT Just-In-Time.

kgCO_2eq kilograms of CO_2-equivalent.

KNN K-Nearest-Neighbor.

kWh kilowatt-hour.

LOOCV Leave-One-Out Cross-Validation.

LR Logistic Regression.

ML Machine Learning.

MLP Multilayer Perceptron.

NLP Natural Language Processing.

NN Neural Network.

NVML NVIDIA Management Library.

OOB Out-of-Bag.

PMC Performance Monitoring Counter.

RAM Random-Access Memory.

RAPL Running Average Power Limit.

ReLU Rectified Linear Unit.

RF Random Forest.

SGD Stochastic Gradient Descent.

SIMT Single Instruction, Multiple Threads.

TDP Thermal Design Power.

TN True Negative.

TP True Positive.

TPE Tree-structured Parzen Estimator.

WF1 Weighted F1-Score.

XGB Extreme Gradient Boosting.

1 Introduction

2 Theoretical Background

This chapter establishes the theoretical foundations required to understand the empirical study presented in the remainder of this thesis. It first frames the research within the broader debate between *Red AI* and *Green AI*, then formalizes the concepts of energy consumption and carbon emissions in Machine Learning (ML). Subsequently, the relevant classification algorithms are described from a purely theoretical standpoint, followed by the metrics used to evaluate their predictive performance, the validation strategy, and Hyperparameter Optimizations (HPOs) techniques. All implementation details are deliberately deferred to Chapter 5.

2.1 Green AI and Red AI

In recent years, a paradigm has emerged in Artificial Intelligence (AI) research that Schwartz et al. refer to as "Red AI " [1]. Red AI describes research approaches that primarily aim to improve prediction accuracy through the massive use of computational power, data, and parameters, while largely disregarding the associated infrastructural and energetic costs. The underlying dynamic driving this trend is that the relationship between model performance and model complexity is at best logarithmic [1]: a linear increase in accuracy requires an exponentially larger model or exponentially more training data. Empirically, since 2012 the compute used to train state-of-the-art models has been doubling approximately every 3.4 months [2]. This is particularly evident in large Transformer architectures. Strubell et al. document that training such models can require days of continuous computation across multiple Graphics Processing Units (GPUs), while foundation models such as GPT-3 scale to 175 billion parameters [3].

In response, the concept of *Green AI* has emerged as a counter-movement within the ML community [1]. Green AI is defined as research that yields novel results while explicitly accounting for computational costs, elevating efficiency to a primary evaluation criterion alongside predictive accuracy. Beyond the environmental dimension, the movement also addresses inclusivity: the prohibitive financial costs of training massive models increasingly restrict participation to a small number of well-resourced institutions [1]. A distinction must also be made between Green AI and *AI for Green*, where Green AI concerns reducing the ecological footprint of AI systems themselves, not applying AI to environmental problems [1, 4].

A core principle of the Green AI movement is evaluating environmental impacts across the entire lifecycle of an AI system, rather than focusing solely on the training phase [4, 5]. This full-lifecycle view encompasses data engineering, system integration, continuous retraining, and inference operations. Empirical work on data-centric Green AI illustrates the potential of this broader perspective: simple dataset preprocessing modifications, such as extracting smaller representative subsets or reducing feature counts, can decrease training energy consumption by up to 92 percent with negligible loss in accuracy [2].

To translate these principles into practice, researchers advocate for standardized reporting of computational work, energy consumption, and carbon emissions alongside accuracy results [1, 6]. Several tools have been developed to support this. The ML CO_2 Impact Calculator [7] and the experiment-impact-tracker [6] allow practitioners to estimate carbon emissions based on hardware configuration, execution time, and local grid carbon intensity. At the organizational level, the Green Software Foundation's Software Carbon Intensity (SCI) specification provides a standardized, consequential carbon accounting metric aimed at directly reducing AI carbon footprints rather than relying on market-based offsets [5, 8].

Realizing efficiency as a measurable criterion requires defining appropriate metrics. Floating-Point Operations (FLOPs) provide a hardware-independent estimate of computational work [1], but do not translate directly to energy or execution time, as these depend on hardware architecture, memory access patterns, and parallelization [6]. Training time is more intuitive but hardware-dependent and difficult to compare across environments. Energy consumption in kilowatt-hour (kWh) and CO_2-equivalent (CO_2eq) sidestep these limitations by capturing the actual physical and ecological footprint of a training run. Although tied to a specific hardware configuration and electricity grid, they provide directly comparable sustainability figures when experiments are conducted under controlled conditions [6, 8].

2.2 Energy and Carbon Emissions in Machine Learning

Operationalizing efficiency requires distinguishing two related but distinct quantities. Energy consumption is a hardware-level physical quantity, the product of power draw and time expressed in kWh, where the actual power draw depends on hardware utilization and thermal design rather than being a fixed property of the components [6]. Carbon emissions then translate that energy into an ecological footprint, measured in CO_2eq, by multiplying it with the *carbon intensity* of the local electricity grid [7]. Two identical training runs can therefore produce the same energy consumption yet vastly different emissions, depending solely on the fuel mix of the grid supplying them. In practice, empirical studies have shown that identical workloads can vary in their carbon footprint by a factor of 40 depending on geographic location [7], and can even fluctuate based on the time of day the models are trained [8].

2.2.1 From Energy to Carbon Emissions

To understand the ecological footprint of ML, it is first necessary to distinguish between power and energy. Power, measured in Watts (W) or Joules per second, represents the instantaneous rate at which electricity is drawn by a system. Energy, on the other hand, is the total quantity of electricity consumed over a specific duration, calculated as the product of power and time ($E = P \cdot t$), and is typically expressed in kWh or Joule [6].

Translating this measured energy consumption into a quantifiable environmental impact requires the concept of carbon intensity [7]. Carbon intensity represents the amount of greenhouse gas emissions produced per unit of electricity generated, expressed in grams of CO_2-equivalent (gCO_2eq) per kWh (gCO_2eq/kWh), and standardizes the global warming potential of various greenhouse gases into a single comparable value.

The operational carbon footprint of a ML model is derived by multiplying the total energy consumed (E, in kWh) by the carbon intensity of the local electricity grid (I), yielding $CO_2eq = E \cdot I$ [8]. Because different geographic regions rely on vastly different fuel mixes, from coal and natural gas to nuclear and renewables, carbon intensity varies drastically by location [6]. Empirical data illustrates this range clearly: training a model in Quebec, Canada, which is heavily reliant on hydroelectricity, faces a carbon intensity of approximately 20 gCO_2eq/kWh, whereas training the same model in a fossil-fuel-dependent region such as Iowa, USA, results in a carbon intensity of over 736 kilograms of CO_2-equivalent ($kgCO_2eq$)/kWh [7].

Carbon intensity is not merely a static geographic property but also a temporally variable one [8]. The emissions factor of a grid can fluctuate significantly throughout the day based on the real-time availability of renewable sources, such as solar during daylight hours or wind at night. Consequently, accurately evaluating the ecological footprint of a ML model requires integrating both the spatial location and the temporal characteristics of the local energy grid into the final CO_2eq conversion.

2.2.2 Computing Hardware and Power Consumption

The execution of ML workloads relies on a combination of hardware components, primarily the Central Processing Unit (CPU), GPU, and Dynamic Random-Access Memory (DRAM) [6]. Understanding the architectural differences between these components is essential for reasoning about their respective energy profiles and the efficiency trade-offs examined in this thesis.

The CPU is a general-purpose processor designed for sequential, low-latency execution of complex, branching instruction streams. Its architecture dedicates a large share of its transistor budget to control logic, branch prediction, and deep cache hierarchies, typically comprising between 4 and 32 high-frequency cores. This design excels at tasks requiring complex decision logic but offers limited parallelism for the arithmetic-intensive operations that dominate Deep Learning (DL).

The GPU, by contrast, is a hardware accelerator originally designed for rendering graphics, a task that requires applying the same transformation to millions of pixels simultaneously. Its architecture instead consists of thousands of smaller, simpler processing cores organized into streaming multiprocessors, optimized for high arithmetic throughput and high memory bandwidth. This massively parallel design maps directly onto the matrix multiplications and element-wise operations that dominate both training and inference in deep learning, which is why GPUs have become the standard compute substrate for computationally intensive DL workloads [8].

This architectural advantage comes at a steep energetic cost. While CPUs typically draw between 10 W and 150 W under load, GPUs are significantly more power-hungry, often consuming between 250 W and 350 W [8]. As a result, the GPU typically accounts for the vast majority of total electrical energy consumed during training, often approaching three-quarters of the total draw [8]. The energy drawn by the CPU and DRAM nevertheless remains a non-negligible contribution that must be recorded for accurate sustainability accounting.

A frequently referenced hardware specification in this context is the Thermal Design Power (TDP). TDP defines the maximum heat generated by a component that its cooling system is designed to dissipate, serving as a theoretical upper bound on sustained power draw [9]. Some simplified methodologies estimate total energy consumption by multiplying a component's TDP by execution time, but this approach is fundamentally flawed [6]: actual power draw fluctuates continuously based on hardware utilization, varying greatly between idle states and peak load. Empirical studies demonstrate that TDP-based estimation leads to severe over- or underestimations of the true energy footprint [6]. Reliable sustainability measurements therefore require direct real-time hardware readings rather than static TDP-based extrapolations.

2.2.3 Measurement Approaches

Accurately evaluating the energy consumption of ML models relies on a spectrum of methodologies, broadly categorized into physical measurements, estimation models, and on-chip sensors [9]. The baseline for ground-truth measurement is the External Power Meter (EPM), which tracks power directly at the wall outlet or motherboard. While EPMs capture the true consumption of the entire system, they are often costly to deploy at scale and cannot provide fine-grained decomposition to isolate the energy used by specific software processes or individual hardware components [9].

To achieve component-level granularity without external hardware, many approaches utilize estimation models. These models calculate energy based on abstract hardware specifications, such as multiplying execution time by the TDP or counting FLOPs, or by leveraging Performance Monitoring Counters (PMCs) to track utilization rates [9]. However, as established in Section 2.2.1, relying solely on abstract analytical models or TDP often leads to severe inaccuracies under dynamic workloads, failing to capture the true, fluctuating real-time power draw of the hardware [6].

Consequently, modern sustainability accounting strongly prefers direct measurements obtained from vendor-specific on-chip sensors [6]. For GPUs, the industry standard is the NVIDIA Management Library (NVML), which polls instantaneous power consumption during execution. For CPUs and DRAM, the standard interface is Running Average Power Limit (RAPL), provided by Intel and AMD, which accesses hardware performance counters to deliver accurate, fine-grained energy measurements directly from the CPU package [9].

Despite its high accuracy, a significant practical limitation of RAPL is its strict dependency on the underlying operating system [6]. RAPL interfaces are primarily supported on Linux environments and often require elevated administrator privileges to access the hardware counters securely. When direct sensor access is unavailable due to these constraints, software tracking tools often revert to problematic TDP-based estimation models as a fallback, which severely compromises measurement accuracy [9].

2.2.4 CodeCarbon

To practically implement the sustainability accounting principles advocated by the Green AI movement, researchers require standardized tools that bridge the gap between physical hardware metrics and environmental impacts. Initially, the ML community relied on a fragmented landscape of independent measurement approaches and preliminary software packages [10]. Early solutions largely consisted of web-based calculators, such as the ML CO_2 Impact calculator [7] and Green Algorithms [11], which required practitioners to manually input hardware specifications, execution times, and locations to estimate carbon footprints.

While these online calculators successfully raised awareness, they lacked real-time monitoring capabilities and had to rely on broad estimations rather than ground-truth physical power draw [10]. To address this, several independent Python packages were developed to track energy dynamically during model training by directly polling hardware sensors during execution, including the experiment-impact-tracker [6] and CarbonTracker [12]. However, the proliferation of tools with varying methodologies, carbon intensity data sources, and hardware support led to inconsistent reporting and discrepancies in carbon footprint measurements across the literature [10].

To unify these fragmented efforts into a reliable standard, the CodeCarbon initiative was established [10]. CodeCarbon is an open-source software library that evolved directly from previous tracking projects, serving as the official successor to the Energy Usage tool [9] and integrating the efforts of the ML CO_2 Impact project [10]. By providing a unified tracking interface, it enables practitioners to quantify the environmental cost of their algorithms and report these metrics transparently [13].

To evaluate a model's energy consumption, the framework leverages the direct measurement approaches introduced in Section 2.2.3. Specifically, it interfaces with vendor-specific on-chip sensors, such as NVML for GPUs and RAPL for Intel and AMD processors, to collect fine-grained power draw measurements directly from the underlying hardware [9]. By periodically polling these hardware counters during execution, the tool computes the total electrical energy consumed by the computing components in kWh.

Once the physical energy consumption is recorded, the framework translates it into greenhouse gas emissions using the conversion detailed in Section 2.2. The tool determines the carbon intensity of the local electricity grid based on the geographic location of the computing infrastructure, then multiplies the measured energy by this value to produce the final operational carbon footprint expressed in $kgCO_2eq$ [8]. By combining hardware-level energy tracking with grid-level carbon intensity data in a single standardized framework, CodeCarbon automates the translation from computational work to ecological impact [6].

2.3 Supervised Classification Algorithms

In ML, supervised classification is the task of inferring a predictive model from historical, annotated data in order to make accurate categorical predictions on new data [14]. Formally, given a training dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$ consisting of n examples, where each $x_i \in \mathcal{X}$ represents a vector of input features and $y_i \in \mathcal{Y}$ represents the corresponding ground-truth label, the objective is to learn a mapping function $f : \mathcal{X} \to \mathcal{Y}$ that reliably predicts the target output for any given input [15]. This reliance on an explicitly provided label space distinguishes supervised learning from unsupervised learning, where algorithms discover hidden structures in unannotated data, and from reinforcement learning, where an agent learns to make sequential decisions by maximizing a reward signal within an interactive environment.

The fundamental objective when learning f is achieving strong *generalization*: the model must perform reliably on new, unseen data drawn from the same underlying distribution rather than merely memorizing training inputs [16]. A central challenge during training is therefore finding the optimal balance to prevent *underfitting*, where the model is too simple to capture the true data patterns, and *overfitting*. Overfitting occurs when a model excessively adapts to the specific "sampling noise" present in the training data, fitting it so closely that its predictive performance on new, unseen data actively degrades [16, 17].

The structure of the label space $\mathcal{Y}$ determines the nature of the classification task. In binary classification, $\mathcal{Y} = \{0, 1\}$. When more than two categories are involved it becomes a multiclass problem with $\mathcal{Y} = \{1, \ldots, C\}$ [15]. Multiclass problems are typically handled via One-vs-Rest strategies, which decompose the problem into C binary classifiers, each distinguishing one class from all others, or by extending binary formulations with a Softmax output that produces a normalized probability distribution across all C classes.

To optimize f during training, a loss function $\mathcal{L}(y, f(x))$ quantifies the discrepancy between predicted outputs and true labels [17]. For classification, the standard objective is Cross-Entropy Loss. In the binary case this reduces to the negative binomial log-likelihood, while for multiclass problems it generalizes to Categorical Cross-Entropy, formalized in Equation (2.1).

$$\mathcal{L} = -\sum_{k=1}^{C} y_k \log\big(p_k(x)\big) \tag{2.1}$$

In Equation (2.1), y_k is the binary indicator for the true class k and $p_k(x)$ is the model's predicted probability for that class. Cross-Entropy is preferred over Mean Squared Error for classification because it produces steeper, more informative gradients when a confident prediction is wrong, and provides a direct probabilistic interpretation of the model output. This foundational loss objective drives the gradient-based optimization for both deep learning and advanced ensemble architectures discussed in the following subsections.

2.3.1 Logistic Regression

Logistic regression, formalized as a statistical classification method by Cox [18], is a parametric linear model that remains a standard baseline for binary classification tasks where interpretability and computational efficiency are paramount.

For a binary classification task ($\mathcal{Y} = \{0, 1\}$), logistic regression predicts the probability that an instance x belongs to the positive class. To ensure the output is bounded strictly between 0 and 1, the linear combination of input features and learned weights w with bias b is passed through the logistic sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$, yielding the prediction model in Equation (2.2).

$$P(y = 1 \mid x) = \sigma(w^\top x + b) = \frac{1}{1 + e^{-(w^\top x + b)}} \tag{2.2}$$

This functional form stems from modeling the log-odds (logit) as a linear function of the inputs, as expressed in Equation (2.3) [14].

$$\ln\left(\frac{P(y = 1 \mid x)}{P(y = 0 \mid x)}\right) = w^\top x + b \tag{2.3}$$

Parameters are estimated via Maximum Likelihood Estimation, which is equivalent to minimising the binary cross-entropy loss and requires iterative numerical solvers since no closed-form solution exists [14]. This study uses the Stochastic Average Gradient (SAG) solver [19], which is particularly well-suited for large datasets: rather than recomputing the full gradient over all training samples at each step, SAG maintains a running average of recently computed individual gradients, substantially reducing both memory usage and required training time. For multiclass problems, Multinomial Logistic Regression with a Softmax function is applied [14].

A key advantage of logistic regression is its interpretability: each feature weight directly represents its directional influence on the log-odds of the prediction. It also trains quickly and is robust to overfitting on simple datasets. Its fundamental limitation is linearity: without manual feature engineering, the model cannot capture non-linear patterns or feature interactions, making it considerably less expressive than the tree-based and deep learning architectures described in the following subsections [14].

2.3.2 Decision Trees

Decision trees represent a fundamental family of non-parametric ML algorithms that approach classification through a recursive binary partitioning strategy [20]. In this architecture, learning proceeds top-down from a root node through internal nodes, each of which applies a threshold test of the form $x_j \leq t$ on a single input feature j with a learned constant threshold t. This threshold determines which branch a given data point follows: samples satisfying the condition are routed left, all others right. The tree routes each sample through a sequence of such decisions until it reaches a terminal leaf node. In essence, the tree divides the input feature space into distinct regions, with each region corresponding to a leaf. All data points that reach the same leaf receive the same class prediction: the majority class among the training samples that were routed there [17].

The induction of a decision tree relies on a greedy split-finding algorithm, which makes locally optimal decisions at each node without revisiting or reconsidering earlier splits. At each internal node, the algorithm evaluates candidate split points across all available features to find the partition that minimizes impurity in the resulting child nodes. The Gini impurity criterion is used, defined as $G = \sum_{k=1}^{C} \hat{p}_k(1 - \hat{p}_k)$, which measures the probability of misclassifying a randomly drawn sample [17]. This is the default criterion in scikit-learn [19].

The structural design of decision trees imparts specific inductive biases that make them particularly well-suited for tabular data, as documented by Grinsztajn et al. [21]. Because they naturally learn piecewise constant functions, decision trees can fit highly irregular, non-smooth patterns without requiring complex mathematical transformations. Furthermore, decision trees operate strictly through axis-aligned splits and are therefore not rotationally invariant, meaning they cannot be altered by rotating the feature space, as splits are always parallel to a single coordinate axis. This is highly advantageous for tabular data, as it prevents the algorithm from creating arbitrary linear combinations of variables, thereby preserving the independent, heterogeneous meaning of the original features [21]. Consequently, trees remain robust even when datasets contain uninformative features [22].

Despite these representational strengths, single decision trees suffer from significant theoretical limitations. When grown deeply, a tree becomes highly susceptible to memorizing sampling noise, leading to high variance, where predictions fluctuate unpredictably across different training sets and severe overfitting, where the model fits training noise rather than underlying patterns, causing poor performance on new data [17]. Structural constraints such as a maximum depth or minimum number of samples per leaf partially mitigate this, but at the cost of increased bias, where the model becomes too restrictive and cannot capture underlying patterns and the need for careful tuning. To more systematically address these vulnerabilities while preserving the tree's ability to model irregular tabular data, the ML community shifted toward combining multiple tree predictors into ensemble methods [22]. This transition from single, unstable base learners to aggregated models forms the foundational motivation for the Random Forest and Gradient Boosted Decision Trees (GBDT) described in the following sections.

2.3.3 Random Forest

As established in the previous section, the inherent instability, high variance, and overfitting risk of single decision trees serve as the primary motivation for aggregating multiple individual trees into powerful ensemble methods [22]. Random Forests, formally introduced by Leo Breiman [22], overcome these limitations by combining a large collection of tree predictors, where each tree depends on the values of a random vector sampled independently and with the same distribution for all trees in the forest.

Training Procedure. The algorithm constructs this ensemble using a two-fold randomization strategy. First, it employs a technique known as *bagging* (bootstrap aggregating), originally proposed by Breiman [23], where each tree is grown on a new training set drawn at random with replacement from the original data. Random Forests extend this by additionally randomizing feature selection during the tree-growing phase: instead of evaluating all available features to determine the optimal split at each internal node, the algorithm selects the best split from a randomly chosen, restricted subset of features [22]. The constituent trees are typically grown to their maximum depth and are not pruned. To classify a new input vector, each tree in the ensemble casts a unit vote, and the forest outputs the majority class.

Theoretical Guarantees. Breiman theoretically proved that the predictive accuracy of a Random Forest is governed by the interplay of two key parameters: the *strength* of the individual tree classifiers and the *correlation* between them [22]. Correlation here means the degree to which trees make similar predictions on the same data points. Decorrelated trees, by contrast, make diverse predictions and thus commit different kinds of errors. An optimal forest consists of individually strong trees that produce decorrelated predictions. By combining bagging with random feature selection, the algorithm deliberately decreases the correlation between individual trees, which substantially reduces the variance of the overall ensemble without substantially compromising the predictive strength of the base learners. Building on this analysis, Breiman further demonstrated via the Strong Law of Large Numbers that as the number of trees grows, the generalization error almost surely converges to a limiting value, guaranteeing that the ensemble is inherently robust against overfitting with respect to the number of trees. While this convergence argument in the original paper was somewhat informal, formal consistency for a close variant of Breiman's algorithm under standard assumptions was later established by Scornet, Biau, and Vert [24].

Parallelism and Out-of-Bag Estimation. A crucial computational advantage of this architecture, particularly relevant for evaluating the efficiency of ML algorithms, is its inherent parallelizability [25]. Because the individual trees are entirely independent of one another, the training process can be distributed independently across multiple processing cores, allowing Random Forests to scale efficiently on multi-core CPU architectures [22, 25]. GPU-accelerated implementations also exist (e.g., RAPIDS cuML, Intel Extension for Scikit-learn), though GPU support remains implementation-dependent rather than a universal property of the algorithm.[1] Additionally, the bagging mechanism naturally provides an internal validation mechanism: since each tree is trained on a bootstrap sample (approximately 63% of the data), the remaining samples, termed *Out-of-Bag (OOB)* samples, form the complement set. For each training sample, the forest can evaluate it using only the subset of trees that did not include it in their training bootstrap, yielding an unbiased generalization error estimate without requiring a separate validation split [22]. In scikit-learn, this mechanism is optional and requires setting the `oob_score=True` parameter at initialization.

Scalability. Despite this parallelism, Random Forests carry a significant computational burden on large datasets. Each tree must be grown independently on a bootstrap sample of the full training set, and because the trees are deep and unpruned, meaning they are grown to full depth without being simplified or trimmed back, the cost per tree grows with both the number of samples and the number of features evaluated at each split [25].

[1] GPU acceleration for Random Forests is not universally available across all libraries. Support depends on the specific implementation: scikit-learn runs on CPUs only, while specialized libraries like RAPIDS cuML provide GPU-accelerated versions.

This cost is then multiplied across all M trees in the ensemble. As the dataset size grows into the millions of samples, the aggregate training cost becomes substantial, limiting the practical applicability of Random Forests on very large datasets without subsampling or other approximations.

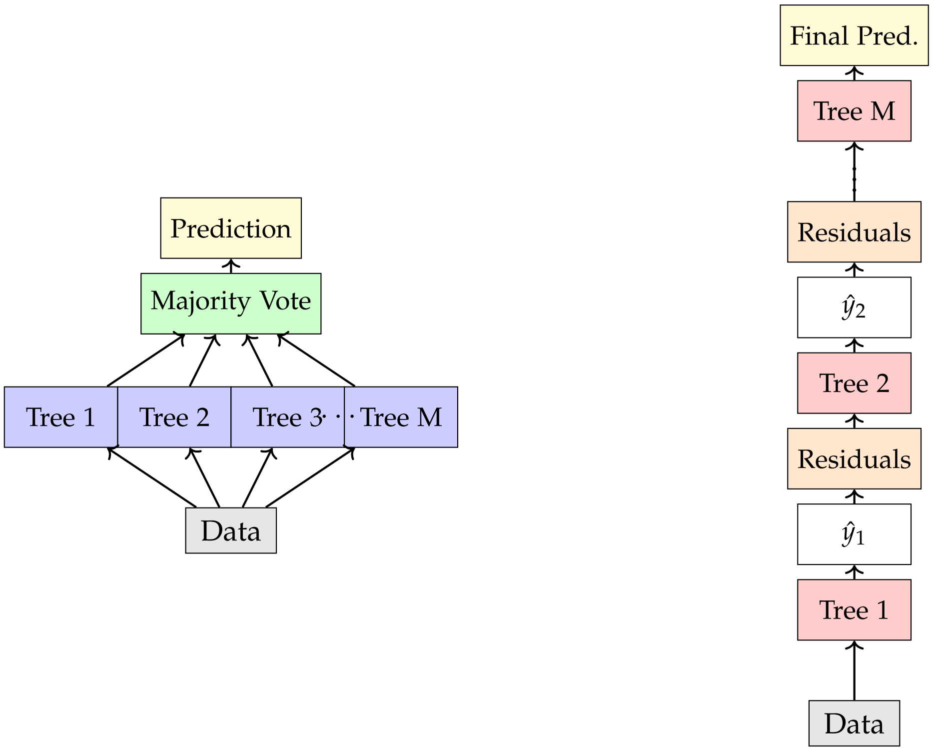
2.3.4 Gradient Boosting and XGBoost

Random Forests and Gradient Boosting represent fundamentally different ensemble strategies. Random Forests combine many independent strong learners: deep, complex trees trained in parallel on bootstrap samples, where the ensemble strength comes from diversity and aggregation. In contrast, GBDT operate sequentially, constructing an ensemble of weak learners (shallow trees) where each successive tree is trained to correct the errors of the previous iterations, iteratively improving overall performance [17]. Introduced by Friedman [17], the gradient boosting machine frames this sequential training as a form of gradient descent in function space. Instead of adjusting numerical parameters, the algorithm iteratively adds a new decision tree to the ensemble that is fitted to the negative gradient of the loss function with respect to the predictions of the previous iteration. This allows the model to minimize arbitrary differentiable loss functions, making it a highly flexible and powerful approach for both regression and classification tasks. Throughout this framework, $\hat{y}_i$ denotes the model's predicted output for training instance i, where the hat symbol ($\hat{\cdot}$) is standard notation distinguishing a model's prediction from the corresponding true label y_i. After each boosting round, a new tree is added whose outputs correct the residual errors of the current ensemble, progressively updating $\hat{y}_i$ toward y_i. Figure 2.1 illustrates the architectural contrast between the two strategies: the left panel shows Random Forest's parallel structure, in which all trees are trained independently on bootstrap samples and their predictions are aggregated by majority vote, while the right panel shows GBDT's sequential pipeline, in which each successive tree is fitted to the errors of the previous ensemble and the predictions are summed to progressively reduce the residual.

Regularized Objective. XGBoost (Extreme Gradient Boosting), introduced by Chen and Guestrin [26], significantly extends this foundational concept to improve both predictive accuracy and computational scalability. To combat the overfitting risks inherent in boosted ensembles (where each tree iteratively fits residual errors of its predecessors, risking the memorization of training noise), XGBoost modifies the training objective by augmenting the task-specific loss function with an explicit regularization term [26].

$$\mathcal{L} = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \Omega(f), \quad \Omega(f) = \gamma T + \tfrac{1}{2}\lambda \|w\|^2 \tag{2.4}$$

In Equation (2.4), n is the number of training instances, y_i the true label and $\hat{y}_i$ the predicted value for instance i, and $l(\cdot)$ is a differentiable task-specific loss function. For binary classification, for example, l takes the form of the logistic loss $l(y_i, \hat{y}_i) = y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i)$. The regularization term $\Omega(f)$ penalizes model complexity directly: T is the number of leaf nodes of tree f, $w \in \mathbb{R}^T$ the vector of leaf output weights, $\gamma \geq 0$ a



Random Forest (Breadth)
Many strong, independent learners

Gradient Boosting (Depth)
Sequential weak learners, iterative correction

Figure 2.1: Random Forest vs Gradient Boosting: Parallel (breadth) vs Sequential (depth) ensemble strategies.

penalty controlling tree size, and $\lambda \geq 0$ the L2 regularization coefficient that shrinks the leaf weights toward zero to reduce variance. Furthermore, while standard gradient boosting relies only on first-order gradients, XGBoost utilizes a second-order Taylor expansion of the loss function [26]. By calculating both the gradient and the Hessian (second derivative) for each training instance, the algorithm can optimize the tree structure more directly and rapidly converge on the optimal split points.

Shrinkage and Column Subsampling. XGBoost also implements *shrinkage* and *column subsampling* to further prevent overfitting [26]. Shrinkage scales each new tree's contribution by a learning rate to slow convergence, which is enabled by default in the XGBoost package via `eta=0.3`. Column subsampling introduces feature randomization analogous to Random Forest, but whereas Random Forest draws a fresh random feature subset at every individual split node, XGBoost's `colsample_bytree` selects a random subset of features once per tree, so that all nodes within that tree compete only over the same fixed subset, though this mechanism must be explicitly configured, as the default value of `colsample_bytree=1.0` retains all features and effectively disables it.

GPU Acceleration. Despite these algorithmic optimizations, the exact greedy split-finding procedure shared by all tree-based methods requires sorting continuous feature values across all candidates, a bottleneck that becomes prohibitive at the scale of millions of instances [26].

To accelerate this, XGBoost was extended to natively leverage GPUs [27]. Formalized by Mitchell and Frank, the GPU-accelerated tree construction algorithm exploits the massive parallelism and high memory bandwidth of modern GPU architectures. Instead of relying on exact sorting, the *histogram method* bins continuous feature values into discrete buckets and aggregates gradient statistics per bin. By evaluating split points using highly optimized parallel primitives over these histograms, the entire decision tree can be constructed directly in device memory [27]. This histogram-based split-finding approach allows XGBoost to scale efficiently to millions of instances, achieving significant speedups over multi-core CPU implementations while fully accommodating the algorithm's second-order mathematical formulations. In their evaluation, Mitchell and Frank demonstrated the memory efficiency of this approach by processing a large-scale dataset of 11 million training instances and 28 features entirely within the memory of a single GPU, confirming its practical viability for large-scale classification tasks [27]. This speedup, however, does not constitute a genuine efficiency gain in the Green AI sense: as Schwartz et al. [1] argue, hardware acceleration merely redistributes computational cost from time to power draw, and true efficiency must come from algorithmic improvements that reduce the total work performed rather than from procuring faster hardware [1].

2.3.5 Multilayer Perceptron

The Multilayer Perceptron (MLP) is a feedforward neural network architecture composed of sequential layers of neurons that apply linear transformations followed by non-linear activation functions. The Universal Approximation Theorem [28] provides the mathematical justification for this architecture, guaranteeing that a single hidden layer is sufficient to approximate any continuous function arbitrarily closely, given enough neurons in that layer. However, unlocking this immense representational capacity requires scaling the network's depth and width, which inherently increases both the model's susceptibility to overfitting and the computational burden of training.

Backpropagation and Optimization. Unlike decision trees, MLPs are optimized continuously using *backpropagation* [29], which propagates the gradient of the loss function backwards through the network layer by layer via the chain rule to compute each parameter's contribution to the error, and Stochastic Gradient Descent (SGD) or its variants to minimize a predefined loss function. In practice, SGD is applied in its mini-batch form, where the gradient is computed not over the full dataset nor a single sample, but over small random subsets of the training data, trading gradient accuracy for computational efficiency and introducing beneficial noise that helps escape local minima. To accelerate this optimization process in modern applications, the Adaptive Moment Estimation (Adam) optimizer is frequently deployed. Adam is a computationally efficient, first-order gradient-based optimization algorithm [30]. It computes individual adaptive learning rates for different parameters from estimates of the first and second moments of the gradients. By maintaining exponential moving averages of the gradient and the squared gradient, and applying an initialization bias correction to counteract initial estimates being biased toward zero, Adam effectively adapts to the geometry of the objective function.

Activation Functions. Historically, training deep architectures was severely hindered by the *vanishing gradient problem*. When utilizing traditional saturating non-linearities like the logistic sigmoid or hyperbolic tangent, the gradients of saturated neurons rapidly approach zero during backpropagation, making it nearly impossible for earlier layers to update effectively [31]. The widespread adoption of the Rectified Linear Unit (ReLU) largely resolved this bottleneck. By computing $y = \max(0, x)$, ReLU prevents gradients from vanishing for positive inputs and yields highly sparse representations [32]. Furthermore, its gradient is exactly 1 for positive inputs and 0 otherwise, making it highly efficient for training deep networks.

Batch Normalization. As MLPs scale in depth, they introduce considerable internal instability during training. During gradient descent, the distribution of inputs to any given hidden layer constantly changes as the parameters of all preceding layers are updated, a phenomenon termed *internal covariate shift* [33]. This forces the optimizer to use excessively low learning rates, substantially slowing down convergence. Batch Normalization overcomes this by explicitly standardizing the intermediate activations across each mini-batch to maintain a stable mean of zero and variance of one, before applying a learned affine transformation with parameters γ and β [33]. By fixing these distributions, Batch Normalization allows for the use of substantially higher learning rates and makes backpropagation highly resilient to parameter scale, resulting in substantially faster training times.

Dropout. To prevent these deep networks from merely memorizing the sampling noise of the training data, regularization is required. *Dropout* provides a computationally elegant solution by randomly deactivating a fraction of hidden units during each forward pass [16]. This prevents neurons from developing complex co-adaptations where they solely rely on specific other units to correct their mistakes. At test time, scaling the outgoing weights seamlessly approximates an equally weighted geometric mean of all these shared-weight subnetworks, significantly reducing generalization error without the prohibitive cost of training multiple large models [16].

Early Stopping. Similarly, because an MLP will eventually begin to overfit if training continues without constraint, *Early Stopping* is employed to monitor a separate validation set and halt the training process once the validation error stops decreasing [34]. Beyond acting as an implicit regularizer, this technique serves as a critical efficiency mechanism, saving significant computational time by avoiding redundant training epochs.

Automatic Mixed Precision. A significant hardware-aware optimization for training deep neural networks is Automatic Mixed Precision (AMP). Standard training typically relies on the 32-bit single-precision floating-point format (FP32) for all calculations. AMP dynamically transitions between FP32 and 16-bit half-precision (FP16 or bfloat16) formats during execution. Operations that are highly sensitive to numerical underflow, such as gradient accumulation or specific loss computations, are preserved in FP32 to maintain numerical stability. Simultaneously, computationally dense operations, such as matrix multiplications, are cast to lower precision. This selective reduction in precision effectively halves the memory bandwidth required for those operations and allows the model to leverage specialized GPU hardware, such as Tensor Cores, which are explicitly optimized for low-precision arithmetic. Consequently, AMP can substantially accelerate training times and reduce the overall energy footprint of DL architectures without causing a perceptible degradation in the final predictive accuracy.

2.3.6 Tree-based Models vs. Deep Learning on Tabular Data

While deep learning has enabled tremendous progress on text and image datasets, its superiority on tabular data remains unproven [21]. For problems with data stored in tabular formats consisting of heterogeneous features, ensembles of decision trees, particularly GBDTs and Random Forests, are consistently recognized as the leading tools for both practitioners and ML competitions [15]. Extensive recent benchmarking has confirmed that tree-based models remain the superior choice on medium-sized tabular datasets, frequently outperforming deep learning architectures even before accounting for their superior training speed. However, this performance gap narrows significantly as the dataset size increases. Tree ensembles dominate in low-data regimes, but DL models become increasingly competitive when the number of training instances scales into the millions [21].

The performance gap between neural networks and tree-based models stems from their fundamentally different inductive biases and data requirements [21]. Neural networks are intrinsically biased toward overly smooth, low-frequency solutions. Consequently, they struggle to learn the highly irregular and non-smooth target functions that frequently characterize tabular datasets. In contrast, models based on decision trees learn piecewise constant functions and do not exhibit this smoothing bias, making them naturally adept at fitting these irregular patterns [21]. Beyond these structural differences, neural networks also exhibit poor sample efficiency compared to tree-based ensembles: Neural Networks (NNs) generally require substantially more data to learn useful representations without overfitting, making them ill-suited for very small tabular datasets where algorithms like Random Forests can quickly capture the underlying structure [21].

Furthermore, tabular datasets typically contain a large proportion of uninformative features [21]. Standard neural networks, such as the MLP, are highly susceptible to these uninformative features, whereas tree-based models remain robust even when up to half of the dataset's features carry no information. Another critical distinction is rotation invariance. Neural networks are typically rotationally invariant, which degrades performance on tabular data because individual tabular features generally carry specific, distinct meanings. Applying rotational transformations mixes features with very different statistical properties, destroying the natural orientation of the data [21]. Conversely, the axis-aligned splits of decision trees preserve this information by avoiding arbitrary linear combinations of distinct features [21].

Despite these challenges, there has been significant research into designing deep learning architectures specifically tailored for tabular data [15]. Examples include differentiable tree ensembles and adaptations of the Transformer architecture, such as FT-Transformer, which applies self-attention mechanisms directly to feature embeddings. While these advanced models close the performance gap and achieve state-of-the-art results among deep networks, they still do not universally outperform GBDT implementations like XGBoost across all tasks, especially when hyperparameters are properly tuned [15]. Crucially, even when complex DL models manage to match or slightly exceed the predictive accuracy of tree-based ensembles on datasets of millions of instances, they demand substantially more computational resources and training time to achieve these results [21]. This fundamental trade-off between predictive gain and computational cost reinforces the need to evaluate models not solely on accuracy, but also on their overall resource and energy efficiency.

2.3.7 Algorithmic Suitability for CPU vs. GPU

In the context of Green AI, optimizing energy efficiency requires aligning the computational profile of a ML algorithm with the architectural strengths of the underlying hardware. While it is common practice to universally accelerate training using GPUs, this approach is not intrinsically energy-efficient for all algorithms. The fundamental divide in hardware suitability lies in the distinction between dense, regular parallelism and irregular, sequential branching.

Hardware platforms are designed with opposing optimization goals: GPUs are optimized for aggregate throughput on parallel tasks, whereas CPUs are optimized for low latency on sequential, single-threaded tasks [35]. GPUs achieve this through a *Single Instruction, Multiple Threads (SIMT)* architecture that executes thousands of lightweight threads concurrently to hide memory latency. This design heavily favors dense, regular computational tasks with uniform memory access patterns, most notably the large-scale matrix multiplications that form the computational core of NN training. When operations can be uniformly distributed across thousands of cores simultaneously, the GPU achieves maximum utilization.

Conversely, CPUs rely on large caches, branch prediction, and speculative execution to handle irregular control flows and sequential decision logic with minimal latency. Algorithms that depend on dynamic branching, non-contiguous memory access, or sequential data traversal perform poorly on GPUs: in a SIMT environment, divergent threads within a warp must serialize, negating the throughput advantage and leaving cores idle [27, 35]. For these workloads, the CPU is the more efficient hardware choice.

Despite the higher peak power draw of GPUs, their throughput on suitable workloads means they process substantially more training samples per joule than a CPU running the same task. A GPU can therefore deliver superior energy efficiency for well-matched workloads despite consuming more power in absolute terms. This advantage is not universal: at small dataset scales, the GPU's higher idle and transfer overhead can make CPU training the more energy-efficient option overall.

2.4 Dataset Scaling and Learning Curves

In predictive modeling, the relationship between the volume of available training data and the resulting classification performance is formally modeled using a learning curve [36]. Empirical learning curves consistently demonstrate that performance improves as more data becomes available, but the rate of improvement eventually diminishes, typically following an inverse power law $P(n) \propto n^{\alpha}$ with $\alpha \in (0, 1)$ [14, 36]. This creates a fundamental tension in the context of Green AI : while computational and energy costs scale roughly linearly with dataset size, performance yields only sub-linear growth, making it increasingly ecologically expensive to achieve stronger results simply by adding more data [1]. Achieving a marginal accuracy gain at the upper end of a learning curve demands a disproportionately large investment in additional data, time, and energy.

Conversely, when dataset sizes are heavily restricted, classifiers generally experience degraded performance because smaller datasets contain fewer representative examples, leaving the model unable to generalize underlying patterns [37]. The risk of overfitting also increases: with limited samples, a model may memorize training noise rather than learn true structure. Crucially, however, the overall performance of a classifier depends more heavily on how well the dataset represents the underlying data distribution than on its sheer size [37]. A small but highly representative dataset can yield better generalization than a large but biased one.

Different classes of ML algorithms exhibit different sensitivities to dataset scale. DL architectures are highly data-dependent: they must fit a large number of parameters via backpropagation, meaning more data directly enables better adjustment and generalization [37]. Tree-based models are also sensitive at the smaller end of the scale, as recursive partitioning means predictions at deeper nodes rest on increasingly small subsets of examples, making tree depth unstable when the initial dataset is limited [22, 37]. In contrast, simpler linear models serve as robust baselines: their constrained functional form imparts a higher model bias but a much lower variance, making them far less prone to overfitting on limited datasets [14].

A complex model's absolute accuracy must therefore be weighed against its computational footprint and the diminishing returns of the learning curve. This trade-off is especially pronounced in a Green AI context, where the ecological cost of scaling up model complexity or dataset size may exceed the marginal accuracy gain.

2.5 Evaluation Metrics

Assessing the ecological efficiency of a ML algorithm requires more than measuring its predictive performance in isolation. An algorithm that achieves high accuracy at disproportionate computational cost is not unconditionally preferable to one that is slightly less accurate but significantly more resource-efficient [1]. This section therefore introduces the theoretical basis for three families of metrics. The first, classification quality, measures how accurately a model predicts class labels. The second, resource consumption, quantifies the energy expenditure and associated carbon emissions of training. The third, composite efficiency, integrates both dimensions into a single score that captures the trade-off between predictive performance and ecological cost. The specific metrics and their experimental implementation are described in Section 4.4.

Confusion Matrix. At the core of evaluating classification quality is the *confusion matrix*, which cross-tabulates a model's predictions against the true labels. For a binary classification problem, this matrix consists of four fundamental components: True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN). This concept extends naturally to multiclass problems by tracking predictions across a $C \times C$ grid, forming the basis for deriving all subsequent performance metrics [14].

Table 2.1: Binary confusion matrix with the four fundamental prediction outcomes.

	Predicted Positive	**Predicted Negative**
Actual Positive	True Positive	False Negative
Actual Negative	False Positive	True Negative

Accuracy. The most intuitive of these metrics is accuracy, defined as the overall proportion of correct predictions: Accuracy (ACC) $= \frac{TP+TN}{N}$, where TP and TN are the correctly classified instances of each class, and N is the total number of instances. While simple to interpret, accuracy exhibits a critical vulnerability when applied to datasets characterized by an uneven distribution of labels, known as *class imbalance*. For example, in a dataset where 99% of the instances belong to a single majority class, a trivial classifier that ignores all input features and simply predicts the majority class every time will achieve 99% accuracy. Despite this high score, the model learns no underlying patterns and fails completely at identifying the minority class.

Precision, Recall, and F1-Score. This limitation motivates the use of more robust metrics, specifically precision and recall. Precision ($\frac{TP}{TP+FP}$) measures the proportion of positive predictions that are actually correct, while recall ($\frac{TP}{TP+FN}$) measures the proportion of actual positive instances successfully identified. Unlike accuracy, precision and recall evaluate performance entirely independently of true negatives, making them highly resilient to datasets where a large, heterogeneous majority class dominates the negative counts [38]. Because improving precision often comes at the expense of recall and vice versa, the F1-score combines them into a single metric calculated as their harmonic mean, formalized in Equation (2.5).

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{2.5}$$

The harmonic mean is explicitly chosen over the arithmetic mean because it heavily penalizes extreme values. A model that achieves near-perfect recall but abysmal precision will yield a low F1-score, forcing the classifier to balance both dimensions effectively. Consequently, the F1-score provides a more encompassing summary of overall model performance and overcomes the representation problems caused by imbalanced data distributions.

F1 Aggregation Strategies. When dealing with multiclass or imbalanced datasets, the F1-score can be aggregated in several ways. Macro F1 calculates the metric independently for each class and averages them, treating all classes equally regardless of their size [38]. Micro F1 aggregates the global contributions of TP, FP, and FN across all classes before calculating the score, an approach that inherently favors larger classes [38]. Weighted F1-Score calculates the score for each class independently but averages them using the true support of each class as a weight.

Among these aggregation strategies, Weighted F1-Score is particularly well-suited to imbalanced settings. By weighting each class's score according to its true support, it yields a more representative picture of overall model performance than either Macro F1, which gives disproportionate influence to rare classes, or simple accuracy, which is entirely dominated by the majority class.

Resource Metrics. Measuring classification quality, however, captures only one dimension of model evaluation. Evaluating models under the Green AI paradigm requires moving beyond pure predictive quality to measure the physical and environmental costs of achieving that performance [1]. The primary resource metrics tracked include training time (measured in seconds), energy consumption (measured in kWh), operational carbon emissions (measured in $kgCO_2eq$), and inference latency (seconds per sample). The foundational definitions and physical measurement approaches for these quantities are established in Section 2.2.

Composite Efficiency Metrics. To evaluate ML models across both dimensions at once, researchers increasingly employ composite efficiency metrics that weigh predictive performance against these resource expenditures in a single score. The general methodological motivation behind these composite metrics is to provide a unified heuristic for identifying algorithms that offer the optimal trade-off between classification quality and ecological impact [1]. In the literature, such approaches include the Energy-Efficiency Ratio, Performance-per-Watt [39], and FLOPs-per-Sample (or per-inference) [1].

However, composite metrics possess inherent theoretical weaknesses. First, the resource-based components of these metrics, such as energy consumed or execution time, are intrinsically hardware-dependent. A composite score calculated in one environment is tied to the specific computational architecture and local energy grid, meaning it cannot be universally generalized or directly compared across different settings [1, 8]. Second, the weighting between performance and resource cost is highly scenario-dependent. Determining how much a marginal gain in predictive accuracy is worth compared to the energy required to achieve it remains an open question that depends heavily on the specific priorities or economic utility of the deployment scenario [6].

2.6 Cross-Validation

The simplest approach to evaluating a model's generalization capability is the *hold-out method*, where the available data is partitioned into a single training and test split (for example, an

80/20 ratio). While computationally efficient, this approach is highly sensitive to the specific data partition and exhibits high variance in its performance estimate, particularly when the dataset is small [14]. To mitigate this instability, K-fold Cross-Validation (CV) is widely employed as a more rigorous evaluation technique [14].

The algorithm partitions the dataset into K roughly equal-sized folds. The model is trained K times. In each iteration, $K-1$ folds are used to fit the model, while the single held-out fold is used for testing [14]. The CV estimator is formalized in Equation (2.6).

$$\mathrm{CV}(K) = \frac{1}{K}\sum_{k=1}^{K} L\big(y_k, \hat{f}^{-k}(x_k)\big) \tag{2.6}$$

In Equation (2.6), L is the chosen evaluation metric, y_k are the true labels of fold k, x_k are the corresponding input features of fold k, and $\hat{f}^{-k}$ denotes the model trained on all folds except the k-th [14]. The standard deviation of L across all K folds provides a quantitative measure of the model's stability across different data samples.

Selecting the number of folds K introduces a fundamental bias-variance trade-off. A small value of K is computationally less expensive but yields a higher estimation bias, as the model is trained on a significantly smaller fraction of the original data [14]. Conversely, setting $K = N$, known as Leave-One-Out Cross-Validation (LOOCV), trains the model N times using $N-1$ samples for each iteration. While LOOCV produces a nearly unbiased estimate of the test error, it is computationally prohibitive for large datasets and can exhibit high variance. Because the N training sets differ by only a single sample, the resulting estimators are highly correlated. Unlike independent estimators, whose averaged variance shrinks by a factor of $\frac{1}{N}$, the variance of correlated estimates does not diminish at this rate, so the averaging step of LOOCV provides little variance reduction [14]. To balance these competing factors, $K = 5$ and $K = 10$ are established as the standard recommendations in statistical learning, offering an optimal compromise between computational feasibility, low bias, and acceptable variance [14]. The specific justification for $K = 5$ is detailed in Section 4.5.

A critical theoretical distinction arises when CV is employed not only for performance evaluation but also for HPO. If the identical CV folds are used to both select the optimal hyperparameters and estimate the final generalization error, the resulting performance metric exhibits an optimistic bias. This occurs because the hyperparameters are explicitly chosen to maximize performance on these specific validation folds, leading to a form of indirect data leakage where the configuration implicitly overfits the validation sets. The magnitude of this bias is governed by two factors. First, it is inversely related to the size of the validation set: a small held-out fold introduces high variance in the model selection criterion, giving the optimizer opportunity to exploit the statistical peculiarities of that specific sample rather than genuine generalizable signal [40]. Increasing the validation set size directly reduces this variance, causing the selected hyperparameters to cluster more tightly around the true optimum [40]. Second, the bias grows with the number of hyperparameters to be tuned, as a larger search space provides more degrees of freedom through which the optimizer can inadvertently fit validation noise rather than the underlying generalisation surface [40]. When the research objective is to obtain a strictly unbiased estimate of absolute generalization error, nested CV is the appropriate remedy: an inner CV loop evaluates hyperparameter candidates, while a completely independent outer loop estimates the true generalization error of the selected configuration. However, the necessity of nested CV depends on the research objective. In comparative benchmarking studies, where the primary goal is to rank models relative to one another rather than to produce deployment-ready error estimates, the single-loop approach is widely adopted [e.g., 15, 21]. When all competing models are tuned and evaluated under an identical CV procedure, the optimistic bias introduced by HPOs is systematic and affects every model uniformly, preserving the integrity of relative performance rankings even if the absolute values are slightly inflated. Beyond this, nested CV imposes a steep computational penalty, requiring $K_{\text{inner}} \times K_{\text{outer}}$ model training cycles per configuration, a cost that rapidly becomes prohibitive for large datasets and, in a Green AI context, constitutes a non-trivial ecological overhead in its own right.

Finally, when evaluating datasets characterized by class imbalance, standard random partitioning may result in folds that do not accurately represent the underlying target distribution. To address this, *stratified CV*, a variant that ensures the original class ratio is preserved within every fold, is applied. This is directly consistent with the choice of Weighted F1-Score (WF1) as the primary evaluation metric: the same class imbalance that motivates a support-weighted metric must also be faithfully reproduced in each fold's test set, ensuring the evaluation procedure and the metric it produces remain coherent.

2.7 Hyperparameter Optimization

ML models are governed by two distinct categories of configurable quantities. Parameters, such as the weights of a NN or the split thresholds of a decision tree, are internal to the model and are learned directly from training data by minimizing a loss function [41]. Hyperparameters, by contrast, are external configuration choices that must be fixed prior to training: examples include the learning rate of an optimizer, the maximum depth of a tree, or the number of estimators in an ensemble [41]. Because hyperparameters govern the model's capacity, its regularization strength, which controls the degree to which overfitting is penalized, and its optimization dynamics, which determine how the learning process converges toward a solution, their values have a direct and often decisive impact on generalization performance [41, 42].

The goal of HPOs is to identify the configuration from a predefined search space Λ that minimizes the expected generalization error, where Λ is the Cartesian product of the individual value ranges of each hyperparameter, as shown in Equation (2.7), and the error is estimated via CV [42]

$$\Lambda = \Lambda_1 \times \Lambda_2 \times \cdots \times \Lambda_d \tag{2.7}$$

where Λ_i denotes the value range of the i-th hyperparameter and d is the total number of hyperparameters. As the number of hyperparameters grows, exhaustive evaluation of Λ becomes exponentially expensive [41, 42]. This combinatorial explosion motivates the development of more efficient search strategies, which are described in the following subsections.

2.7.1 Grid Search and Random Search

The most traditional approach to hyperparameter tuning is Grid Search, an exhaustive strategy that evaluates the Cartesian product of a user-specified, finite set of values for each parameter [41]. While conceptually simple, Grid Search suffers severely from the *curse of dimensionality* [41, 42]. As the number of hyperparameters increases, the computational complexity grows exponentially. For instance, optimizing five hyperparameters with ten candidate values each across three models and three datasets would require an exhaustive evaluation of 900,000 distinct configurations. This combinatorial explosion renders Grid Search computationally prohibitive for complex ML architectures with limited budgets [41].

To overcome the inefficiency of exhaustive evaluation, Random Search was proposed as a highly effective alternative [42]. Instead of testing predetermined values on a rigid grid, Random Search selects a predefined number of parameter combinations randomly from the specified search space distributions [41]. Empirical and theoretical analyses demonstrate that Random Search is significantly more efficient than Grid Search in high-dimensional spaces [42]. The key insight is that the mapping from hyperparameter configurations to model performance, the so-called response surface, behaves as if it only has a small number of relevant dimensions, even when many hyperparameters are nominally present. In practice, only a small subset of hyperparameters substantially impacts the generalization error, while the others leave the response surface nearly flat and can be varied freely without consequence [42]. Because Grid Search forces allocations across all dimensions equally, it wastes the vast majority of its computational budget exploring irrelevant parameters. Random Search, by contrast, tests a unique value for every parameter in every trial, allowing it to thoroughly explore the relevant subspaces with far fewer evaluations [42].

Despite their differences in candidate selection, both Grid Search and Random Search share a fundamental methodological limitation: they are entirely uninformed algorithms [41]. Every trial in their search process is independent, meaning neither method utilizes the performance results of previously evaluated configurations to influence the selection of future points [41]. Consequently, they inevitably waste massive amounts of time and computing resources evaluating poorly performing areas of the configuration space without learning from past failures [41]. This inherent inefficiency establishes the primary motivation for adopting sequential, history-dependent strategies such as Bayesian Optimization (BO).

2.7.2 Bayesian Optimization

To overcome the limitations of uninformed search strategies, BO offers a sequential, model-based approach that utilizes the history of previously evaluated configurations to intelligently determine the next hyperparameter values to test [41]. BO operates using two primary components: a probabilistic *surrogate model* that approximates the true, computationally expensive objective function, and an *acquisition function*, such as Expected Improvement (EI), which selects the optimal next point for evaluation [41, 43]. The acquisition function explicitly balances the fundamental optimization trade-off between *exploration* (sampling from under-explored regions of the parameter space where uncertainty is high) and *exploitation* (sampling from regions already known to yield promising results) [41].

While traditional BO relies on Gaussian Processs (GPs) as surrogate models to directly estimate the conditional probability $p(y|x)$ of the objective value y given the hyperparameters x, the Tree-structured Parzen Estimator (TPE) takes a fundamentally different approach [43]. Instead of asking "given these hyperparameters, what performance can I expect?", TPE inverts the question: "given that a trial turned out well or poorly, what kinds of hyperparameter values were likely to produce it?" [43]. Formally, this corresponds to modeling $p(x|y)$, the distribution of hyperparameter configurations x conditioned on the observed performance y, rather than $p(y|x)$ directly. TPE implements this by splitting all past trials at a performance threshold γ into two groups and fitting a separate density to each: $\ell(x)$ captures the distribution of configurations that performed well, and $g(x)$ captures those that performed poorly [41, 43]. The next candidate is then chosen to maximize the ratio $\ell(x)/g(x)$, favouring configurations that are likely under the good density and unlikely under the poor one [43].

This inverse modeling formulation grants TPE a significant advantage in complex configuration spaces. Because the Parzen estimators are organized in a tree structure, TPE naturally handles conditional hyperparameters, that is, parameters that only exist or only matter depending on the value of another parameter [41]. For example, a dropout rate is only relevant if dropout is enabled at all. If it is disabled, the dropout rate has no effect on performance regardless of its value. A standard GP treats all hyperparameters as independent continuous dimensions and cannot represent such if-then relationships. TPE's tree structure, by contrast, can branch on a parent hyperparameter's value and only then consider its dependent children, faithfully reflecting the actual structure of the search space [41, 43].

Furthermore, TPE offers a substantial computational advantage over both GPs and exhaustive search methods. While GP-based models scale cubically with the number of past observations $\mathcal{O}(n^3)$, making them prohibitively slow as the search history grows, TPE achieves a much lower time complexity of $\mathcal{O}(n \log n)$ [41]. By sequentially directing the search only toward the most promising regions and maintaining low computational overhead per trial, sequential strategies like TPE bypass the exponential combinatorial explosion that plagues Grid Search, enabling the efficient optimization of complex ML architectures with many hyperparameters [41].

Table 2.2: Comparison of HPO strategies (n: trials, d: hyperparameters).

Method	**Strategy**	**History**	**Complexity**
Grid Search	Exhaustive	No	$\mathcal{O}(n^d)$
Random Search	Random sampling	No	$\mathcal{O}(n)$
BO (GP)	Sequential, model-based	Yes	$\mathcal{O}(n^3)$
TPE (Optuna)	Sequential, model-based	Yes	$\mathcal{O}(n \log n)$

2.7.3 Optuna as a Practical Framework

Optuna is a modern HPO framework designed around a *define-by-run* Application Programming Interface (API) [44], which allows the search space to be constructed dynamically during runtime rather than requiring a static definition beforehand. Unlike traditional tools that require the search space to be statically defined prior to execution, Optuna constructs the hyperparameter search space dynamically within the objective function during runtime, accommodating complex dependencies without relying on rigid external configurations. Optuna utilizes the TPE as its default sampling algorithm, allowing for efficient exploration of these dynamically generated spaces. To ensure reproducibility, the stochasticity of the TPE sampler can be controlled by fixing a global random seed.

Beyond efficient sampling, Optuna accelerates the optimization process through automated trial pruning, which refers to the early termination of unpromising training runs based on intermediate results. This mechanism, formally referred to as *pruning*, periodically monitors the intermediate learning curves of iterative algorithms and preemptively terminates trials that fall below a predefined performance threshold, such as the median performance of previously completed trials. By allocating computational resources exclusively to the most promising configurations, pruning drastically reduces the overall runtime [44]. The specific details regarding whether and how pruning is applied are deferred to Section 5.4.

Ultimately, the adoption of an efficient framework like Optuna is not merely a mathematical convenience, but a fundamental Green AI strategy. The total computational and ecological cost of a ML result grows linearly with the number of hyperparameter trials executed [1]. By utilizing BO to strategically minimize the number of required evaluations, and employing pruning to terminate poor configurations early, the overall energy consumption and resulting carbon footprint of the entire model development process are significantly reduced.

2.8 Chapter Summary

This chapter established the theoretical foundations for the empirical study. The Green AI framework motivated the treatment of ecological cost as a first-class evaluation criterion alongside predictive accuracy. The energy and emissions section formalized how power draw translates into carbon footprint through grid carbon intensity, established why TDP-based estimation is unreliable, and introduced CodeCarbon as the standardized measurement tool. The algorithms section described the five classifiers under examination, their inductive biases on tabular data, and the conditions under which GPU acceleration is genuinely energy-efficient rather than merely faster. Dataset scaling and learning curves established the sub-linear relationship between data volume and performance gains, reinforcing the Green AI argument that ecological costs can outpace accuracy returns. The evaluation metrics section introduced WF1 as the primary classification quality metric and outlined the theoretical limitations of composite efficiency metrics. CV was formalized as the estimation procedure, with stratification linking directly to the metric choice under class imbalance.

Finally, HPO was covered from exhaustive search through BOs to Optuna, with HPO trial count identified as an ecological cost in its own right. These concepts collectively provide the vocabulary and methodology for the experimental design presented in Chapter 4.

3 Related Work

This chapter surveys empirical studies that have measured the energy consumption and carbon emissions of machine learning systems. The focus is on what prior work has concretely found, not on the underlying concepts, which are established in Chapter 2. Each section closes by identifying what prior studies leave unmeasured or unresolved, collectively motivating the specific design choices of this study.

3.1 Empirical Energy Benchmarks for Machine Learning

As empirical benchmarking has replaced theoretical complexity analysis as the primary mode of model evaluation, a body of literature has emerged that measures the actual energy and carbon cost of training machine learning systems. This literature has, however, grown unevenly: the studies that sparked the Green AI movement focused almost exclusively on large-scale deep learning, while the empirical benchmarks covering tabular data and classical algorithms ignored energy entirely.

The case for treating energy as a first-class evaluation metric was made most forcefully by early measurements of DL workloads. Strubell et al. [3] were among the first to directly measure hardware power interfaces during Natural Language Processing (NLP) model training, finding that a single large transformer architecture could emit as much carbon as five cars over their entire lifespans, and helping to establish the Green AI research agenda. Follow-up work examined the structural drivers of this footprint. Dodge et al. [8] expanded the empirical scope to 16 cloud computing regions, demonstrating that hardware choice and grid carbon intensity are as consequential as algorithmic decisions, with the same training run emitting between 7,000 and 26,000 grams of CO_2 depending on the data center. Desislavov et al. [39] extended this perspective to the inference phase, showing that the operational footprint of deployed models can rival the cost of training. A step toward more modest workloads was taken by Henderson et al. [6], whose experiment-impact-tracker framework demonstrated that everyday reinforcement learning experiments on standard hardware generate clearly measurable emissions, establishing ecological efficiency as a concern for ordinary research rather than exclusively for large-scale industrial training. Despite this broadening scope, the entire strand of Green AI benchmarking has remained focused on neural networks trained on unstructured data such as text and images.

The empirical literature on tabular data took a different path entirely. A series of large-scale benchmarks asked which model family generalizes best to structured, heterogeneous datasets and consistently found that classical algorithms hold their own against deep learning. Grinsztajn et al. [21] evaluated 45 tabular datasets and found tree-based models such as Random Forest and XGBoost to outperform deep neural networks, a result echoed by Gorishniy et al. [15] in a separate benchmark that reaffirmed the dominance of gradient boosting on this data type. Both studies, however, treated energy as entirely outside their scope: runtime was noted as a secondary metric, but physical power consumption and carbon emissions went unmeasured.

Only recently have a small number of studies begun to bridge these two strands. Rodriguez et al. [9] evaluated software-based energy trackers against physical hardware meters, measuring classical ML algorithms alongside DL models as part of their methodology. Verdecchia et al. [2] took a data-centric approach, empirically measuring how reducing data points and features affects the training energy of six AI algorithms and finding that data-centric dataset modifications can substantially reduce training energy, though the accuracy cost varies depending on whether data points or features are reduced. These contributions are important, but they remain limited to datasets of a few thousand instances and do not examine how energy and accuracy co-evolve across the orders-of-magnitude variation in dataset size that characterizes real-world applications.

The existing literature therefore reveals a clear gap at the intersection of two well-developed but isolated fields. Where Green AI benchmarks have established energy as a measurable, consequential quantity, they have done so exclusively for deep learning on unstructured data [3], [8]. Where experimental benchmarks have rigorously evaluated classical algorithms on tabular data, they have focused entirely on predictive accuracy [15], [21]. A systematic analysis of how energy consumption, carbon footprint, and predictive accuracy jointly scale with dataset size for classical classification algorithms remains absent. The present thesis addresses this gap by empirically investigating these efficiency-performance trade-offs across varying tabular dataset sizes, aiming to identify where simpler, less energy-intensive models become ecologically and practically advantageous.

3.2 Dataset Size Effects on Performance and Energy Consumption

The tension between data volume and model quality has a specific shape. Schwartz et al. [1] conceptualize this in their Equation of Red AI, which models the computational cost of a result as growing linearly with training dataset size. At the same time, accuracy does not grow linearly with data: empirical learning curves consistently follow an inverse power law, meaning that performance gains become increasingly marginal as the training set expands [36]. The practical consequence is that energy costs scale proportionally with data volume while accuracy returns diminish, making large datasets progressively less efficient investments. Koshute et al. [36] formalize the point where these curves cross as the Sufficient Training Set Size (STSS): the minimum volume beyond which additional data is ecologically unjustifiable. A complementary finding from Althnian et al. [37], based on medical classification tasks, adds that the key factor is not absolute size but representativeness, meaning that a small, well-sampled dataset can generalize as well as a much larger but biased one.

The energy implications of dataset reduction have been directly and concretely measured by Verdecchia et al. [2], who trained six AI algorithms while systematically reducing both data points and features in 10% increments. Two distinct patterns emerge from their results.

Reducing the number of features yields energy savings of up to 76% while largely preserving predictive performance, making it a near-cost-free optimization. Reducing the number of data points produces larger savings, up to 92.16% for Random Forest, but at a meaningful accuracy cost that must be weighed explicitly. The effect is also algorithm-specific: K-Nearest-Neighbor (KNN) achieves a maximum reduction of only 61.72% from fewer rows, while ensemble methods such as Random Forest are considerably more sensitive to training set size. Beyond dataset modifications, the study finds that switching to a less complex algorithm can alone yield energy savings of up to two orders of magnitude, establishing algorithm selection as one of the most powerful levers for ecological efficiency.

These findings rest, however, on a single dataset of 5,574 instances, run exclusively on a standard quad-core CPU and without GPU-accelerated or DL baselines. Whether the same energy-accuracy patterns hold at the scale of millions of instances, and whether they generalize across fundamentally different model families, remains empirically unexplored.

3.3 CPU vs. GPU Efficiency: Empirical Evidence

Whether GPU training is more energy-efficient than CPU training cannot be answered by measuring power draw alone. A GPU that draws five times the wattage of a CPU can still consume less total energy, if it finishes the same training task more than five times faster. The relevant quantity is energy, specifically power integrated over time, and how that relationship plays out depends heavily on the algorithm and the dataset.

The power side of this equation is well established empirically. Observed measurements from cloud training environments report GPU draws of 250 W to 350 W under load, compared to 10 W to 150 W for CPUs [8]. During Bidirectional Encoder Representations from Transformers (BERT) training, a workload representative of high-utilization deep learning, the GPU alone accounted for 74% of total system electricity. For neural network architectures of this type, the elevated power draw is more than offset by execution speed: the parallelism of deep learning maps efficiently onto GPU hardware, and training that would take days on a CPU completes in hours. For this class of workload, the GPU is the more energy-efficient device.

Classical ML algorithms on tabular data present a different picture. Mitchell and Frank [27] measured GPU acceleration for XGBoost specifically, finding speedups of $3\times$ to $6\times$ over a four-core desktop CPU and $1.2\times$ over a dual-socket server with 24 CPU cores combined. These gains are real but considerably smaller than those achieved for neural networks. Given the GPU's substantially higher power draw, a $3\times$ to $6\times$ speedup may not be enough to recover the additional energy cost, and Mitchell and Frank measured only execution time, not energy consumption.

The question of where the break-even point lies for algorithms like XGBoost therefore remains open. A dataset may exist below which the CPU is the ecologically preferable device and above which the GPU's speedup compensates for its power draw, but this has not been empirically established for gradient boosting on tabular data.

3.4 Measurement Tool Accuracy: Software Estimation vs. Hardware Sensors

As Green AI research has grown, so has demand for accessible tools that quantify the energy and carbon cost of model training. The emerging ecosystem of software trackers and analytical estimators has simplified emissions measurement. However, recent empirical work reveals a significant problem: different tools often report strikingly different results for the same experiment. These discrepancies are not random but represent systematic errors in the underlying estimation models.

The available measurement approaches fall into three categories [9]. EPMs at the wall outlet capture total system consumption but cannot isolate individual processes. On-chip sensors such as Intel's RAPL and NVIDIA's NVML offer process-level readings at fine granularity but require specific operating system permissions. Software estimation models infer energy from analytical heuristics such as TDP or operation counts. In practice, EPMs are expensive to deploy at scale and sensor access is often unavailable, so many researchers rely on estimation-based tools, despite evidence that these introduce substantial and directional errors.

How large these errors can be is documented by Bannour et al. [10], who compared six tracking tools on the same NLP training runs, including ML CO_2 Impact, CarbonTracker, experiment-impact-tracker, and Green Algorithms. Tools relying on analytical models and user-supplied parameters systematically overestimated consumption, while those reading directly from hardware sensors produced more consistent results. The underlying reason for this divergence is that analytical approaches cannot capture the dynamic fluctuations of real hardware. Henderson et al. [6] demonstrated this empirically: TDP-based estimates significantly over- or underestimate actual energy compared to sensor readings, because they ignore real-time utilization shifts and dynamic memory effects, and FLOPs-based estimates often show little correlation with actual consumption across architectures due to hardware-specific firmware behaviors. Rodriguez et al. [9] confirmed this pattern in practice, finding that purely analytical tools consistently deviated from measured ground truth, while sensor-based tools remained closer, though still subject to platform-specific constraints.

More recent work has moved toward direct validation of widely adopted trackers against physical ground truth. Fischer [45] compared CodeCarbon against external power meters across a range of workloads, finding that CodeCarbon consistently underestimates total system power draw by 20 to 30%, as software profilers cannot account for cooling overhead or power supply inefficiencies. Critically, this underestimation is substantially worse for CPU-only workloads than for GPU workloads, with relative errors exceeding GPU-equivalent errors by more than an order of magnitude in some configurations, a finding with direct consequences for any study benchmarking classical ML, which runs predominantly on CPU.

3.5 Seasonal and Geographic Carbon Intensity Effects

The carbon footprint of a machine learning experiment is not determined solely by algorithmic and hardware choices. Because electricity grids draw on different fuel mixes depending on geography, time of day, and season, two identical training runs executed on different infrastructure can produce vastly different emissions, making geographic and temporal factors a distinct dimension of Green AI research.

The geographic component of this variability has been measured at scale. Lacoste et al. [7] show that a server in a hydroelectric region such as Quebec emits approximately 20 gCO_2eq/kWh, while a server in a fossil-fuel-dependent region like Iowa emits up to 736.6 gCO_2eq/kWh, a factor of nearly 40 for the same computational workload. Dodge et al. [8] confirmed this pattern across 16 cloud computing regions, finding that choosing the most carbon-efficient data center can reduce training emissions from 26,000 to 7,000 grams of CO_2. Tools such as Green Algorithms [11] and the ML CO_2 Impact Calculator [7] were developed to help practitioners account for these regional differences at the planning stage, making workload placement a practical lever for emissions reduction.

Temporal variation tells a more nuanced story. Dodge et al. [8] find that diurnal variation is meaningful: starting a training run at midnight rather than during peak solar generation hours can increase emissions by up to 8% in certain regions, and they propose time-shifting or dynamically pausing workloads based on real-time grid data. Seasonal variation, however, proves negligible across their 12-month tracking of globally distributed cloud instances, leading them to conclude that month-to-month emissions differences within a single cloud region are small.

This conclusion may not transfer to local electricity grids with a high share of seasonally dependent renewables, which can exhibit considerably stronger swings than global cloud averages suggest.

3.6 Hyperparameter Tuning Efficiency

To achieve state-of-the-art predictive performance, machine learning models require extensive hyperparameter tuning. Since this process involves training architectures multiple times across various configurations, a critical tension emerges between the ecological cost of finding the optimal configuration and the performance benefits it yields.

The Red AI paradigm has normalized evaluating only the final, best-performing model, a practice that obscures the computational budget spent during the development and tuning phases [1]. Practical measurements reveal that this hidden cost can be substantial. Strubell et al. [3], in an empirical case study tracking the development of a NLP pipeline, quantified this overhead directly. A single tuning cycle evaluating 24 configurations multiplied the computational cost of the baseline model by a factor of 24. Further the complete development process, which required training 4,789 model variants to find the optimal hyperparameters, consumed 239,942 GPU hours in total. This demonstrates that the tuning phase can multiply the baseline energy footprint of a single model by several orders of magnitude.

To reduce these impacts, researchers must move away from exhaustive methods like grid search. Lacoste et al. [7] explicitly state that grid search is highly inefficient in terms of both model performance and environmental impact, and that replacing it with random or model-based search strategies can accelerate convergence and directly reduce carbon emissions. Experimental benchmarks support this: Bergstra and Bengio [42] show that as few as 8 to 32 random trials can consistently match or outperform exhaustive grid search depending on the size of the search space. Going further, Bergstra et al. [43] illustrate the practical scale of this problem on a Deep Belief Network with 32 hyperparameters: evaluating only two values per parameter under grid search yields over four billion combinations, which at roughly one hour of training each would require nearly half a million years of compute. By distributing asynchronous TPE proposals across five GPUs, they evaluated only 200 configurations in 24 hours of wall time and identified a result that outperformed expert-tuned baselines [43].

While the literature clearly documents the empirical tuning costs for DL models in NLP and computer vision [3], there is a lack of equivalent benchmarks for classical DL on tabular data. How much energy and CO_2 can be saved by reducing the number of tuning trials for algorithms like Random Forest or XGBoost, and at what trial count the marginal accuracy gain ceases to justify the ecological cost, remains unquantified. The present thesis addresses this gap by tracking the carbon emissions of Bayesian HPO via Optuna for Random Forest, XGBoost, and MLP on tabular datasets, providing a real-world basis for evaluating whether the ecological overhead of tuning is proportionate to the performance gains it yields for classical DL algorithms.

3.7 Lifecycle Emissions: The Ecological Cost of Deployment and Inference

While the machine learning community has heavily scrutinized the energy and carbon footprint of the training phase, this narrow focus risks obscuring the true ecological cost of an AI system's complete lifecycle. A critical tension thus emerges: does the cumulative energy consumed during the deployment phase eventually outweigh the initial, highly-publicized training footprint?

Industry-scale empirical studies demonstrate that the inference phase is often the dominant factor in a model's total energy consumption. As models are integrated into everyday applications, they are queried millions of times, creating a multiplicative effect where the ongoing energy demand of continuous deployment quickly eclipses the one-time cost of training [39]. This dominance is reflected in industry estimates cited by Patterson et al. [46], who report that both NVIDIA and Amazon Web Services estimated that 80 to 90% of total machine learning computing demand in large-scale deployed systems is attributable to inference. Wu et al. [47] provide further empirical support at the production level, finding that for deployed language models at Meta, the inference phase accounts for 65% of total lifecycle resource consumption, significantly outweighing the training phase.

Consequently, recent research has begun explicitly measuring the inference footprint of pre-trained models at deployment scale. By systematically benchmarking energy consumption across diverse model architectures, Luccioni et al. [48] found that multi-purpose generative models require orders of magnitude more energy per inference than task-specific systems, even when controlling for the number of model parameters. They also formalize the concept of a cost parity point: the number of predictions at which cumulative inference emissions surpass the initial training footprint, providing a concrete metric for lifecycle analysis. Accurately evaluating this break-even requires accounting for hardware specializations, as Desislavov et al. [39] note that inference efficiency depends heavily on whether a model can fully exploit the underlying accelerator's optimizations.

Despite the growing acknowledgment of inference costs, the current literature exhibits a significant blind spot. The lifecycle analyses and inference benchmarks provided by these studies rely almost exclusively on complex DL architectures applied to unstructured data: Luccioni et al. [48] evaluate only text and image modalities, Desislavov et al. [39] focus on ImageNet and GLUE benchmarks, Patterson et al. [46] restrict their analysis to large NLP models, and Wu et al. [47] examine DL recommendation and language models. For classical DL algorithms operating on tabular data, direct experimental evidence on lifecycle emissions is near-absent. It therefore remains unquantified at what number of predictions the cumulative inference CO_2 footprint of a model such as Logistic Regression or Random Forest surpasses its training footprint, and how this lifecycle scaling compares across algorithms of varying complexity. The present thesis addresses this gap directly by performing a dedicated lifecycle analysis that systematically scales predictions to empirically measure these break-even points for classical classification algorithms on tabular data.

3.8 Composite Efficiency Metrics: Balancing Performance and Ecological Cost

The singular focus on predictive accuracy in machine learning has historically obscured the underlying computational costs of model training, raising a critical methodological question: how can researchers systematically evaluate both model quality and environmental footprint within a single, standardized framework?

To counteract the tendency to achieve marginal accuracy gains through massive computational expenditure, Schwartz et al. [1] argue that efficiency must be elevated to a primary evaluation criterion alongside accuracy. Measuring algorithmic progress exclusively through predictive performance is fundamentally incomplete. Hernandez and Brown [49] demonstrate that true progress must be quantified by holding performance constant and measuring the reduction in computational cost required to achieve it. Taken together, these theoretical arguments advocate for composite metrics that mathematically integrate compute costs with model capabilities to provide a holistic view of algorithmic efficiency.

In practice, the machine learning community has already recognized the necessity of such combined measurements. Coleman et al. [50] introduced composite metrics such as Time-to-Accuracy and Cost-to-Accuracy in the DAWNBench benchmark, demonstrating that the community accepts end-to-end metrics that evaluate the time and financial budget required to reach a predefined quality threshold. Building upon this precedent, the MLPerf benchmark suite formally adopted Time-to-Train as its primary evaluation metric, deliberately shifting the focus away from raw accuracy towards the holistic efficiency of the training system [51].

While these benchmarks validate the use of composite metrics, their efficiency components rely exclusively on execution time or financial cost [50], [51]. As established in previous sections, neither time nor cost necessarily correlates with the actual carbon footprint, particularly when hardware power draw and regional grid carbon intensities vary. Recent theoretical frameworks have begun to recognize this shortcoming by proposing conceptual metrics, such as the Job Quality per Cost Rate (JQCR), which aims to balance the quality of a computing job against its sustainability costs [5]. However, this approach remains strictly conceptual and lacks a defined mathematical formula linking specific accuracy measurements to carbon emissions [5]. To date, no established, mathematically operationalized composite metric explicitly incorporates a direct ecological component alongside predictive accuracy. The Efficiency-Weighted F1 (EWF1) metric introduced in this thesis addresses this exact gap: by mathematically integrating a model's F1-score with its operational CO_2 emissions, EWF1 builds upon the accepted paradigms of DAWNBench and MLPerf to provide a composite score explicitly dedicated to measuring ecological efficiency.

4 Methodology

This chapter outlines the design decisions underlying the empirical evaluation of this study. It describes the research questions, the selected datasets and models, the evaluation metrics, the validation strategy, and the hyperparameter tuning approach that together form the methodological foundation for the experiments conducted in Chapter 5.

4.1 Research Questions

The main research question that this study aims to answer is: *How does the trade-off between energy consumption and predictive performance vary across Logistic Regression, Random Forest, XGBoost and MLP models when the complexity of the models and the size of the dataset are systematically varied?* The following sub-questions will also be addressed in the course of this thesis:

- Under what conditions do the resource consumption of more complex models exceed the achievable accuracy gain over simpler baselines?
- How does energy consumption and CO_2 output scale with increasing neural network complexity (number of layers and neurons)?
- In what way does energy consumption differ between CPU-based and GPU-based training for equivalent tasks?
- How does reducing the number of training instances on large datasets affect model accuracy and energy consumption across different algorithms?
- At what prediction volume does the cumulative inference CO_2 footprint of a model surpass its training footprint, and how does this break-even point differ across model families?

4.2 Datasets

Dataset size is a primary driver of both computational demand and energy consumption during model training [9]. To evaluate how this relationship varies across different scales, three classification datasets were selected that span several orders of magnitude in size. Classification was chosen as the task type since it is well-established across all selected models and allows for direct comparison of predictive performance via standardized metrics such as accuracy and weighted F1 score. The datasets are summarized in Table 4.1.

Table 4.1: Overview of datasets used in this analysis

Dataset	Instances	Features	Classes	Scale
Wine [52]	178	13	3	Small
Default of Credit Card Clients [53]	30,000	23	2	Medium
HIGGS [54]	11,000,000	28	2	Large

The Wine dataset contains 178 instances across 13 features, where each feature describes a chemical property of wines from the same region in Italy, derived from three different cultivars.

The Default of Credit Card Clients (hereafter: Credit) dataset contains 30,000 instances across 23 features and focuses on predicting payment default among credit card clients in Taiwan.

The HIGGS dataset is the largest dataset used in this study, comprising 11,000,000 instances across 28 features, seven of which are high-level features derived by physicists to help discriminate between the two classes [55]. The classification goal is to distinguish between signal events produced by Higgs bosons and background noise processes.

These three datasets vary significantly in size, enabling a systematic evaluation of how model performance and energy consumption scale across different levels of data complexity. The jump from 30,000 to 11,000,000 instances is large, spanning nearly three orders of magnitude. This gap is addressed by the dataset scaling experiment described in Section 4.7, which trains models on seven intermediate HIGGS subset sizes ranging from 1,000 to 11,000,000 instances and thereby provides a continuous view of how performance and emissions evolve across the full size range.

4.3 Models

Selecting models of varying complexity is central to answering the research questions of this study. A model that achieves competitive accuracy with significantly lower energy consumption represents a more ecologically efficient choice, but this trade-off can only be quantified if the comparison spans a meaningful range of complexity. For this reason, four classification models were selected, ranging from a linear baseline to tree-based ensemble methods and a neural network, covering low, medium, and high computational demand respectively.

Logistic Regression (LR) serves as the lightweight baseline in this study. Despite its simplicity as a linear classifier, it often achieves competitive Accuracy (ACC), making it well suited to quantifying whether the additional resource consumption of more complex models yields a meaningful gain in predictive performance.

Random Forest (RF) was selected as a mid-complexity representative of tree-based ensemble methods. Relevant to this study, Random Forest does not support GPU-accelerated training and is therefore run exclusively on the CPU.

Building on the tree-based approach, Extreme Gradient Boosting (XGB) was chosen as a second mid-complexity model, with the key distinction that it also supports GPU-accelerated training. This makes it possible to directly compare the energy consumption and training time of CPU- and GPU-based tree ensemble methods under identical conditions, which is central to one of the research questions of this study. The suitability of GPU-accelerated XGBoost for datasets at this scale has been demonstrated by Mitchell and Frank, who show that the histogram-based GPU implementation can process a dataset of 10 million instances with 28 features entirely within GPU memory [27].

The MLP was selected as the most complex model, representing the neural network category. Its highly configurable architecture, particularly the number of hidden layers and neurons per layer, makes it well suited for the architectural complexity experiments conducted in this study, where these parameters are systematically varied to assess their impact on predictive performance and energy consumption.

Table 4.2: Overview of classification models used in this study

Model	**Type**	**Device**	**Role**
LR	Linear	CPU	Baseline
RF	Ensemble	CPU	Mid-complexity
XGB	Gradient Boosting	CPU & GPU	Mid-complexity
MLP	NN	GPU	High-complexity

Random Forest is the only model excluded from one dataset combination: due to prohibitive training runtimes at 11 million instances, it is not evaluated on HIGGS in the main benchmark. It is, however, included in the dataset scaling experiment for subset sizes up to 500,000 instances, where runtimes remain manageable. This provides data on the point at which Random Forest becomes computationally impractical relative to the other models. All other model–dataset combinations are included in the main benchmark.

Together, the selected models cover a broad spectrum of algorithmic complexity, from a linear baseline to gradient boosting and deep learning, enabling a meaningful comparison of both predictive performance and energy efficiency across fundamentally different model types. The following section defines the metrics used to evaluate these dimensions.

4.4 Evaluation Metrics

To answer the research questions of this study, two categories of metrics were employed: classification performance and resource consumption. Accuracy and Weighted F1-Score were selected to measure predictive performance, as they provide a standardized basis for comparing models across datasets of varying class distributions. Energy consumption in kWh and CO_2 emissions in kgCO_2eq were chosen as the primary resource metrics, directly reflecting the ecological cost of model training.

Training time in seconds was included as a secondary resource metric, as it captures computational demand independently of hardware-specific power draw. Furthermore, inference time was measured across all experimental setups to account for the operational footprint. In large-scale deployment scenarios, the cumulative energy demand of repeated predictions can often rival or even exceed the initial training costs [39]. Together, these metrics enable a direct assessment of whether gains in predictive performance justify the associated increase in resource consumption.

Inspired by the efficiency framing proposed in Schwartz et al., a custom composite metric was developed for this study, named the Efficiency-Weighted F1 (EWF1). Equation (4.1) defines this metric, where the numerator is Weighted F1-Score:

$$\text{EWF1} = \frac{WF1}{1 + \lambda \cdot \hat{c}} \tag{4.1}$$

where $\hat{c}$ denotes the min-max normalized CO_2 emissions within each dataset and λ controls the relative weight of ecological cost. The normalization maps the observed emission range per dataset to $[0, 1]$, ensuring the denominator remains in the bounded range $[1, 2]$ and that each model's penalty is expressed relative to the other models in the same comparison.

The default weight $\lambda = 1$ assigns equal importance to the ecological penalty and the WF1 numerator. To verify that the conclusions of this study do not depend on this choice, EWF1 rankings were computed for $\lambda \in \{0.5, 1, 2\}$ across all three datasets. The resulting model orderings were identical in every case, confirming that the qualitative ranking is robust to the specific value of λ and is not an artefact of the chosen weight. Higher scores indicate a more favorable trade-off between predictive performance and ecological cost.

It should be noted that $\hat{c}$ is inherently location-dependent, as CO_2 emissions are tied to the carbon intensity of the local electricity grid. Consequently, EWF1 scores reflect the ecological efficiency of the specific experimental setup used in this study and may not generalize directly to other energy infrastructures.

Emissions are tracked exclusively over the CV loop. Data loading and preprocessing are deliberately excluded from the measurement interval, as their cost scales with dataset size rather than model architecture and would therefore introduce noise into cross-model comparisons. This ensures that all reported emissions reflect model training and evaluation cost alone.

4.5 Validation Strategy

To obtain reliable and generalizable performance estimates, a CV strategy was employed rather than a single train-test split.

Specifically, K-Fold CV was applied with $k = 5$ folds, a value widely recommended in the literature as a practical balance between computational cost and estimation stability [14]. Each model is trained and evaluated five times on non-overlapping subsets of the data, and the final performance score is averaged across all folds. This approach is consistent across all models and datasets, ensuring comparability of results.

To ensure reproducibility, a fixed random state was used for all CV splits, model initialization, and data shuffling. The specific value and its technical implementation are described in Section 5.1.2.

Because two of the three datasets exhibit class imbalance, stratified CV is applied to ensure each fold preserves the original class distribution. This is directly coherent with the choice of WF1 as the primary metric, as described in Section 4.4.

No separate held-out test set is used. The conventional alternative would be to reserve a fixed partition of the data for final evaluation and restrict CV to the tuning phase. For Wine, with 178 instances, an 80/20 split would leave approximately 35 samples for testing, which is too small to produce a stable performance estimate. Five-fold CV over all available instances evaluates each data point exactly once and therefore yields a lower-variance estimate than any fixed held-out subset of comparable size. For Credit and HIGGS, where each fold alone contains thousands to millions of instances, the variance of the performance estimate is negligible regardless of whether a separate test partition is used, and the choice has no practical consequence for the reported figures.

The remaining concern is that tuning and evaluation share the same folds, which introduces a slight optimistic bias through indirect data leakage: the selected hyperparameters are chosen to perform well on the validation folds, and those same folds contribute to the final reported score. This bias is, however, uniform across all models and datasets, because every configuration is subject to the identical procedure. When all competing models are subject to the same inflation, relative rankings are unaffected. The primary claims of this study are comparative rather than absolute, and the reported figures should be interpreted accordingly.

The theoretically exact remedy for this bias is nested CV, in which an inner loop handles tuning and a fully independent outer loop estimates generalization error. On HIGGS, a five-fold outer loop would multiply HPO evaluations fivefold per model–dataset combination, adding weeks of additional compute at substantial energy cost. Nested CV is therefore rejected on ecological grounds consistent with the framing of this study, as discussed in Section 2.6.

4.6 Hyperparameter Tuning

Hyperparameter tuning was applied to all models except Logistic Regression. Without tuning, default parameters tend to favor certain model families over others, potentially skewing both performance and resource consumption comparisons [56]. Logistic Regression is deliberately excluded: its role as a lightweight linear baseline depends on its simplicity, and tuning it would undermine that function.

Optuna was selected as the optimization framework, using TPE as its search strategy. As discussed in Section 2.7, BO-based methods require substantially fewer evaluations than grid or random search to reach competitive configurations, which directly reduces the energy cost of the tuning process itself. Each run was configured to execute 40 trials per model–dataset combination, a budget chosen to balance search thoroughness against ecological overhead.

Tuning was performed separately for each dataset, as the datasets differ substantially in size, feature space, and class distribution. The optimization objective is WF1 in all cases, consistent with the primary evaluation metric defined in Section 4.4. The concrete search spaces and implementation details are described in Section 5.4.

A limitation of the fixed 40-trial budget is that it does not account for differences in search space dimensionality across models. Random Forest exposes five hyperparameters and

XGBoost six, both in well-defined, low-dimensional spaces that TPE can navigate reliably within 40 evaluations. The MLP search space is larger and structurally more complex: it includes a discrete depth parameter (`n_layers`, ranging from 1 to 4) that conditionally activates up to four additional layer-width dimensions, plus continuous parameters for dropout rate, learning rate, and batch size, yielding an effective dimensionality of five to eight depending on the sampled depth. Conditional search spaces of this kind require more evaluations for TPE to build an accurate surrogate model. In addition, each MLP trial on HIGGS involves training a neural network and is substantially more expensive than a comparable tree-based trial, so the 40-trial budget represents a greater practical constraint for the MLP than for Random Forest or XGBoost. As a result, the MLP may not have reached the same degree of optimization as the tree-based models, and the reported MLP performance figures should be interpreted as a lower bound on what a more exhaustive search might achieve. Increasing the trial budget selectively for the MLP would, however, break the comparability constraint: allocating more search effort to one model than to others would confound any observed performance difference with tuning thoroughness rather than isolating it as a property of the algorithm itself. The uniform budget is therefore a deliberate design choice to keep the search effort symmetric across all models.

4.7 Experimental Protocol

The research questions are operationalized through five experiments. Each isolates a specific dimension of the energy-performance trade-off while holding the remaining factors constant. Implementation details for all five are described in Section 5.8.

The **main cross-dataset benchmark** trains all five models on all three datasets, yielding 14 model–dataset combinations (Random Forest on HIGGS is excluded, as described in Section 4.3). It forms the empirical backbone of the study and addresses the overall energy-performance trade-off across model complexity and dataset scale.

The **dataset scaling experiment** holds everything constant except the number of training instances. All five models are included at subset sizes up to 500,000 instances; beyond that threshold Random Forest is excluded due to prohibitive runtimes, leaving four models for the larger subsets up to 11,000,000. Because all other variables are fixed, observed differences in performance and emissions can be attributed exclusively to dataset size. The gap between the 30,000-instance Credit dataset and the full 11-million-instance HIGGS dataset is bridged by the intermediate subset sizes in this experiment, which cover the range that is typical of many real-world tabular classification tasks. The experiment directly addresses RQ4 and, through the XGBoost CPU/GPU pairing, also feeds into RQ3.

The **MLP architecture variation** isolates the effect of network depth and width by training a grid of 12 architectures on HIGGS: three widths (128, 256, 512 neurons) crossed with four depths (1–4 hidden layers). Non-architectural hyperparameters are fixed to neutral defaults to prevent any single configuration from being inadvertently favoured by the tuning results. This directly addresses RQ2.

The **temporal carbon intensity analysis** is purely retrospective: it re-weights the energy already recorded in the main benchmark against eight carbon intensity scenarios derived from one year of hourly API data for the German grid. This quantifies the sensitivity of reported emissions to training time and location, and bounds the uncertainty introduced by CodeCarbon's static intensity factor.

The **lifecycle break-even analysis** combines the one-time training footprint from the main benchmark with per-prediction inference latency to estimate at what prediction volume cumulative inference emissions surpass training emissions. It addresses RQ5 without requiring any additional training runs.

5 Implementation and Measurement

This chapter describes how the experimental pipeline was built, how measurements were collected, and which experiments were conducted. It covers the hardware and software environment, the data loading architecture, each model implementation, the hyperparameter tuning setup, and the emissions measurement infrastructure including the custom CPU power monitoring solution. The final section defines the individual experiments that make up this study, directly preceding the results reported in Chapter 6.

5.1 Experimental Setup

5.1.1 Hardware Setup

A critical decision in the experimental design was the selection of the computing platform on which all experiments would be conducted. Two options were considered: the university's shared computing infrastructure or a local machine. The latter was ultimately chosen, as it provides several key advantages for this study. First, it ensures complete reproducibility, since all experiments are executed on identical hardware configuration, eliminating interfering variables that could arise from shared resource conflicts or system variability. Second, it provides consistent availability and direct control over system configuration, which is particularly important given the extended runtime of the large-scale experiments (particularly on the 11M-instance HIGGS dataset). Third, local execution facilitates fine-grained performance monitoring and measurement of energy consumption, a primary objective of this study.

The hardware platform selected for all experiments is a consumer-grade desktop system with the following specifications: an NVIDIA RTX 4070 GPU (12 GB GDDR6 memory, 5,888 Compute Unified Device Architecture (CUDA) cores), an AMD Ryzen 7 5700X processor (8 cores / 16 threads, base clock 3.6 GHz) and 32 GB of DDR4 Random-Access Memory (RAM). The system runs Windows 11 with all code implemented in Python 3.13.0.

Table 5.1: Hardware specification of the experimental system

Component	**Specification**
CPU	AMD Ryzen 7 5700X (8 cores / 16 threads, 3.6 GHz base)
GPU	NVIDIA RTX 4070 (12 GB GDDR6, 5,888 CUDA cores)
RAM	32 GB DDR4
Operating System	Windows 11

The choice of hardware reflects a deliberate trade-off between experimental scope and practical feasibility. The RTX 4070 represents a mid-to-high-end consumer GPU with sufficient compute capability to execute GPU-accelerated training for XGBoost and MLP models in reasonable time, while remaining accessible to many practitioners. The Ryzen 7 5700X provides adequate CPU performance for baseline models (Logistic Regression, Random Forest) and CPU-based XGBoost training. Together, this configuration enables direct comparison of CPU- and GPU-accelerated training on the same platform, which is central to answering one of the research questions (*How does energy consumption differ between CPU-based and GPU-based training for equivalent tasks?*).

It is important to note that energy consumption and CO2 emissions measured in this study are specific to this hardware configuration and the German electricity grid, and may not directly generalize to other systems or geographic regions. However, the relative performance comparisons (e.g., the energy efficiency ranking of different algorithms) are more likely to generalize, as they reflect algorithmic properties rather than hardware specifics alone.

5.1.2 Software Environment and Reproducibility

All experiments were implemented in Python 3.13.0. The main libraries used are described below.

Scikit-learn served as the foundation of the evaluation pipeline. It provided the `LogisticRegression` and `RandomForestClassifier` implementations, as well as `KFold` and `cross_validate` for CV, `StandardScaler` for feature normalization, `make_pipeline` for chaining preprocessing and model steps, and WF1 and `make_scorer` for evaluation metrics.

The XGBoost library was used for the gradient boosting model, as it provides a highly optimized implementation that supports both CPU and GPU training.

For the MLP, PyTorch [57] was used as the DL backend. To integrate the PyTorch model into the scikit-learn evaluation pipeline, the Skorch library [58] was used, which wraps PyTorch models in a scikit-learn-compatible interface.

Hyperparameter tuning for all models except Logistic Regression was performed using Optuna, as described in Section 4.6.

Carbon emissions and energy consumption were tracked using CodeCarbon, which is described in detail in Section 5.5.

To ensure reproducibility, a fixed random seed of 42 was used for all data splits, CV folds, and model initialization. A `requirements.txt` file is provided so that the exact library versions used in this study can be reproduced.

The implementation code was developed with the assistance of Claude Code [59], an AI-powered development tool by Anthropic. All generated code was reviewed, tested, and integrated by the author.

The evaluation relies on `KFold` with five splits and shuffling enabled, executed through `cross_validate()`. Performance is measured using a scoring dictionary that tracks both Accuracy and Weighted F1-Score via `make_scorer(f1_score, average="weighted")`. Final scores are the mean across all five folds.

5.2 Dataset Integration

The three datasets used in this study, Wine, Credit, and HIGGS, were introduced in Section 4.2. This section describes how they are integrated into the software architecture, covering the central configuration file, the data loading pipeline, and dataset-specific constraints.

To ensure a consistent foundation across all model scripts, a central configuration file (`config.py`) was created. It serves as a single source of truth for all dataset-related settings, so that each script accesses the same parameters without redundant definitions.

For each dataset, the configuration stores the file path, the name of the target column, and an optional row limit (`nrows`). The row limit was particularly relevant for the HIGGS dataset, which contains 11 million instances, allowing experiments to be run on smaller subsets where needed. Beyond these common fields, the configuration also captures dataset-specific properties. For example, the Credit dataset includes a header row that must be skipped during loading, and the Wine dataset does not contain column names in the source file, which are therefore defined in the configuration directly. Additionally, a label offset is applied to the Wine target column, as its class labels are encoded as 1, 2, and 3 rather than starting at 0, which is required by the classifiers used in this study.

The global constants `RANDOM_STATE` and `CV_FOLDS` are also defined here, ensuring that all scripts use identical values for random seeding and CV, which is essential for the comparability of results.

To read the datasets from their respective files, a custom `load_data` method was implemented. Depending on the dataset, the appropriate loading mechanism is selected. The Wine and Credit datasets are loaded using pandas' `read_csv` function. For the HIGGS dataset, `read_parquet` is utilized. This was a deliberate decision to save storage space by storing the large HIGGS dataset as a Parquet file rather than a standard CSV.

Following the initial loading phase, the HIGGS data is reduced to a representative subset using a sampling approach controlled by the `nrows` parameter. Once the data size is managed, any columns specified in the configuration file are dropped from the respective dataset. The data is then split into the feature matrix X and the target vector y. If an offset is defined in the config, which is currently only the case for the Credit dataset, it is applied as described in the previous section. Finally, the method returns X and y for further use in the subsequent scripts. This process is visualized in Figure 5.1.

Due to the substantial scale of the HIGGS dataset, several constraints were implemented to ensure the stability of the execution environment. Training a Random Forest model on this specific data led to unmanageable processing times. For this reason, the Random Forest algorithm is explicitly bypassed for the HIGGS dataset and the program terminates using `sys.exit(0)` to prevent system crashes. This allows the process to stay within the boundaries of the available hardware resources.

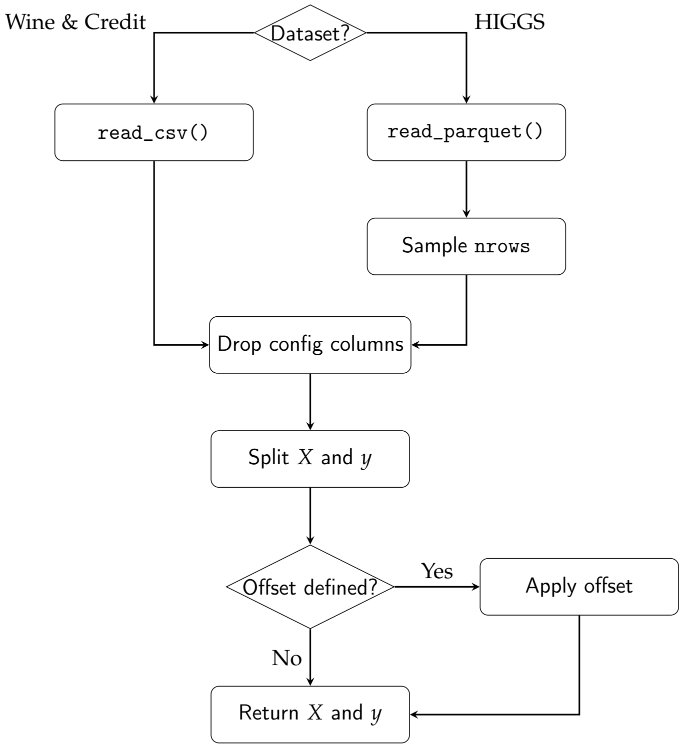


Figure 5.1: Visualization of the data loading pipeline and preprocessing steps within the `load_data()` method.

5.3 Model Implementations

As outlined in Section 4.3, the following classification algorithms were employed to address the research questions mentioned previously. This section therefore details the specific implementation of each model.

5.3.1 Logistic Regression

The first baseline model is a standard logistic regression. To ensure the model processes all features on an equal scale, the algorithm is embedded in a pipeline starting with a `StandardScaler`. This initial scaling step is crucial for the stability and overall performance of the subsequent classification.

The regression is configured to use the SAG solver by setting `solver='sag'`, chosen for its efficiency on large datasets as described in Section 2.3.1. To ensure proper convergence, the maximum number of iterations is capped at 1000 using the `max_iter` parameter.

This model setup deliberately excludes any HPO. Instead, it acts as a stable baseline to evaluate the raw computational time without the extra overhead of complex search processes.

It is worth noting that scikit-learn's `LogisticRegression` is single-threaded for the core optimization step regardless of solver choice: none of the available solvers (`lbfgs`, `newton-cg`, `sag`, `saga`, `liblinear`) parallelise a single model fit. The `n_jobs` parameter of `LogisticRegression` only applies to One-vs-Rest (OvR) multiclass decomposition, which provides no benefit for the two binary-classification datasets in this study. As a result, training runs on a single CPU core and produces roughly 10% utilisation on the eight-core system used here. The five CV folds therefore also run sequentially. The implications of this single-threaded constraint for energy comparability are discussed in Section 5.5.

5.3.2 Random Forest

The second baseline algorithm is the Random Forest classifier. This method belongs to the family of ensemble learning techniques. Unlike logistic regression, this algorithm does not require prior feature scaling. To maximize computational efficiency, the execution is fully parallelized across all available CPU cores by setting the `n_jobs` parameter to `-1`. This configuration allows the processor to run at maximum capacity, which directly influences the subsequent energy consumption tracking and overall runtime measurements.

As established in Section 5.2, Random Forest is explicitly excluded from the HIGGS dataset due to unmanageable training times at 11M instances. Including it would produce extreme runtimes without contributing meaningful insight into the scaling behavior studied here.

To ensure maximum predictive capabilities for the remaining data, the model configurations were optimized using Optuna. The optimization process specifically targeted maximizing the WF1. For strict reproducibility, the final parameter combinations for the Wine and Credit datasets are documented in Table 5.2.

Table 5.2: Optimized hyperparameters for the Random Forest classifier obtained via Optuna optimization.

Parameter	Wine	Credit
`n_estimators`	250	495
`max_depth`	20	20
`min_samples_split`	15	17
`min_samples_leaf`	6	3
`max_features`	`sqrt`	`None`

5.3.3 XGBoost

The next tree-based ensemble model is the XGBoost classifier. This model serves as a direct point of comparison to the Random Forest. A primary reason for including XGBoost in this study is that it allows for a direct comparison of computational efficiency between CPU and GPU processing. To achieve this, the configuration dictionaries are kept identical for both executions. The only difference is the hardware device parameter, which is set to use the GPU. This controlled setup ensures a precise evaluation of the energy and emissions generated by the exact same model architecture on different hardware. This setup directly addresses the third research question: *How does energy consumption differ between CPU-based and GPU-based training for equivalent tasks?*.

The CPU versus GPU comparison is derived in the plotting pipeline by reading all HIGGS subset rows from `results.csv` and fitting a log-log model separately for each device using `numpy.polyfit`, with the energy crossover point computed analytically from the intersection of the two fitted lines.

Additionally, XGBoost has a specific technical requirement for handling target variables. While the Random Forest algorithm natively adapts to different class structures, XGBoost requires explicit instructions for its learning objective. As mentioned in Section 4.2, the Wine dataset consists of three distinct categories. Therefore, the objective parameter is set to `multi:softmax`, paired with the `mlogloss` evaluation metric and a specified number of classes. In contrast, the Credit and HIGGS datasets contain only two categories. Consequently, their objective is configured as `binary:logistic` and evaluated using the standard `logloss` metric.

Similar to the previous baseline algorithm, the parameters tailored to each dataset were determined using the Optuna optimization framework. The tuning runs used GPU-accelerated execution to reduce the search time across 40 trials. The resulting hyperparameters are applied to both the CPU and GPU evaluation runs without modification, which is valid because the histogram-based algorithm is identical on both devices and converges to the same optimal configuration regardless of which hardware executes the computation. Furthermore, the algorithm processes the unscaled feature matrix X directly. It also uses the globally defined random state parameter to guarantee a deterministic execution across all hardware setups. The resulting configurations used for the final training phase are detailed in Table 5.3.

Table 5.3: Optimized hyperparameters for the XGBoost classifier across all datasets.

Parameter	Wine	Credit	HIGGS
`objective`	`multi:softmax`	`binary:logistic`	`binary:logistic`
`eval_metric`	`mlogloss`	`logloss`	`logloss`
`num_class`	3	N/A	N/A
`n_estimators`	273	452	406
`max_depth`	4	3	8
`learning_rate`	0.0801	0.0411	0.2263
`subsample`	0.6558	0.7319	0.8910
`colsample_bytree`	0.7169	0.6901	0.8222
`min_child_weight`	4	7	3

5.3.4 Multilayer Perceptron

As discussed in Section 4.3, a MLP was selected to represent the NN family. The implementation uses PyTorch as the primary deep-learning framework, combined with the Skorch library. Skorch wraps the PyTorch model in an interface that is fully compatible with the scikit-learn ecosystem. This is necessary because the entire evaluation logic, including CV and pipeline construction, relies heavily on scikit-learn.

A dynamic architecture was chosen to ensure the network structure directly reflects the optimal parameters found during the Optuna tuning process. At runtime, the script reads the best parameter set from a JSON file generated during the tuning phase. It extracts the number of hidden layers via `n_layers` and the neuron count for each individual layer via `layer_i`. These values are then assembled into a list called `layer_sizes`. This list is subsequently passed to the `MLPModule` class. This class constructs the full network dynamically upon instantiation, making the architecture entirely data-driven. The resulting layer-wise architecture, including parameter counts, is shown in Table 5.4. The specific configurations resulting from this process for each dataset are detailed in Table 5.5.

Table 5.4: Resolved layer-wise architecture of the tuned MLP for each dataset. Each hidden block consists of a Linear layer, Batch Normalization, ReLU, and Dropout. Parameter counts include weights, biases, and Batch Normalization scale and shift parameters.

(a) Wine

Lyr	In/Out	Par.
HL 1	13 → 128	2,048
Out	128 → 3	387
Total		**2,435**

(b) Credit

Lyr	In/Out	Par.
HL 1	23 → 512	13,312
HL 2	512 → 64	32,960
HL 3	64 → 128	8,576
HL 4	128 → 256	33,536
Out	256 → 2	514
Total		**88,898**

(c) HIGGS

Lyr	In/Out	Par.
HL 1	28 → 512	15,872
HL 2	512 → 256	131,840
HL 3	256 → 512	132,608
Out	512 → 2	1,026
Total		**281,346**

To minimize the overall training time, the implementation checks for CUDA availability using `torch.cuda.is_available()`. The algorithm runs on the GPU if one is present and automatically falls back to the CPU otherwise. PyTorch requires highly specific numeric types, and NNs react sensitively to implicit type mismatches. Therefore, the feature matrix X is explicitly cast to `float32` and the target vector y to `int64` before the training begins. Implementation-level optimizations such as AMP were intentionally omitted to maintain comparability across all tested algorithms and avoid hardware-specific implementation artifacts.

Input features are standardized using a `StandardScaler`, embedded together with the NN in a scikit-learn `Pipeline`. Since MLPs lack built-in scale invariance, standardizing the inputs to a mean of zero and unit variance is necessary to keep the gradient flow stable. The `NeuralNetClassifier` provided by Skorch receives all architecture parameters through a specific naming convention. It uses the `module__*` prefix to forward keyword arguments directly to the constructor of the `MLPModule`. The Adam optimizer and the `CrossEntropyLoss` criterion are used consistently across all datasets. The learning rate and the batch size are taken directly from the tuning results and are therefore tailored to each dataset. A fixed random seed is applied using `torch.manual_seed` to guarantee exact reproducibility across all iterations.

The maximum number of training epochs is set to 200. This value acts as an upper boundary rather than a strict target. An early stopping callback (`skorch.callbacks.EarlyStopping`) monitors the validation loss after each individual epoch. It halts the training process immediately if no improvement is observed for 10 consecutive epochs. This mechanism prevents unnecessarily long training runs and provides an additional layer of protection against overfitting. Consequently, the upper limit of 200 epochs is rarely reached in practice.

Table 5.5: Optimized hyperparameters for the MLP across all datasets.

Parameter	Wine	Credit	HIGGS
n_layers	1	4	3
layer_sizes	[128]	[512, 64, 128, 256]	[512, 256, 512]
dropout_rate	0.2936	0.3690	0.0334
lr	0.0030	0.0002	0.0077
batch_size	1024	1024	8192

5.4 Hyperparameter Optimization

Building on the design outlined in Section 4.6, this section describes how Optuna was integrated into the evaluation pipeline and which search spaces were defined for each model. The tuning logic is encapsulated in three dedicated scripts (`tune_xgb.py`, `tune_rfc.py`, and `tune_mlp.py`), one per tunable model. All three follow the same structural pattern and differ only in the model that is instantiated and the search space that is queried from the trial object. The target dataset is selected via a command-line argument, and the number of trials is read from the `N_TRIALS` environment variable, with a default of 40. This separation between configuration and code allows individual tuning runs to be triggered without modifying the source files. The search spaces used across all three tuning scripts are summarized in Table 5.6. Integer and continuous parameters define an inclusive range, while categorical parameters list the discrete choices available to the sampler.

Table 5.6: Hyperparameter search spaces defined in the Optuna tuning scripts.

Model	**Parameter**	**Type**	**Range / Values**
Random Forest	n_estimators	integer	[100, 500]
	max_depth	integer	[3, 20]
	min_samples_split	integer	[2, 20]
	min_samples_leaf	integer	[1, 10]
	max_features	categorical	{sqrt, log2, None}
XGBoost	n_estimators	integer	[100, 500]
	max_depth	integer	[3, 8]
	learning_rate	float (log)	[0.01, 0.3]
	subsample	float	[0.6, 1.0]
	colsample_bytree	float	[0.6, 1.0]
	min_child_weight	integer	[1, 10]
MLP	n_layers	integer	[1, 4]
	layer_i (per layer)	categorical	{64, 128, 256, 512}
	dropout_rate	float	[0.0, 0.5]
	lr	float (log)	[10^{-4}, 10^{-1}]
	batch_size	categorical	{1024, 4096, 8192}

Each tuned parameter corresponds directly to a mechanism described in the theoretical background. For Random Forest, `max_features` implements the random feature selection that reduces inter-tree correlation and drives the variance reduction established in

Section 2.3.3: restricting the candidate feature set at each split forces trees to diversify their splitting decisions, lowering ensemble variance without sacrificing individual tree strength. The `n_estimators` budget determines how closely the ensemble approaches the generalization error limit guaranteed by Breiman's convergence argument; `max_depth`, `min_samples_split`, and `min_samples_leaf` are structural constraints that directly trade off the bias-variance balance of each base tree. For XGBoost, `learning_rate` is the shrinkage factor described in Section 2.3.4 that scales each new tree's contribution and slows convergence to prevent overfitting; `colsample_bytree` is the per-tree feature subsampling also described there; and `min_child_weight` acts as a leaf-pruning mechanism that complements the explicit tree-complexity penalty γT in Equation (2.4) by requiring a minimum sum of second-order gradient statistics before a split is retained. For the MLP, `n_layers` and the per-layer width choices directly govern the depth-width trade-off discussed in the context of the Universal Approximation Theorem in Section 2.3.5; `dropout_rate` maps directly to the Dropout regularization described there [16]; and `lr` and `batch_size` govern the Adam optimizer and mini-batch gradient estimation, both introduced in Section 2.3.5.

After the dataset has been loaded through the shared `load_data()` function described in Section 5.2, each script constructs a `KFold` splitter using the same global constants `CV_FOLDS` and `RANDOM_STATE` that are also applied during the final evaluation. Reusing the identical splitter ensures that the F1 score returned to Optuna is computed under the same validation conditions as the metrics reported later in Chapter 6, eliminating any discrepancy between the score the optimizer maximizes and the score the study ultimately reports.

The core of every script is an `objective(trial)` function that draws a candidate hyperparameter configuration from the trial object, instantiates the corresponding classifier, and returns the mean WF1 obtained from a five-fold CV via `cross_val_score` with `make_scorer(f1_score, average="weighted")`. This is the only output that Optuna is exposed to. Everything else, such as emissions tracking and timing, is handled outside the objective function.

Each study is created via `optuna.create_study(direction="maximize")` and parameterized with a `TPESampler` seeded with `RANDOM_STATE=42`. Seeding the sampler is what makes the trial sequence reproducible across re-runs. Without it, the suggested hyperparameter combinations would differ on every execution, and the resulting tuning emissions would no longer be comparable. The optimization itself is launched through `study.optimize(objective, n_trials=N_TRIALS)`.

To capture the energy footprint of the tuning process itself, the entire `study.optimize` call is wrapped in a CodeCarbon `EmissionsTracker` with a script-specific `project_name` (e.g. `tune_mlp_higgs`), so that tuning runs are clearly distinguishable from training runs in the `emissions/` directory. The same tracker is used across all three scripts and is configured identically to the trackers used during training, as described in Section 5.5.

The five-fold CV used within the Optuna objective function relies on the same data partitions as the final model evaluation. This choice is deliberate and consistent with the validation strategy established in Section 4.5, where the rationale is discussed in full: the variance advantage of full-data CV over a fixed held-out partition, the structural uniformity of the optimistic bias across all models, and the prohibitive ecological cost of nested CV at the dataset scales used in this study. In implementation terms, reusing the identical `KFold` splitter also ensures that the F1 score returned to Optuna is computed under precisely the same validation conditions as the metrics reported later in Chapter 6, eliminating any discrepancy between the score the optimizer maximizes and the score the study ultimately reports.

Once a study completes, the best score, the corresponding hyperparameter dictionary, the duration, the emissions reported by the tracker, and the trial count are written to a shared `best_params.json` file via the `save_best_params()` utility. The function performs an upsert keyed by model and dataset, so that re-running a single tuning configuration overwrites only its own entry and leaves the other results untouched. The model scripts introduced in Section 5.3 consume this file through the corresponding `load_best_params()` helper, which decouples tuning from training entirely: the final training pipeline never instantiates Optuna, it merely reads the previously persisted hyperparameter dictionary and passes it to the classifier constructor.

5.5 Emissions Measurement

Measuring energy consumption and CO_2 emissions is the central objective of this study, and the measurement infrastructure therefore required particularly careful design. This section describes how CodeCarbon was integrated into each script, how measurement coverage was defined across model and dataset combinations, and what hardware-specific constraints and limitations apply to the collected values.

Energy consumption and CO_2 emissions were tracked using CodeCarbon as described in Section 2.2.4. In each model script, an `EmissionsTracker` instance is created before the CV loop begins and configured with two key parameters: `output_dir`, which points to the shared `emissions/` directory at the project root, and `project_name`, which uniquely identifies the run. The tracker is then started by calling `tracker.start()` immediately before the `cross_validate()` call, and stopped via `tracker.stop()` once all folds have completed. The return value of `tracker.stop()` is a single float representing the total CO_2-equivalent emissions in kilograms accumulated over the entire tracked interval. This value is what gets passed to `save_results()` and written to `results.csv`.

The placement of the tracker boundaries was a deliberate choice. Data loading and preprocessing happen before `tracker.start()`, so they are excluded from the measurement. Only the CV itself, which includes all five training and evaluation passes, falls inside the tracked interval. This ensures that the reported emissions value reflects the cost of model training and evaluation exclusively, and that values are directly comparable across models and datasets without interference. While standard deployment practices typically conclude with a final retraining phase on the entire dataset, this study deliberately evaluates the cumulative resource footprint of the CV loop. This approach captures the realistic, unavoidable computational burden of model evaluation and avoids the redundant ecological cost of an additional training pass, as the relative efficiency rankings between algorithms remain structurally unaffected.

Figure 5.2 illustrates this structure. Prior to training, data is loaded and, where required, features are normalized using `StandardScaler`. Scaling was applied to Logistic Regression and MLP, as these models are sensitive to the magnitude of input features, while tree-based models (Random Forest, XGBoost) were trained on the raw input data.

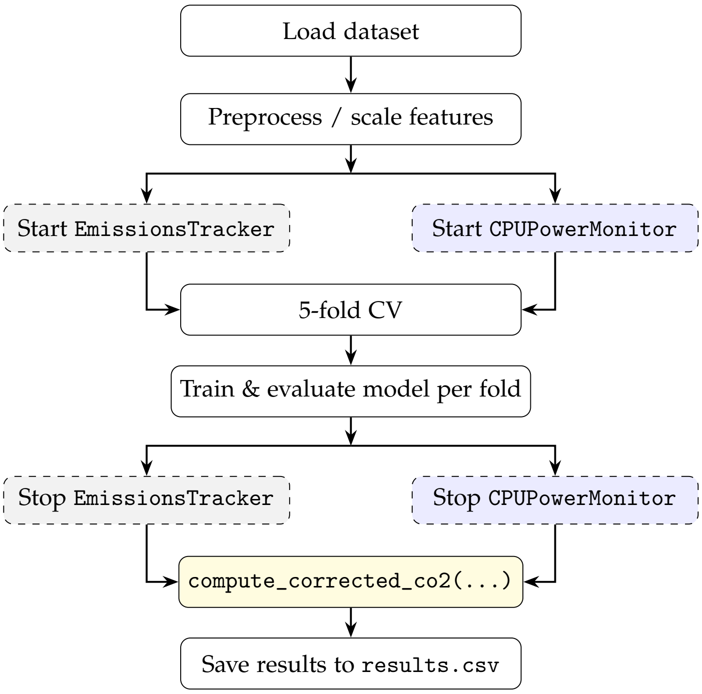


Figure 5.2: Training and emissions tracking procedure per model-dataset combination. Dashed gray boxes indicate CodeCarbon `EmissionsTracker` calls, and dashed blue boxes indicate `CPUPowerMonitor` calls (background thread, 0.25 s sampling). Both run in parallel during CV. After training, `compute_corrected_co2` replaces CodeCarbon's TDP-based CPU estimate with the hardware sensor value.

To allow fine-grained analysis, each model–dataset combination is tracked by a dedicated `EmissionsTracker` instance, identified by a unique `project_name` following the pattern `model_dataset`, for example `lr_wine`, `rf_credit`, or `xgb_gpu_higgs`. This scheme makes individual runs immediately identifiable in the `emissions/` directory, where CodeCarbon writes one CSV file per tracked run in addition to returning the scalar value in code. Tuning scripts follow the same pattern preceded by `tune_`, for example `tune_mlp_higgs` or `tune_xgb_cpu_wine`. In total, the experiment produces tracked emissions records for 14 training runs and up to eight tuning runs. The separate CSV files serve as a raw audit trail, while the values extracted via `tracker.stop()` are the primary source for the analysis presented in Chapter 6.

The general measurement approach used by CodeCarbon, including direct GPU readout via NVML, RAM estimation per DIMM slot, and TDP-based CPU fallback on Windows, is described in Section 2.2.4. The following paragraphs focus on the concrete implications for the hardware used in this study.

CodeCarbon's TDP-based CPU estimation on Windows has been replaced in this study by a custom `CPUPowerMonitor` class implemented in `models/power_monitor.py`. It runs in a background thread and polls the CPU Package Power sensor every 0.25 seconds via the LibreHardwareMonitor library (`HardwareMonitor` Python package [60]). This library reads the CPU Package Power sensor directly from the hardware, providing true power values rather than a TDP approximation. In practice, CodeCarbon underestimated CPU power by a factor of roughly 9× compared to the hardware sensor (e.g. 6.6 W vs. 57.8 W for Logistic Regression on the Credit dataset). The corrected CO_2 value in `results.csv` therefore replaces CodeCarbon's CPU contribution with the hardware-measured value, while GPU and RAM energy are still taken from CodeCarbon. The original CodeCarbon total is retained as `co2eq_codecarbon_kg` for reference.

This approach does, however, come with notable limitations. Because LibreHardware-Monitor requires privileged access to hardware sensors, all measurements in this study were conducted with the process running under administrator privileges. Without elevated rights, the `CPUPowerMonitor` fails to retrieve sensor data and the corrected CO_2 estimate cannot be computed. Beyond the access requirement, the approach is inherently platform-specific. LibreHardwareMonitor is a Windows-only solution, adopted here precisely because Intel RAPL, the interface CodeCarbon uses for direct CPU energy readings on Linux, is not accessible on Windows. The hardware-corrected emissions values reported in this study are therefore tied to this Windows-specific stack and cannot be reproduced directly on other operating systems.

A secondary limitation is that CodeCarbon measures at the process level, meaning background system activity on Windows 11 contributes to the recorded energy. This effect is expected to be small relative to the dominant training workload but cannot be fully eliminated.

A final, and more structurally significant, limitation concerns the differences in CPU parallelization across models. Logistic Regression relies on the SAG solver, which runs on a single core, whereas Random Forest and XGBoost exploit multi-threading across all available cores via `n_jobs=-1`. On the eight-core system used here, Logistic Regression therefore operates at roughly 10% CPU utilisation during training, while the tree-based models saturate all threads.

This difference has a direct consequence for the comparability of emissions figures. The `CPUPowerMonitor` records the total CPU Package Power, which includes the static base draw of the processor irrespective of utilisation. At 10% load, this idle component represents a disproportionately large share of the measured wattage, meaning a substantial fraction of the energy attributed to Logistic Regression reflects background system load rather than algorithmic computation. At full load, the idle fraction is negligible relative to the dominant compute draw, so the tree-based model measurements are not distorted in the same way. Isolating true algorithmic energy would require subtracting a separately recorded idle baseline from all measurements, but no such baseline was captured in this study.

The practical consequence is that Logistic Regression's reported emissions likely overstate its true algorithmic energy cost relative to the multi-threaded models. Any direct comparison of absolute emission figures between Logistic Regression and tree-based models therefore carries a methodological caveat: the measured difference partially reflects parallelization architecture and system idle power, not purely the intrinsic computational demands of the respective algorithms. This limitation is acknowledged in the cross-model comparisons in Chapter 6.

5.6 Inference Latency Measurement

Inference latency was measured separately after training, outside the emissions tracker. For each model, a single row from the dataset was passed through the trained model repeatedly, and the elapsed time was captured using `time.perf_counter()`. This provides a hardware-level estimate of per-sample prediction latency, which is relevant in deployment scenarios where repeated inference can accumulate significant energy costs [39].

Only a single row was passed to the model rather than a full batch. This was intentional: batching would measure throughput rather than the actual time it takes to process one request, which is the more relevant figure in deployment. A single-prediction latency can be extrapolated to real-world usage, where a model may handle millions of requests and the per-sample cost accumulates into a meaningful energy expenditure.

The model used for this measurement is taken from the first CV fold via `cv_results["estimator"][0]`, which is made available by passing `return_estimator=True` to `cross_validate()`. Because inference latency is strictly a function of the model's fixed architecture rather than the size of its training set, utilizing the estimator from the first fold provides an exact timing measurement without the computational redundancy of a full retraining phase. To obtain a stable latency estimate, the measurement is repeated 100 times on the same single-row slice `X[:1]`, preceded by one discarded warmup call to eliminate cold-start effects such as Just-In-Time (JIT) compilation or cache misses. The median of the 100 measurements is recorded as the inference time, making it robust to transient system load spikes that would distort a single-shot measurement.

The inference latency measurement is also instrumented with a dedicated `CPUPowerMonitor` instance, running in a background thread during the 100-repetition timing loop. This provides a direct measurement of the average CPU power draw specifically during inference, stored as `cpu_power_inference_w` in `inference_time.csv`. Training on the full dataset involves gradient computation, backward passes, and tree-building, all of which put the processor under substantially higher load than a simple forward pass. Using the training-phase power as a proxy for inference power would therefore overestimate the per-prediction energy cost and understate the break-even point. By measuring inference power directly, this limitation is resolved.

The recorded latency and inference-phase power values are subsequently used in a dedicated lifecycle analysis (`run_lifecycle_analysis.py`), which estimates the number of predictions at which the cumulative inference CO_2 footprint of each model surpasses its one-time training footprint. The per-prediction CO_2 is computed using the measured inference power rather than training power, producing an estimate grounded in the actual hardware load during prediction rather than a structurally mismatched proxy.

5.7 Results Storage and Orchestration

The results of every model run are written to `results/results.csv`, which serves as the central data source for all subsequent analysis and visualisation. Each row represents one complete model-dataset combination and is appended to the file rather than overwriting it, so that results from multiple runs accumulate without data loss. The file is managed using Python's `csv.DictWriter`, which writes a header row on first creation and appends subsequent rows without repeating the header. A consequence of the append-only design is that re-running a configuration produces duplicate rows, which must be filtered during analysis. Table 5.7 lists all columns.

Table 5.7: Column schema of `results.csv` including component-level energy tracking.

Column	Format	Description
`timestamp`	datetime	Date and time of the run
`model`	string	Model name
`dataset`	string	Dataset name
`nrows`	int / `all`	Subset size; `all` for full dataset
`accuracy`	float	Mean CV accuracy
`f1`	float	Mean WF1
`co2eq_kg`	float	Corrected CO_2 (Hardware CPU + CC GPU/RAM)
`co2eq_codecarbon_kg`	float	Original CC estimate (static 381 g/kWh factor)
`cpu_power_hw_w`	float	Average CPU package power (W)
`cpu_energy_hw_wh`	float	Total CPU energy (Wh)
`gpu_energy_wh`	float	Total GPU energy from CodeCarbon (Wh)
`ram_energy_wh`	float	Total RAM energy from CodeCarbon (Wh)
`training_time_s`	float	Total CV wall-clock time (s)

The primary emissions value is `co2eq_kg`, which combines the hardware-sensor CPU reading with CodeCarbon's GPU and RAM estimates as described in Section 5.5. The column `co2eq_codecarbon_kg` retains the original CodeCarbon total for reference. The difference between the two columns is examined in Chapter 6.

Inference latency is stored separately in `results/inference_time.csv`, as it is measured outside the emissions tracker and has a different structure. Like `results.csv`, it uses append-only writes via `csv.DictWriter`. Table 5.8 shows its columns.

Table 5.8: Column schema of `inference_time.csv`.

Column	Format	Description
`timestamp`	datetime	Date and time of the run
`model`	string	Model name
`dataset`	string	Dataset name
`nrows`	int / `all`	Subset size; `all` for full dataset
`inference_time`	float	Single-row prediction latency (s)
`co2eq_kg`	float	Corrected CO_2 of the training run
`cpu_power_inference_w`	float	Average CPU package power during inference loop (W)

Four core orchestration scripts coordinate the data collection phase of the experiment. Each model script is launched sequentially as an isolated subprocess rather than being imported directly. This is important because CodeCarbon operates at the Python process level: running multiple trackers within the same process risks state leakage between measurements. Subprocess isolation ensures that each tracker starts and stops cleanly in a fresh runtime environment. However, as noted in Section 5.5, the hardware sensors still measure whole-system power draw during this isolated execution, meaning Windows background activity remains part of the final recorded footprint.

`run_all.py` is the main entry point for the full cross-dataset benchmark. It runs in two sequential phases. Phase 1 executes all tuning scripts for Random Forest, XGBoost, and MLP across their respective datasets, saving the best parameters to `best_params.json`. Phase 2 then runs all five model scripts across all three datasets using those saved parameters (except Random Forest on the HIGGS dataset as mentioned earlier).

The remaining three scripts execute specific experimental variations. `run_xgb_breakeven.py` runs XGBoost CPU and GPU directly on nine HIGGS subset sizes to provide data for the hardware efficiency crossover analysis. `run_scaling_all_models.py` runs the remaining algorithms across designated HIGGS subset sizes to capture scaling behavior. Both subset scripts control the dataset size externally via the `TEST_NROWS` environment variable, which the data loader reads to draw a stratified sample. Finally, `run_mlp_variation.py` isolates the effect of NN complexity by training a grid of 12 MLP architectures on HIGGS, overriding the tuned hyperparameters with neutral defaults to ensure a fair comparison across different depths and widths.

5.8 Experiments

Having established the implementation and measurement infrastructure, this section describes the individual experiments conducted in this study. Each experiment is motivated by one or more of the research questions stated in Section 4.1 and addresses a specific dimension of the energy-performance trade-off.

5.8.1 Main Cross-Dataset Benchmark

The main cross-dataset benchmark forms the empirical backbone of this study. It trains all five models on all three datasets, with the single exception of Random Forest on HIGGS, which is excluded due to prohibitive runtimes at 11 million instances. The resulting 14 model–dataset combinations are evaluated under identical conditions: dataset-specific hyperparameters from the Optuna tuning runs, five-fold CV with a fixed random state, and the corrected emissions measurement pipeline described in Section 5.5. Logistic Regression is the only model not subject to hyperparameter tuning, as its role is that of a lightweight linear baseline whose value lies precisely in its simplicity.

For each of the 14 runs, accuracy and WF1 are averaged across the five folds, while CO_2 emissions and training time are recorded as totals over the full CV loop. Inference latency is measured separately on a single held-out sample after training. From these values, the EWF1 is computed during analysis and serves as the primary composite metric for comparing the performance-to-cost ratio across models and datasets.

The benchmark is designed to support comparison along two axes simultaneously. On the model complexity axis, ranging from Logistic Regression through the tree ensembles to MLP, it exposes at what point additional complexity stops producing accuracy gains that justify the associated energy cost. On the dataset scale axis, spanning three orders of magnitude from Wine to HIGGS, it reveals how the relative standing of models shifts as training data grows. The inclusion of both XGBoost CPU and XGBoost GPU as separate entries also directly feeds into the CPU versus GPU efficiency comparison developed in the scaling experiment.

5.8.2 Dataset Scaling Experiment

The main benchmark trains each model on fixed datasets, which makes it impossible to separate the effect of dataset scale from other algorithmic differences. The scaling experiment addresses this by holding everything constant except the number of training instances. HIGGS is the only dataset large enough to make this meaningful: with 11 million instances, it can be subsampled across a wide enough range for the scaling behavior to become visible. Wine and Credit are too small to show any trend across multiple subset sizes.

The experiment covers seven HIGGS subset sizes: 1,000, 10,000, 100,000, 500,000, 1,000,000, 5,000,000, and 11,000,000 instances. The range starts at 1,000 rows to capture the low-data regime where models may not yet converge and where resource costs are dominated by fixed overhead rather than computation. The upper endpoint is the full dataset. All five models are trained at the four smaller subset sizes (up to 500,000 instances). Random Forest is excluded from the three larger sizes (1,000,000 and above) because training times at that scale become prohibitive, consistent with its exclusion from the full HIGGS benchmark. Including it at the smaller subsets provides a direct view of where its scalability breaks down relative to the gradient-boosting methods, yielding 32 model–subset combinations in total. Consistent with the CV strategy established in Section 2.6, subsets are drawn via stratified sampling to preserve the original class distribution across all sizes.

The hyperparameters used at every subset size are those obtained from tuning on the full HIGGS dataset and are not re-optimised per subset. Re-tuning at each size would conflate scale effects with configuration effects and make it impossible to attribute observed differences to dataset size alone. The practical consequence is that at smaller subsets the hyperparameters are over-specified for the data, so performance figures at 1,000 or 10,000 instances reflect a model configured for 11 million rows, not the best achievable result at that size. The direction of this over-specification is conservative relative to the study's primary claims: at small scales, reported accuracy figures are lower bounds on what a properly tuned model could achieve. The experiment therefore neither overstates the accuracy advantage of larger data nor artificially flatters models at intermediate scales. The energy and emissions trends, which are the central focus of the analysis, are unaffected by this choice, since they reflect the cost of executing the fixed configuration and are not confounded by any tuning-induced variation.

For each combination, CO_2 emissions, training time, and WF1 are recorded using the same measurement pipeline as the main benchmark. Because XGBoost is the only model present in both a CPU and a GPU variant, the scaling data also directly supports the CPU versus GPU comparison: it shows at which dataset size the GPU's higher power draw starts to be offset by its faster training time, making the energy crossover visible as part of the broader scaling picture rather than as an isolated finding.

5.8.3 MLP Architecture Variation

The architectural variation experiment isolates the effect of MLP complexity on training emissions and predictive performance. The main benchmark uses one tuned architecture per dataset and therefore cannot answer how those metrics scale as layers and neurons are added. This experiment fills that gap by training the MLP on HIGGS across a systematic grid of 12 architectures: three widths (128, 256, and 512 neurons per layer) crossed with four depths (one, two, three, and four hidden layers). Each layer in a given configuration uses the same width, which keeps the design space tractable and ensures the depth and width axes can be interpreted independently of one another. The full grid is shown in Table 5.9 together with the resulting parameter counts on HIGGS (28 input features, 2 output classes), which range across more than two orders of magnitude.

Table 5.9: MLP architectures evaluated in the variation experiment, with trainable parameter counts for HIGGS. Counts include linear layer weights and biases.

Depth	**Width 128**	**Width 256**	**Width 512**
1 layer	3,970	7,938	15,874
2 layers	20,482	73,730	278,530
3 layers	36,994	139,522	541,186
4 layers	53,506	205,314	803,842

HIGGS is the only one of the three datasets where architectural variation has a meaningful effect to measure. Wine has too few instances to support a deep network at all, and Credit is structurally simple enough that even Logistic Regression achieves accuracy comparable to a tuned MLP, making any architecture a wash in performance terms. With 11 million instances and a non-trivial signal-versus-background classification task, HIGGS gives the network enough room to actually distinguish between configurations.

The non-architectural hyperparameters (learning rate, dropout rate, and batch size) are deliberately not taken from the HIGGS tuning results. The Optuna run optimised those values jointly with one specific architecture, so reusing them would silently favour that architecture against the rest of the grid. The experiment instead fixes neutral, architecture-independent defaults: a learning rate of 10^{-3} (the Adam default from Kingma and Ba), a dropout rate of 0.3 (a standard mid-range regularisation value), and a batch size of 4,096 (the middle value of the Optuna search space defined in Table 5.6). This choice may produce slightly suboptimal absolute results for any single configuration, but the relative comparison across the grid becomes considerably more defensible because no architecture starts with a hidden advantage from the tuning phase.

Each of the 12 runs follows the same measurement pipeline as the main benchmark: five-fold CV with the shared random state, CO_2 and training time tracked over the full CV loop, WF1 averaged across folds, and inference latency measured separately. The resulting data supports two orthogonal comparisons. Holding width constant while increasing depth reveals how layer stacking contributes to emissions and accuracy. Holding depth constant while increasing width reveals the same for neuron count. Together, these two views directly answer the research question of how energy consumption and CO_2 output scale with NN complexity.

5.8.4 Temporal Carbon Intensity Analysis

CodeCarbon converts measured energy into CO_2 using a static carbon intensity factor of 381 gCO_2eq/kWh for Germany. This single number averages out two sources of variation that the literature has identified as significant for the German grid: a strong diurnal cycle driven by solar production, and a seasonal swing driven by the winter coal share versus summer renewable share [8]. The static factor is therefore convenient but unrepresentative of the actual emissions of any training run that was not coincidentally executed at the annual mean intensity. This analysis quantifies how much the reported emissions of this study would have differed under realistic timing.

The experiment is purely retrospective and requires no retraining. A full year of hourly carbon intensity data for the German bidding zone is fetched from the ElectricityMaps API [61], covering 8,760 observations across all four seasons and the full diurnal cycle. The actual energy in kWh consumed by every training run is derived from the recorded CO_2 value by inverting the static factor:

$$E_{\text{kWh}} = \frac{\text{CO}_{2,\text{recorded}}}{0.381\,\text{kg/kWh}} \tag{5.1}$$

Each run's energy is then re-multiplied by carbon intensity values drawn from the API data to produce counterfactual emissions under the eight scenarios defined in Table 5.10. These scenarios isolate three different sources of variation: the global average across the full year, the extremes of best- and worst-case timing, and the structurally informed seasonal-diurnal combinations where renewables or fossil sources dominate.

Table 5.10: Carbon intensity scenarios used in the counterfactual analysis. Each scenario is computed from one year of hourly ElectricityMaps data for the German bidding zone.

Scenario	**Definition**
Static (CodeCarbon)	Fixed annual factor used by CodeCarbon, 381 g/kWh
Year mean	Mean over all 8,760 hourly observations
Year best hour	Single lowest-intensity hour of the year
Year worst hour	Single highest-intensity hour of the year
Winter mean	Mean over December, January, February
Summer mean	Mean over June, July, August
Summer midday	Mean of summer hours 11–14 UTC (peak solar)
Winter evening	Mean of winter hours 17–20 UTC (peak coal and gas)

Two questions guide the analysis. First, how large is the gap between the static-factor estimate and the realistic best- and worst-case timing for the same workload. If the gap is small, CodeCarbon's static factor is defensible as a practical approximation. If the gap is large, the reported emissions of this study should be read as one point on a wide distribution rather than as a fixed value. Second, the analysis identifies whether the diurnal or the seasonal axis dominates the variation, which has direct implications for practitioners deciding when to schedule large training jobs on local infrastructure. The script outputs a heatmap of mean intensity by hour-of-day and month, plus a per-run counterfactual CSV with the recomputed emissions for every scenario.

5.8.5 Lifecycle Break-even Analysis

The main benchmark measures the one-time training footprint of each model, but training CO_2 is only part of a model's total ecological cost. In production, a deployed model is queried repeatedly, and the cumulative inference footprint grows linearly with the number of predictions made. The lifecycle analysis quantifies at what prediction volume this cumulative inference CO_2 surpasses the initial training footprint, and how this break-even point differs across model families.

The analysis is purely post-hoc and requires no additional training runs. It combines two data sources already collected during the main benchmark: the corrected training CO_2 from `results.csv` and the median single-row inference latency from `inference_time.csv`. The CO_2 emitted per single prediction is estimated as:

$$\mathrm{CO}_{2,\text{inference}} = \frac{t_{\text{inference}} \times P_{\text{infer}}}{3{,}600{,}000} \times I_{\text{carbon}} \tag{5.2}$$

where $t_{\text{inference}}$ is the median inference latency in seconds, P_{infer} is the average CPU package power in watts measured directly during the inference timing loop, and I_{carbon} is the carbon intensity in $\mathrm{kg\,CO_2/kWh}$ derived from the training measurements as $I_{\text{carbon}} = \mathrm{CO}_{2,\text{train}}/E_{\text{train}}$. The break-even prediction count follows directly as $N_{\text{break-even}} = \mathrm{CO}_{2,\text{train}}/\mathrm{CO}_{2,\text{inference}}$.

Unlike earlier approaches that approximate inference power via the training-phase CPU measurement, this study uses P_{infer} obtained from a dedicated `CPUPowerMonitor` session restricted to the 100-repetition inference loop. Inference involves only a forward pass with no gradient computation or tree-building, and therefore draws substantially less power than training. Using training power as a proxy would overestimate per-prediction energy and undercount the break-even threshold; the direct measurement avoids this distortion.

6 Results and Discussion

Across 317 CodeCarbon-tracked runs conducted between March and May 2026, the experiments in this study consumed approximately 10.7 kWh of electricity and 151.7 hours of compute time, with CodeCarbon logging a cumulative 4.06 kgCO_2eq, equivalent to driving approximately 35 km in an average German car [62] or charging a smartphone roughly 535 times on the German grid. Given the systematic underestimation documented in Section 6.1, the corrected total is substantially higher. The 4.06 kg should be read as a lower bound. Hyperparameter tuning for the MLP on HIGGS alone accounts for 2.47 kg of that figure, more than 60 % of the logged total, illustrating that the experimental overhead of finding a competitive configuration can dwarf the cost of the benchmark runs themselves.

The chapter is organised in eight sections that together address the five sub-questions set out in Section 4.1. Section 6.1 first establishes the hardware-corrected measurement baseline on which all subsequent comparisons rest; Section 6.2 then quantifies the tuning overhead that standard benchmark tables omit. The main analysis proceeds in four thematic blocks: the cross-dataset benchmark (Section 6.3), the CPU–GPU hardware comparison (Section 6.4), the HIGGS scaling and MLP architecture variation experiments (Sections 6.5 and 6.6), and finally the temporal and lifecycle analyses (Sections 6.7 and 6.8) that extend the efficiency picture beyond training time alone.

6.1 Emissions Measurement Validation

Any comparison of model efficiencies is only as valid as the measurements it rests on. As described in Section 5.5, CodeCarbon's TDP-based CPU estimate was replaced in this study by direct hardware-sensor readings. Figure 6.1 shows the ratio between the two for every model-dataset combination.

Every one of the 14 model-dataset combinations shows a hardware-corrected value that exceeds the raw CodeCarbon figure. The ratio ranges from 1.32 to 2.63 with a median of 1.87, meaning CodeCarbon, as deployed without modification, would have captured on average only about 54 % of the true emissions value. Fischer [45] found a consistent underestimation of 20 to 30% when validating CodeCarbon against external power meters across a range of workloads. The CPU-intensive runs in this study exceed even that range. Logistic Regression on Wine reaches a ratio of 2.62, meaning CodeCarbon captured less than 40% of the actual energy. The cause is the TDP fallback described in Section 5.5. Where RAPL is unavailable, CodeCarbon estimates CPU power from the thermal design envelope rather than measuring it, and that envelope does not track how the chip actually behaves under load [6].

The error is not uniformly distributed, and two structural factors account for most of the

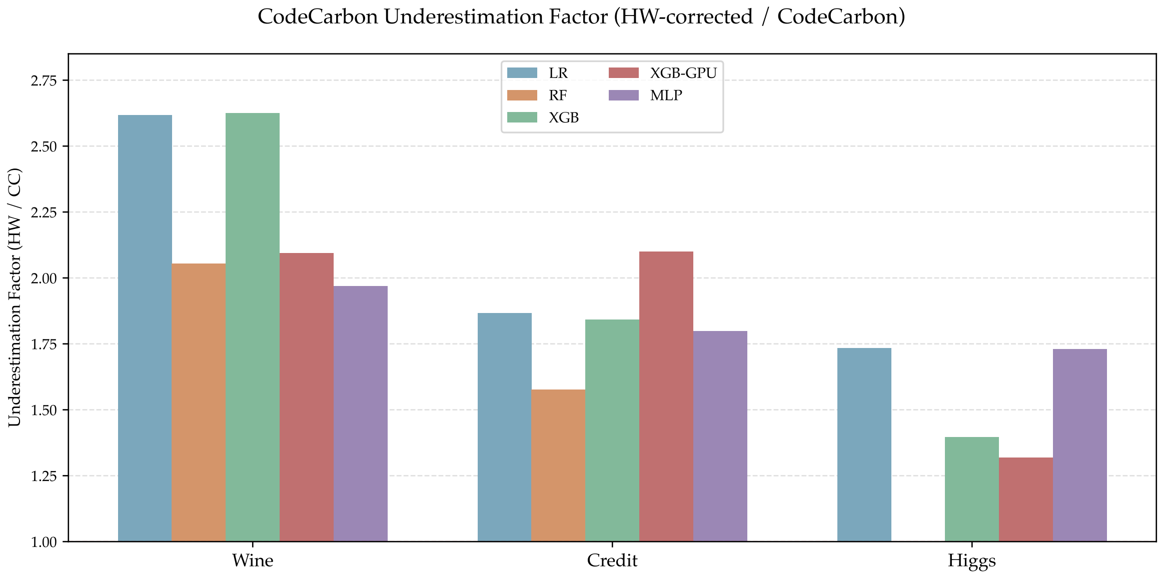


Figure 6.1: CodeCarbon underestimation factor (hardware-corrected / CodeCarbon) per model and dataset. All values exceed 1, confirming systematic undercounting. Higher factors for Wine and Credit reflect TDP-based estimation being least accurate on short training runs. Random Forest has no HIGGS measurement.

variation. Short runs are hit hardest. Wine combinations, where training times range from 0.04 to 3.35 seconds, show a mean ratio of 2.27, while HIGGS combinations (90 to 11,175 seconds) land at 1.55. On a sub-second run the TDP-based integrator barely accumulates anything while the chip executes a real burst of work. As duration grows and load sustains, the heuristic becomes a closer approximation and the ratio shrinks. The GPU share is the second factor. XGBoost GPU on HIGGS, where the GPU accounts for 58 % of total energy and the CPU for only 31 %, produces the lowest ratio at 1.32. Since the correction touches only the CPU slice, a smaller CPU fraction produces a smaller total distortion even when the underlying power discrepancy is comparable to other runs. Fischer [45] document the same asymmetry, finding that CPU-only workloads showed errors exceeding GPU equivalents by more than an order of magnitude in some configurations.

The measurement error is largest for the runs that correspond to the algorithms Green AI positions as the efficient choice. Logistic Regression on Wine carries a ratio of 2.62, XGBoost CPU on Wine 2.63. These are the models a practitioner would reach for to minimize ecological footprint, and they are precisely the ones where TDP-based estimation fails most severely. Without correction, they would appear even more energy-efficient relative to the heavier workloads than the corrected figures show, because the measurement error and the efficiency claim point in the same direction. The efficiency advantage of lightweight models is confirmed by the corrected data, but the gap is smaller than uncorrected measurements would suggest. All emissions values in the remainder of this chapter use the hardware-corrected figures. The relative rankings are unchanged, but the absolute scale is accurate. Published studies relying on uncorrected CodeCarbon values on comparable hardware should be read as reporting lower bounds rather than ground truth.

6.2 Hyperparameter Tuning Overhead

Benchmark tables that compare training emissions report only a fraction of a model's true ecological initialization cost. Before a single benchmark run is executed, each model must be brought to a competitive configuration through hyperparameter search, and the footprint of that search is never reflected in the training figure. For computationally inexpensive models on small datasets this omission is negligible. For the MLP on HIGGS it is not: as shown in Figure 6.2, a deliberately constrained 40-trial TPE search required 95.4 hours and emitted 2.15 kg of CO_2, equivalent to approximately 18 km of car travel [62] or roughly 283 full smartphone charges on the German grid, a figure that exceeds the combined training emissions of all Wine and Credit runs in this study. This is not the outcome of an exhaustive search of the kind documented by Strubell et al. [3], who found that large NLP development pipelines required hundreds of thousands of GPU hours to converge. It is the cost of the minimum credible search budget, chosen specifically to limit overhead.

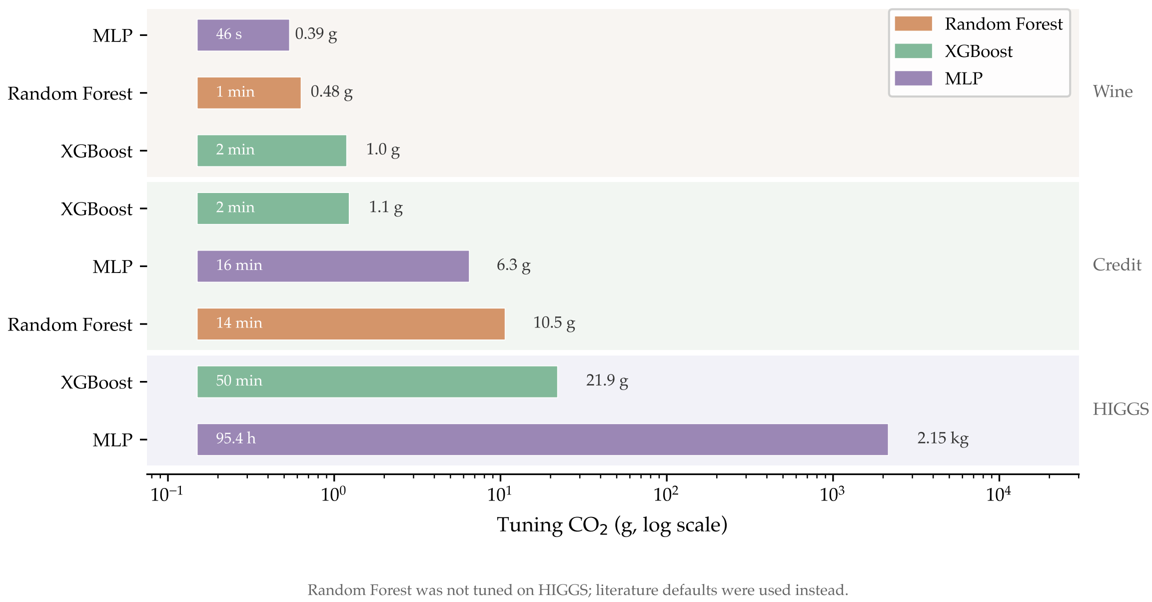


Figure 6.2: CO_2 emissions and duration of every Optuna tuning study (40 trials each). Bars show tuning duration. CO_2 totals are annotated to the right. Random Forest was not tuned on HIGGS. Literature defaults were used instead.

How expensive that search becomes is structurally determined by how well an algorithm's training procedure maps onto available hardware during each trial evaluation. On the Credit dataset, this difference is already quantifiable. XGBoost completed its tuning study in 130 seconds and 1.08 g CO_2, accelerating each trial on GPU. Random Forest arrived at a functionally identical result, a WF1 of 0.800 against XGBoost's 0.801, in 815 seconds and 10.5 g CO_2: ten times the tuning cost for a predictive difference that is statistically negligible. The gap reflects an architectural property that scales with search rather than just with training: constructing an ensemble of independent decision trees requires sequential or coarse-grained parallel computation that does not exploit GPU cores. Every trial inherits that cost, and forty trials compound it. This is why Random Forest was excluded from HIGGS

Table 6.1: Hyperparameter tuning cost per model and dataset (40 Optuna TPE trials each). Duration in hours; CO_2 is hardware-corrected. Best WF1 is the highest achieved across all 40 trials. Logistic Regression used sklearn defaults and was not tuned. Random Forest was not tuned on HIGGS; literature defaults were used instead.

Model	Dataset	Duration (h)	CO_2 (g)	Best WF1
Random Forest	Wine	0.02	0.48	0.983
	Credit	0.23	10.51	0.800
XGBoost	Wine	0.03	1.04	0.989
	Credit	0.04	1.08	0.801
	HIGGS	0.83	21.89	0.760
MLP	Wine	0.01	0.39	0.994
	Credit	0.26	6.34	0.804
	HIGGS	95.40	2,149.3	0.787

tuning altogether, with literature defaults substituted instead. Subjecting that per-trial cost to eleven million rows across forty iterations would have been ecologically unjustifiable relative to any predictive gain it could have delivered, an assessment the scaling analysis in Section 6.5 corroborates.

The consequence for how the benchmark results in the following sections should be interpreted is direct. The training emissions in the benchmark table represent the cost of a single run with already-optimized hyperparameters. For the HIGGS MLP, the initialization cost that precedes that run amounts to 2.15 kg against a training footprint of 145.9 g: tuning is roughly 15 times costlier than the benchmark run it makes possible. That debt does not reset at deployment. The nested CV approach rejected on ecological grounds in Section 4.6 would have multiplied these costs fivefold, adding weeks of additional compute on HIGGS alone. The 40-trial single-loop design is the result of an active ecological trade-off, and the 2.15 kg figure is the lower bound of what responsible tuning entails at this scale. Whether the MLP's efficient inference architecture can serve enough predictions to offset that initialization cost over a realistic deployment lifetime is a question the lifecycle analysis in Section 6.8 addresses directly.

6.3 Main Cross-Dataset Benchmark

The benchmark spans 14 model-dataset combinations across three datasets that differ by three orders of magnitude in size. Figures 6.3 and 6.4 lay out the two dimensions of the comparison. Figure 6.5 synthesizes them into the EWF1 ranking. The central finding is that no model dominates across all three settings. Ecological efficiency is not a fixed property of an algorithm. It emerges from the alignment between a model's computational profile, the hardware it runs on, and the scale of the task it faces. Each of the three datasets exposes a different axis of that contingency.

Table 6.2: Main benchmark results for all 14 model-dataset combinations. CO_2 is the hardware-corrected training footprint in grams. Training time is in seconds. EWF1 is computed with $\lambda = 1$ using min-max normalized emissions per dataset (see Section 4.4). Bold marks the best value per column per dataset; for CO_2 and time, lowest is best. Random Forest was not run on HIGGS.

Dataset	Model	Acc.	WF1	CO_2 (g)	Time (s)	EWF1
Wine	Logistic Regression	**0.989**	**0.989**	**0.014**	**0.04**	**0.989**
	Random Forest	0.983	0.983	0.029	1.3	0.736
	XGBoost (CPU)	**0.989**	**0.989**	0.020	0.4	0.860
	XGBoost (GPU)	**0.989**	**0.989**	0.051	2.4	0.539
	MLP	**0.989**	**0.989**	0.059	3.4	0.494
Credit	Logistic Regression	0.810	0.771	0.061	3.9	0.755
	Random Forest	0.821	0.800	0.832	41.7	0.400
	XGBoost (CPU)	0.821	0.800	**0.043**	**1.5**	**0.800**
	XGBoost (GPU)	**0.822**	0.801	0.052	2.4	0.792
	MLP	0.818	**0.804**	0.633	47.6	0.460
HIGGS	Logistic Regression	0.642	0.637	5.148	436	0.624
	XGBoost (CPU)	0.760	0.760	14.650	828	0.699
	XGBoost (GPU)	0.760	0.760	**2.104**	**90**	**0.760**
	MLP	**0.788**	**0.787**	145.891	11,175	0.394

The Low-Data Regime: When Hardware Overhead Dominates Focusing first on the low-data regime, the leftmost cluster in Figure 6.3 reveals that four of the five models reach a WF1 of 0.989 on the Wine dataset. Random Forest at 0.983 is the only outlier, and the margin carries no practical significance. With predictive differentiation absent, the entire comparison reduces to emissions, where Figure 6.4 shows a fourfold spread across models that deliver the exact same outcome. The most striking observation within that spread is that XGBoost GPU emits 2.5 times as much CO_2 as XGBoost CPU for an identical result.

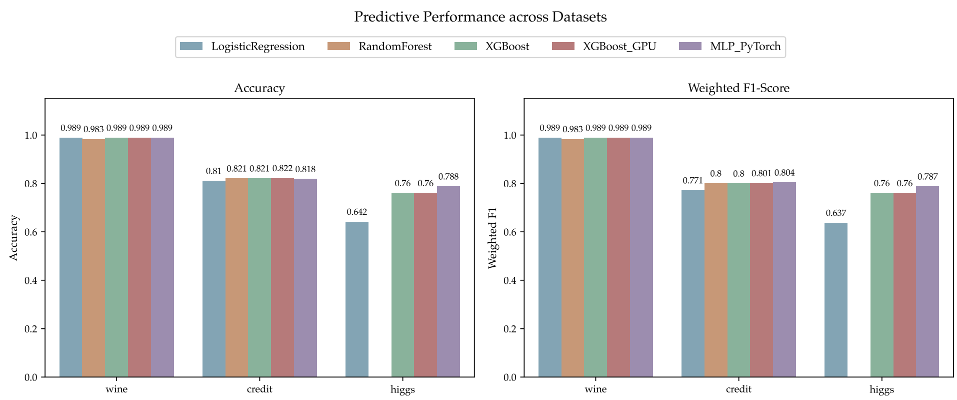


Figure 6.3: Predictive performance (accuracy and weighted F1-score) across all 14 model-dataset combinations. On Wine, four of five models converge on a WF1 of 0.989, making emissions the only meaningful dimension of comparison. On HIGGS, Logistic Regression saturates well below the non-linear models, and MLP achieves the highest single result in the study.

The mechanism behind this reversal is the fixed cost of GPU initialization and data movement. On a dataset of 178 rows, a Wine training run completes in fractions of a second.

Device initialization and host-to-device memory transfers via the PCIe interface consume a fixed block of time and energy regardless of how little data follows. Consequently, the thousands of parallel cores the GPU provides remain almost entirely idle for the actual computation. The overhead dominates before the workload can even begin. This is the structural condition under which SIMT parallelism becomes a liability rather than an advantage: when data volume is too small to saturate the available multiprocessors, the architecture's fixed entry costs cannot be amortized.

The EWF1 metric is designed to surface exactly this kind of disproportionality. Logistic Regression, which requires no GPU and imposes the lowest computational overhead, leads the EWF1 ranking at 0.989. This result perfectly illustrates a core principle of Green AI: applying complex and power-hungry hardware to a problem that a linear model solves equally well is not merely suboptimal but ecologically wasteful.

The Medium-Data Regime: Algorithmic Efficiency Disparities Moving to the medium-scale Credit dataset (center cluster in Figure 6.3 and Figure 6.4), genuine predictive differentiation appears for the first time. MLP achieves the highest WF1 at 0.804, and XGBoost CPU reaches 0.800 at a training cost of 43.3 mg CO_2. The critical data point, however, is Random Forest. It matches XGBoost CPU's WF1 to three decimal places (0.800 vs 0.800) while incurring nearly twenty times the carbon cost. Two models, statistically indistinguishable predictive outcomes, and an order-of-magnitude difference in ecological cost.

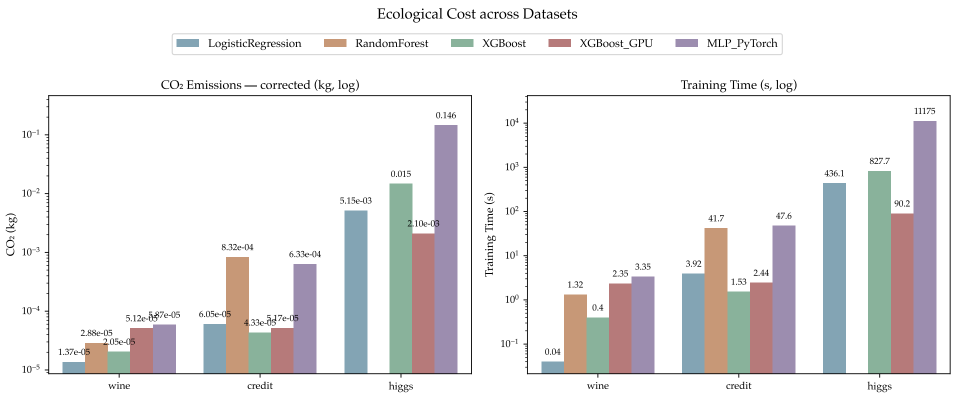


Figure 6.4: Hardware-corrected CO_2 emissions and training time (log scale) across all 14 model-dataset combinations.

The explanation is algorithmic rather than incidental. Random Forest constructs an ensemble of independently grown decision trees, evaluating candidate splits at each node across every tree in full. XGBoost's boosted tree construction, by contrast, uses highly-optimized histogram-based split-finding and grows trees sequentially, each correcting the residual errors of the last. This procedure achieves equivalent predictive capacity at a fraction of the compute. Crucially, this is the same structural property that inflated Random Forest's tuning cost in Section 6.2: the per-tree overhead that makes tuning expensive makes training expensive for identical reasons, and no amount of hyperparameter optimisation removes it.

The EWF1 metric captures this disproportionality directly. Random Forest drops to last place at 0.400, the steepest single-model penalty in the entire benchmark. XGBoost CPU, delivering equivalent accuracy at a fraction of the cost, takes first at 0.800. Two results further down the ranking are also worth noting. Logistic Regression, despite posting the lowest predictive score on this dataset (WF1 of 0.771), ranks second in EWF1 at 0.755: its trivially cheap training cost outweighs its predictive disadvantage. MLP, conversely, achieves the best WF1 at 0.804, but emits 633 mg CO_2, nearly fifteen times XGBoost CPU's 43 mg, for a predictive gain of just 0.004 points. Its EWF1 (0.460) falls well below both XGBoost variants. The Credit ranking therefore illustrates two complementary efficiency failure modes: the severe algorithmic overhead of Random Forest at the bottom, and the disproportionate cost of marginal neural-network gains in the middle. The implication extends beyond this dataset. When algorithm families differ structurally in computational profile, choosing the right family is at least as consequential for ecological efficiency as any hardware decision [2].

The High-Data Regime: Hardware Saturation and Role Reversals Finally, as shown in the rightmost clusters of Figure 6.3 and Figure 6.4, both dimensions of the comparison shift decisively at eleven million rows. Random Forest is excluded from this dataset for the reasons established in Section 6.2: the per-trial training cost made a full 40-trial HIGGS run ecologically unjustifiable. Logistic Regression saturates at a WF1 of 0.637, a performance ceiling that additional data cannot raise, as the scaling analysis in Section 6.5 confirms. Among non-linear models, MLP achieves the highest WF1 in the entire benchmark at 0.787, but its training footprint of 145.9 g CO_2 is the most expensive single run in the study. Most strikingly, XGBoost GPU emits one seventh of what XGBoost CPU does for an equivalent WF1 of 0.760. On Wine, the GPU was a liability. On HIGGS, it is the ecologically dominant choice.

The reversal follows directly from the SIMT execution model discussed in Chapter 2. At the scale of eleven million rows, the data volume finally saturates the GPU's parallel cores. The fixed costs of device initialization and memory transfer that dominated the Wine result are now fully amortized across the dataset. XGBoost's histogram-based tree construction maps naturally onto this architecture, distributing split-finding workload across thousands of cores in a way that sequential CPU-bound computation cannot match. Logistic Regression's ceiling, by contrast, is independent of hardware: a linear decision boundary cannot partition the HIGGS feature space with sufficient fidelity regardless of the compute invested. The MLP, meanwhile, requires iterative backpropagation passes over eleven million samples, driving its energy consumption to the highest single-run cost in the entire benchmark.

The EWF1 ranking on HIGGS inverts relative to raw F1. XGBoost GPU leads at 0.760, and MLP, despite its predictive advantage, finishes last at 0.394. The min-max normalized emissions penalty in the EWF1 formula assigns the MLP the maximum relative cost, as its training footprint dwarfs every other model on HIGGS, and its predictive lead cannot compensate for it. Whether deployment-scale inference changes this assessment is the subject of Section 6.8.

Synthesis: The Contingency of Ecological Efficiency No single model leads across all three datasets. Logistic Regression wins on Wine, where a linear boundary suffices and hardware overhead dominates. XGBoost CPU wins on Credit, where the right algorithm family delivers equivalent accuracy at a fraction of the cost of its nearest competitor. XGBoost GPU wins on HIGGS, where dataset scale finally amortizes the fixed costs of GPU execution.

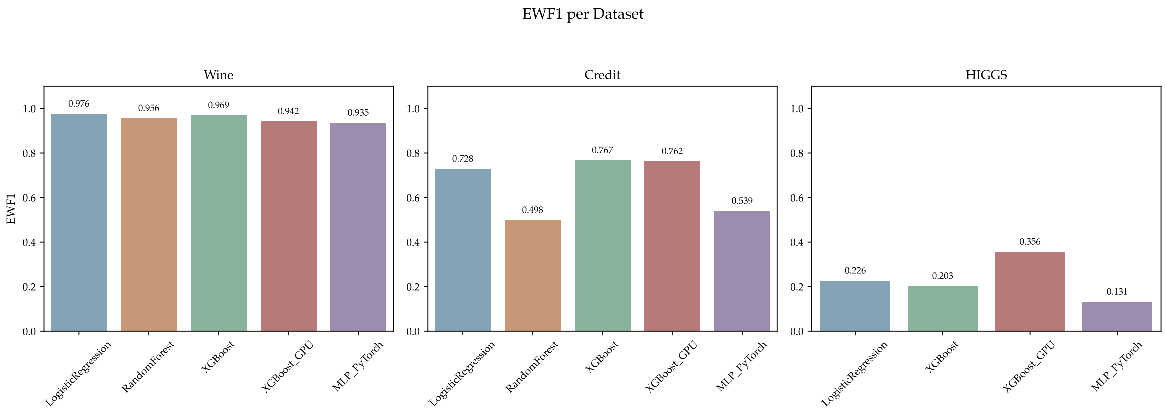


Figure 6.5: EWF1 per dataset. Rankings diverge from raw F1 wherever emissions cost is disproportionate to the predictive gain.

Figure 6.5 makes this contingency visible in a single figure. The EWF1 rankings shift substantially between datasets, and in each case the shift is mechanistically traceable: to hardware initialization overhead at small scale, to algorithmic structure at medium scale, and to SIMT saturation at large scale. The composite metric does not merely reorder a static ranking. It captures a different cost regime at each level of the analysis. This is the empirical answer to the study's primary research question. Ecological efficiency is not a property that any model either has or lacks. It is the dynamic outcome of a specific alignment between computational profile, hardware, and task scale [47].

6.4 XGBoost: CPU vs. GPU

As established in the main benchmark, hardware acceleration is not unconditionally efficient. Figure 6.6 isolates the performance of XGBoost on CPU and GPU across all three datasets, revealing a contrast in ecological cost that shifts fundamentally with dataset scale.

On Wine and Credit, GPU acceleration provides no emissions benefit, but rather an ecological liability. XGBoost GPU emits 51.2 mg CO_2 on Wine against the CPU's 20.5 mg, a 2.5-fold increase at no predictive benefit. Training time mirrors this: 2.35 seconds on GPU versus 0.40 seconds on CPU. This inversion occurs because the fixed overhead of device initialization and host-to-device memory transfer via the PCIe interface completely dominates the actual computation time, as analysed in Section 2.3.7. The thousands of available cores remain starved of data, leaving the GPU's higher power draw unamortized. On Credit the gap narrows: 51.7 mg (GPU) against 43.3 mg (CPU), a 19 percent premium, but GPU execution still fails to pay for itself.

The HIGGS result reverses the picture completely. XGBoost GPU emits 2.10 g CO_2 and completes in 90.2 seconds. XGBoost CPU emits 14.65 g and takes 827.7 seconds. The GPU

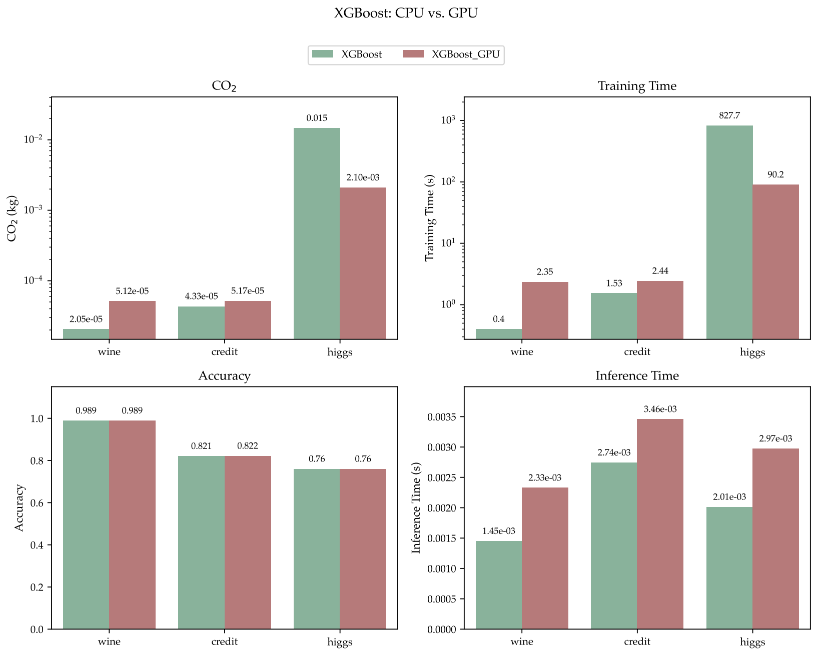


Figure 6.6: XGBoost CPU vs. GPU across datasets. Top row: CO_2 emissions and training time (log scale). Bottom row: accuracy and per-sample inference time. Accuracy is near-identical across variants on all three datasets; the ecological and speed differences are the operative comparison.

variant is therefore seven times more emissions-efficient and nine times faster. To put the absolute scale in perspective: the GPU's 2.10 g CO_2 is roughly equivalent to a third of a smartphone charge, and the CPU's 14.65 g to about two full charges. At eleven million rows the data volume finally saturates the GPU's SIMT architecture [35]. The histogram-based tree construction distributes efficiently across thousands of cores, and the fixed memory transfer costs that dominated at small scale are now fully amortized by the parallelization gains.

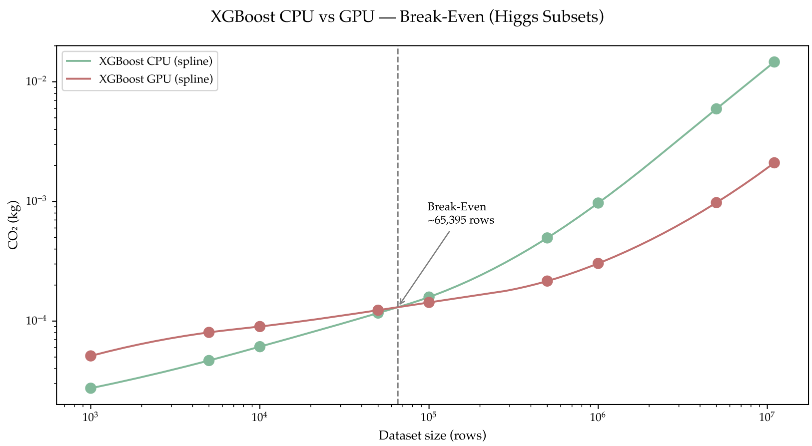


Figure 6.7: XGBoost break-even point comparison between different subset sizes of the HIGGS dataset.

To pinpoint where this shift occurs, Figure 6.7 maps the emissions crossover across intermediate HIGGS subsets (explored further in Section 6.5). The analysis identifies a hard break-even threshold at approximately 65,000 rows. Below this volume, the GPU's higher instantaneous power draw remains unamortized, transforming the accelerator into an ecological penalty. Above it, the compounding speedup of massive parallelisation outpaces the wattage premium, and the emissions gap widens favorably with each additional row.

This finding provides a direct, empirical answer to the study's research question regarding hardware efficiency: the ecological viability of GPU acceleration on tabular data is strictly bounded by dataset scale. More importantly, it challenges a pervasive assumption in modern machine learning. In practice, GPU execution is frequently treated as a universal default: a switch flipped to "accelerate" computation regardless of the task. The evidence here demonstrates that for tabular data, this default is fundamentally flawed. Blindly applying a GPU to a dataset of ten thousand rows does not accelerate learning. It merely burns energy to initialize a device and transfer data across a PCIe bus. True Green AI requires practitioners to actively break this habit and align their hardware choices with the complexity of their workloads, treating accelerators not as an unconditional default, but as a specialised tool justified only by sufficient scale.

6.5 HIGGS Dataset Scaling

Figure 6.8 tracks CO_2 emissions and predictive performance across nine HIGGS subsets ranging from 1,000 to 11,000,000 rows. By holding the dataset complexity constant and isolating the volume of data, the scaling analysis exposes the exact thresholds at which the ecological cost of processing more data eclipses the predictive benefit.

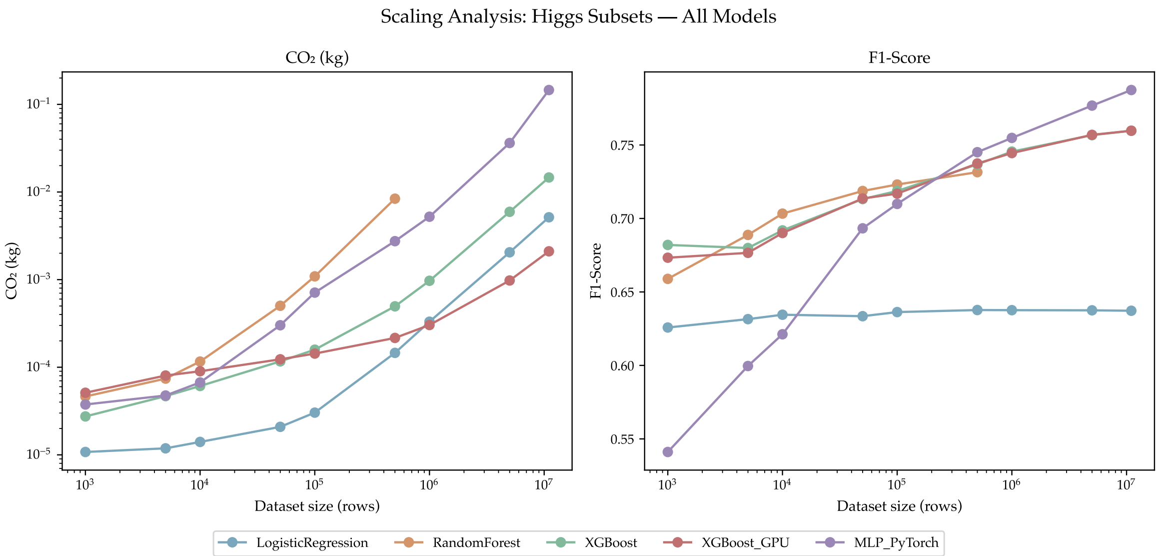


Figure 6.8: CO_2 emissions (log scale) and weighted F1-score across nine HIGGS subsets for all models. Random Forest excluded at full scale; XGBoost GPU break-even visible around 500,000 rows.

Capacity Saturation in Simple Models At the lower end of the scaling curve, the linear boundary of Logistic Regression hits a hard capacity ceiling remarkably early. The model's

WF1 improves from 0.626 at 1,000 rows to 0.636 at 100,000 rows, but remains essentially flat across all subsequent, larger subsets. Because the model lacks the representational capacity to capture the non-linear intricacies of the HIGGS dataset, additional data yields virtually no predictive return. Consequently, any compute expended to train this model beyond 100,000 rows is ecologically wasted, demonstrating that data volume must always be matched to algorithmic capacity.

The Architectural Bottleneck of Ensembles As the dataset scales to 500,000 rows, a striking reversal in the expected cost of model complexity emerges. Random Forest, despite its conceptual simplicity relative to deep learning, becomes the most ecologically expensive model in the study. At this scale, Random Forest emits 8.39 g CO_2 to achieve a WF1 of 0.732. In stark contrast, the MLP achieves a higher WF1 of 0.745 while emitting less than one third of the carbon (2.75 g), and XGBoost CPU achieves 0.737 for just 0.50 g. Random Forest is ultimately excluded from runs above 500,000 rows due to prohibitive training times. The underlying reason is architectural: training an ensemble of independent, fully-grown decision trees relies on coarse-grained parallelism that scales super-linearly with data volume and cannot easily exploit GPU acceleration (see Section 2.3.3). This proves that a model's ecological footprint is not determined by its abstract mathematical complexity, but by how efficiently its training procedure maps onto available hardware parallelism.

The Law of Diminishing Returns In the high-data regime above 1,000,000 rows, all non-linear models face the fundamental tension of empirical learning curves (introduced in Section 2.4): predictive performance scales sub-linearly, while energy consumption scales linearly or worse. The MLP illustrates this trade-off most dramatically. Expanding its training data from 1 million to 11 million rows yields the largest absolute performance gain in this regime, improving the WF1 from 0.755 to 0.787. However, this 3 percentage point improvement requires a 28-fold increase in CO_2 emissions. Conversely, XGBoost GPU navigates this regime with extreme efficiency, reaching a WF1 of 0.760 at 11 million rows while emitting only 2.10 g CO_2, roughly one seventh of its CPU counterpart (14.65 g).

Synthesis: Rethinking the Big Data Dogma Assessed through the EWF1 lens, the diverse scaling patterns compress into a single, clarifying trend: as dataset size grows, the emissions gap between hardware-aligned models and inefficient algorithms drastically widens. XGBoost GPU sustains the highest EWF1 across every scale above its hardware break-even point, proving that large-scale learning is only ecologically viable when software is perfectly aligned with SIMT acceleration. At five million rows it reaches an EWF1 of 0.757 against 0.663 for XGBoost CPU at the same F1 of 0.757, a gap that widens further at the full eleven million row dataset.

More profoundly, these scaling curves challenge the pervasive assumption that more data is unconditionally better, which drives modern machine learning. In the pursuit of state-of-the-art accuracy, the default industrial practice is to feed algorithms the largest dataset available [1]. The empirical evidence here proves that this practice is often an ecological liability. At the tail end of a learning curve, forcing a model to ingest ten million additional rows to extract a marginal gain in F1-score results in a disproportionate explosion of carbon emissions. True Green AI requires practitioners to abandon the assumption that all available data must be used. Instead, as recent data-centric AI studies argue, we must proactively identify the threshold where predictive returns diminish [2], treating dataset size not as a fixed asset, but as a hyperparameter that must be aggressively tuned to halt training before the ecological cost eclipses the scientific value.

6.6 MLP Architecture Variation

Figure 6.9 shows weighted F1 and CO_2 emissions for all twelve MLP configurations (depth $\in \{1, 2, 3, 4\}$, width $\in \{128, 256, 512\}$) trained on the full HIGGS dataset.

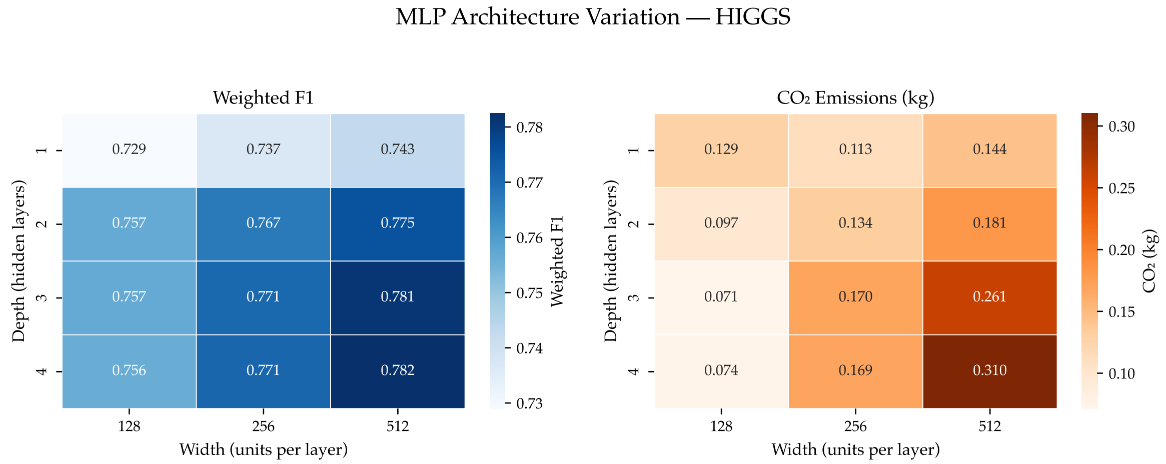


Figure 6.9: MLP architecture variation on HIGGS: weighted F1 (left) and CO_2 emissions (right) for all depth-width combinations. Darker cells indicate higher values.

The variation grid reveals that width has a substantially stronger effect on performance than depth. Moving from 128 to 512 units per layer increases WF1 by up to 0.026 across all depth levels, whereas adding layers beyond two yields marginal gains of at most 0.007 at the widest setting and near zero for narrower networks. The predictive ceiling is reached early: the 2×512 architecture achieves a WF1 of 0.776, trailing the best-observed configuration (4×512, WF1 = 0.782) by only 0.007.

The CO_2 picture is more nuanced. For narrow networks (width = 128), emissions actually *decrease* as the network gets deeper, falling from 0.129 kg at depth 1 to 0.071 kg at depth 3. This reversal is mechanically driven by early stopping. Deep, narrow networks lack the representational capacity required for complex tabular features, causing them to rapidly overfit and trigger the early stopping patience criterion in fewer epochs. Consequently, the total compute per training run is lower despite the larger parameter count. For wider networks, this effect vanishes. At width = 512, emissions increase monotonically with depth from 0.144 kg to 0.310 kg, as the network successfully utilizes its capacity and trains for more epochs before saturating.

Taken together, these results highlight a structural mismatch between standard deep learning practice and tabular data. In computer vision or natural language processing, stacking layers is necessary to extract hierarchical spatial or sequential representations. Tabular datasets like HIGGS lack this strict local hierarchy, so excessive depth provides almost no predictive value and functions primarily as a computational overhead. The 4×512 configuration illustrates this directly: achieving the final 0.007 improvement in WF1 required scaling to maximum depth and width, nearly doubling the CO_2 footprint from 0.181 kg to 0.310 kg for a gain well within normal run-to-run variance. The data here corroborate the broader Green AI observation that predictive returns on model scale follow diminishing returns, while energy costs do not [1].

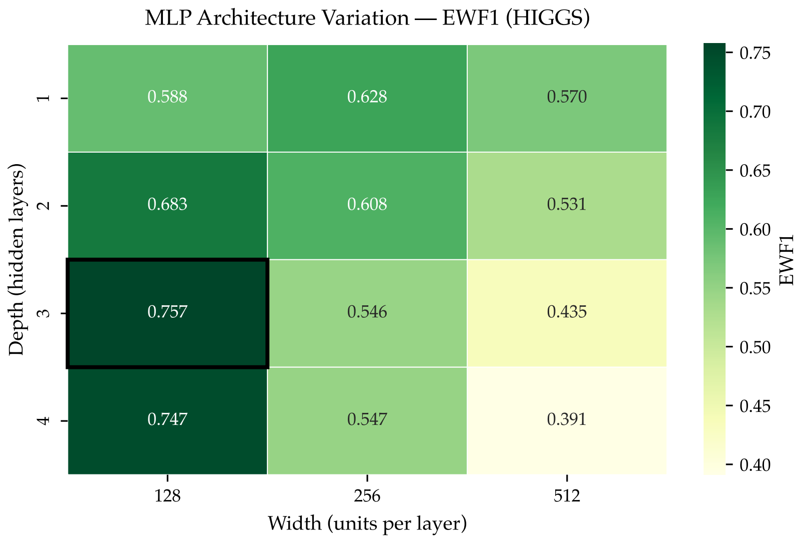
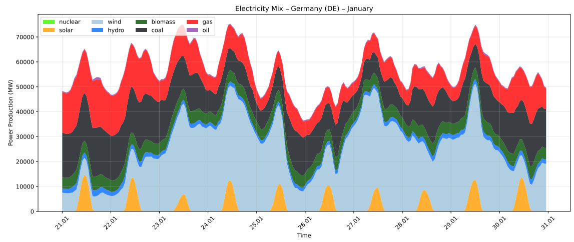


Figure 6.10: MLP architecture variation on HIGGS: Efficiency-Weighted F1 (EWF1) for all depth-width combinations. Darker cells indicate higher values.

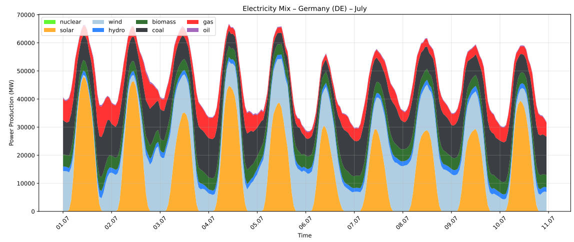
Evaluating the twelve configurations through the EWF1 lens, as seen in Figure 6.10 makes this trade-off explicit. The min-max normalized emissions penalty favors shallower, narrower architectures whose modest accuracy reductions are more than offset by their lower training costs. The 3×128 configuration achieves the highest EWF1 at 0.757, entirely displacing the 4×512 configuration that posted the highest raw WF1 but only an EWF1 of 0.391. For neural networks applied to tabular data, this suggests treating depth not as a default but as a resource justified only by proportional predictive returns.

6.7 Carbon Intensity Analysis

The analyses conducted in the preceding sections treat CO_2 emissions as a fixed function of the training run: given the same model, dataset, and hardware, the reported figure does not change. This is a consequence of how CodeCarbon works: it converts measured energy into CO_2 using a static intensity factor of 381 gCO_2eq/kWh for Germany, a calculated average that cannot reflect when a run actually takes place (see Section 2.2.4). The German grid, however, draws on a fundamentally different fuel mix in January than in July, and during peak evening demand than during solar-rich midday hours, a structure visible directly in the hourly ElectricityMaps data used in this analysis.



(a) January 2025 (21–30 Jan). Gas, coal, and nuclear dominate, with solar generation near zero.



(b) July 2025 (1–10 Jul). Solar becomes the dominant midday source, sharply reducing the fossil fuel share during daylight hours.

Figure 6.11: Electricity production mix in Germany for representative ten-day windows in winter and summer.

Figures 6.11a and 6.11b illustrate this contrast directly. In January, dispatchable sources (gas, coal, and nuclear) form the base of the stack, with wind providing a variable but significant contribution and solar generation remaining negligible throughout the day. In July, the picture reverses: solar becomes the dominant midday source, regularly exceeding 20,000 MW, while gas and coal drop to a much smaller share. It is this structural shift in fuel composition that drives the seasonal variation in carbon intensity quantified in the analysis below.

If this temporal variation is large enough, then the timing of a training run is itself a meaningful ecological decision, one that requires no changes to code, hardware, or model architecture. This section evaluates that hypothesis directly by counterfactually re-assigning the energy of every training run in this study to different timing scenarios derived from one year of hourly ElectricityMaps data, as described in Section 5.8.4.

Annual Spread and Scenario Intensities The first result is somewhat reassuring for the existing measurements: as shown in Figure 6.12, the static CodeCarbon factor of 381 gCO_2eq/kWh is a defensible approximation of the annual mean, which the empirical data places at 341 gCO_2eq/kWh. A workload executed at a uniformly random point during the year would therefore be assigned only a modest systematic error by CodeCarbon. The more important finding is the spread around that average. The best recorded hour of 2024 reaches 115 gCO_2eq/kWh, and the worst reaches 658 gCO_2eq/kWh, a factor of 5.7$\times$ between two hours. The absolute extremes are not practically exploitable, since no practitioner can guarantee their run coincides with the single cleanest or dirtiest hour of the year. The structurally informed seasonal-diurnal combinations are the actionable range: summer midday hours average 193 gCO_2eq/kWh while winter evening hours average 389 gCO_2eq/kWh, a factor of two achievable through deliberate scheduling alone.

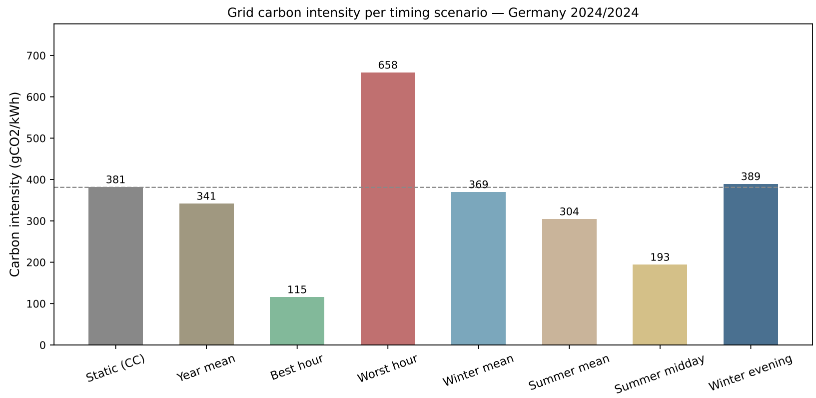


Figure 6.12: Carbon intensity (gCO_2eq/kWh) for eight timing scenarios

Per-Run Emissions Counterfactuals Because training energy is a fixed physical quantity for any given run, every percentage difference in grid intensity translates directly into the same percentage difference in CO_2. Figure 6.13 expresses this proportionality relative to the static CodeCarbon baseline, making the stakes concrete. Scheduling during summer mean conditions would reduce the reported emissions of this study by approximately 20 % compared to the static estimate, requiring no modification to the experimental setup whatsoever. The best-hour scenario pushes this reduction to 70 %, while the worst-hour scenario reverses it into a 73 % increase. These percentages are identical for every model and dataset, because the intensity factor scales uniformly. What differs is the absolute magnitude of the effect. For lightweight runs such as Logistic Regression on Wine, the difference between any two scenarios is negligible in absolute terms. For the MLP trained on HIGGS, the gap between best-hour and worst-hour timing amounts to approximately 208 g CO_2, a quantity that exceeds the total training footprint of every Wine and Credit run in this study combined. This asymmetry reveals an important practical principle: grid-aware scheduling is most valuable precisely where algorithmic and hardware efficiency decisions also matter most, namely on large, compute-intensive workloads where the absolute energy expenditure is high enough for timing to make a meaningful difference.

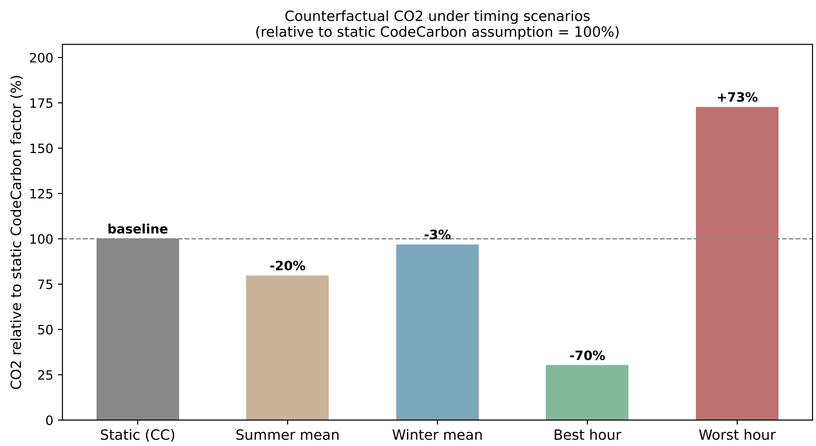


Figure 6.13: Counterfactual CO_2 relative to the static CodeCarbon baseline (= 100 %)

The Diurnal Dimension To understand why the diurnal dimension matters as much as the seasonal one, consider the solar duck curve visible in Figure 6.14 for 24 July 2024. Carbon intensity declines from roughly 300 gCO_2eq/kWh in the morning as solar generation ramps up, reaches a minimum of 163 gCO_2eq/kWh near midday, then recovers sharply to a peak of 461 gCO_2eq/kWh once solar output falls in the evening. The static CodeCarbon factor of 381 gCO_2eq/kWh intersects this curve only during the early morning and late evening hours, the two dirtiest windows of the day on a summer date with strong solar generation. During the core daylight hours, the static factor overestimates the actual intensity by up to a factor of two. This discrepancy empirically validates recent arguments that relying on generation-based average rates is inadequate for true consequential carbon accounting, which instead requires time-specific marginal emissions data [8]. This is not an anomaly specific to this particular date: it is the structural consequence of Germany's growing solar capacity, which consistently depresses midday intensity throughout the summer months. A practitioner who runs training jobs during office hours on summer days is inadvertently benefiting from this effect, while their CodeCarbon reports attribute a higher carbon cost than actually occurred.

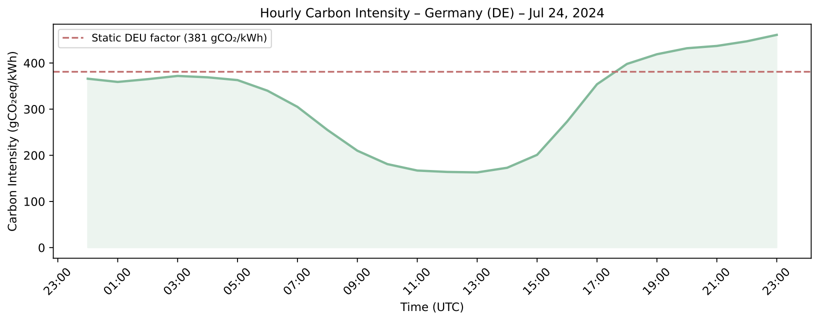


Figure 6.14: Hourly carbon intensity on 24 July 2024 in Germany

The Seasonal Dimension The seasonal dimension adds a second layer of variation on top of the diurnal one. The ten-day windows shown in Figure 6.15 reveal that winter and autumn both sit above the static CodeCarbon factor on average, meaning CodeCarbon systematically *underestimates* winter emissions rather than overestimating them. The winter window averages 474 gCO_2eq/kWh, nearly 25 % above the static factor, while spring (347 gCO_2eq/kWh) and summer (361 gCO_2eq/kWh) fall meaningfully below it. This directional asymmetry matters for transparency: the runs in this study were conducted during periods of varying grid intensity, and the static factor provides no indication of whether a given run's reported footprint is an over- or underestimate. An equally important finding is that within-season variance is large enough to undermine coarse seasonal scheduling as a reliable strategy. The winter window alone spans from 311 to 614 gCO_2eq/kWh across ten consecutive days, a range wider than the gap between winter and summer means. A practitioner who decides to schedule training in July rather than January improves their expected footprint, but cannot assume low intensity on any given July day without consulting real-time grid data.

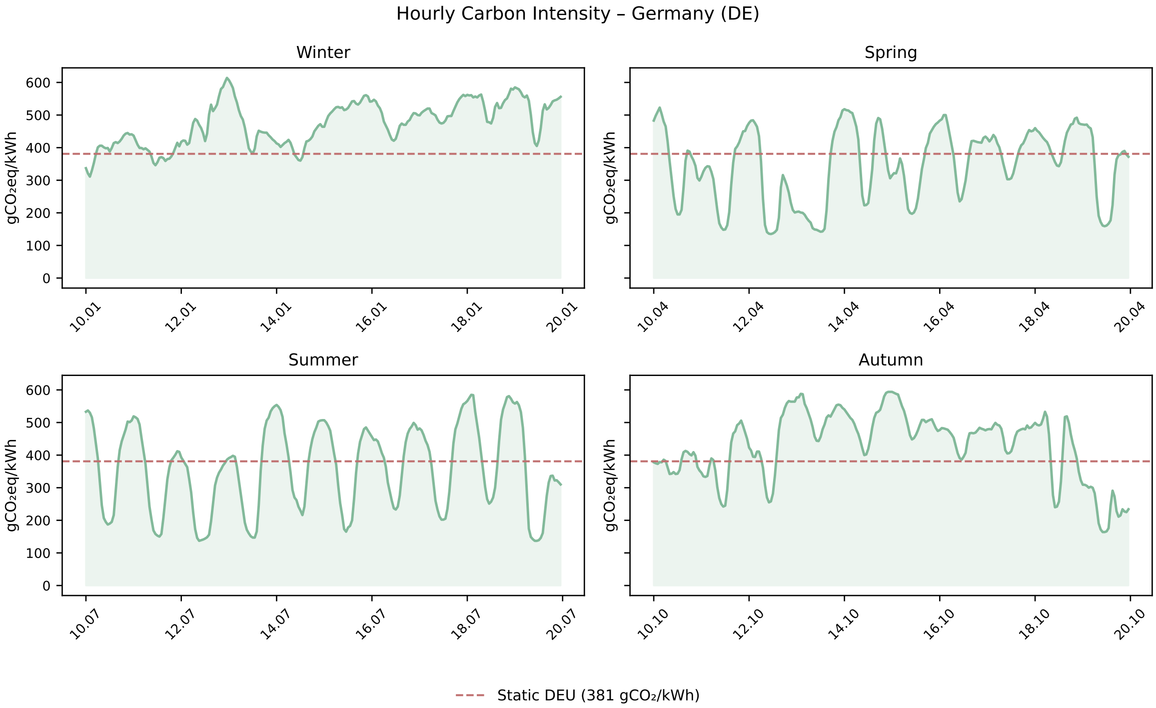


Figure 6.15: Hourly carbon intensity for representative ten-day windows per season, Germany 2025

Predictable Structure and Actionable Scheduling The year-long heatmap in Figure 6.16 brings both dimensions together and makes their joint structure immediately legible. The lowest-intensity region of the entire year is summer midday: a consistent band of green running from May through August between approximately 10:00 and 15:00, reflecting the hours when solar generation is both abundant and at its seasonal peak. The highest-intensity region is winter evenings: the band of red and orange concentrated between 17:00 and 21:00 from November through February, when solar output has already dropped to near zero and space heating drives gas and coal generation. What the heatmap makes clear is that these two extremes are not separated by chance. They are structurally determined by the interaction of Germany's solar build-out with the seasonal daylight cycle and heating demand patterns. A practitioner optimizing scheduling on local infrastructure is therefore not searching for a random low-intensity moment. They are navigating a predictable structure where the best and worst times are largely known in advance. The summer-midday to winter-evening gradient spans the full two-to-one ratio identified earlier, and represents the maximum emissions reduction achievable through scheduling on the German grid without any hardware or algorithmic changes. Exploiting these predictable patterns through elastic, carbon-aware workload scheduling thus emerges as a primary strategy for decarbonizing machine learning [8, 47].

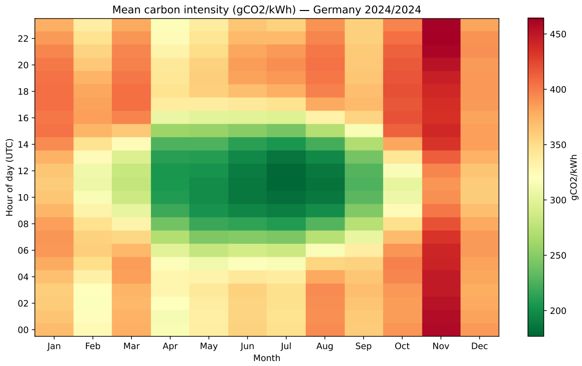


Figure 6.16: Mean carbon intensity (gCO_2eq/kWh) by hour of day and calendar month for the German grid (2024), derived from 8,760 hourly observations.

Synthesis Taken together, these findings introduce a dimension of ecological efficiency that the main benchmark cannot capture: timing. The static CodeCarbon factor is not grossly wrong as an annual average, but it flattens a distribution that is wide enough to change the practical interpretation of the results. It is worth noting that this intensity overestimation of roughly 12 % (381 versus 341 gCO_2eq/kWh) acts in the *opposite* direction to the TDP-based underestimation documented in Section 6.1, which inflates the corrected value by a median factor of 1.87. The intensity error is small enough that the energy-measurement error dominates in every case; the net effect of uncorrected CodeCarbon is still substantial undercounting. The emissions reported in this study should therefore be understood as contingent on when the runs were executed, not as intrinsic properties of the algorithms. For small datasets such as Wine and Credit, this contingency is inconsequential in absolute terms, as the runs are too cheap for timing to shift the total by a meaningful amount. For large-scale workloads such as the MLP on HIGGS, grid-aware scheduling would have reduced the training footprint by the equivalent of the entire Credit dataset's combined model training budget. At that scale, scheduling is no longer a marginal consideration but a first-order efficiency lever, comparable in impact to moderate algorithmic improvements, and achievable without touching a single line of model code.

Figure 6.17 extends this picture across six complete calendar years (2020–2025), confirming that the diurnal and seasonal structure identified above is not an artifact of a single year but a stable, recurring property of the German grid.

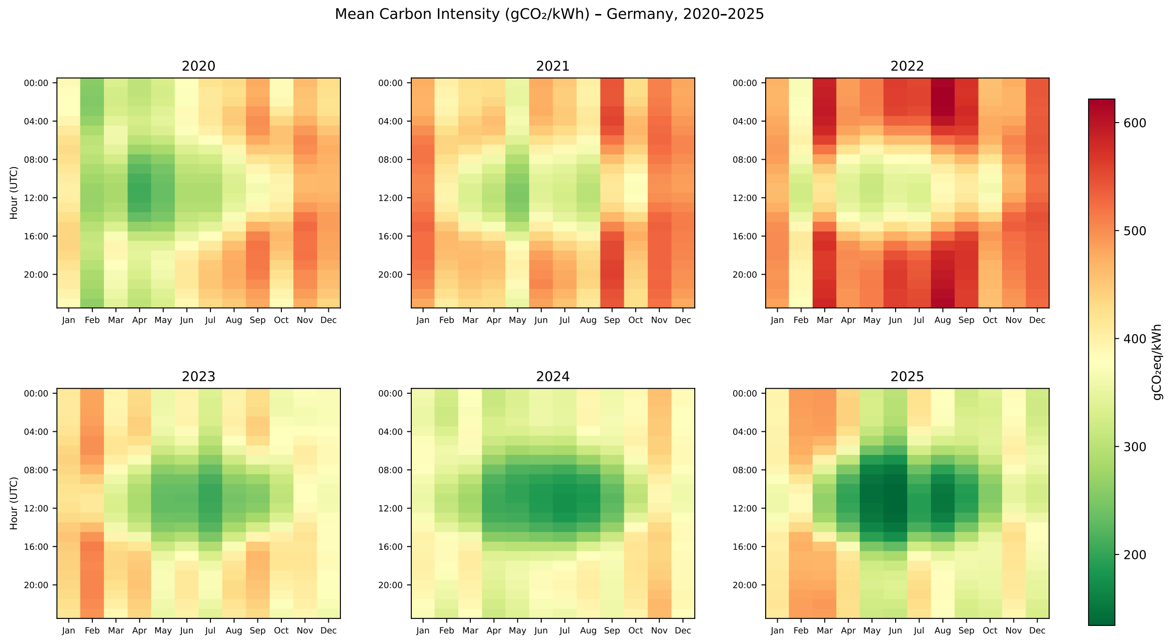


Figure 6.17: Mean carbon intensity (gCO_2eq/kWh) by hour of day and calendar month for the German grid, 2020–2025. Each panel is derived from 8,760 hourly observations (8,784 in leap years 2020 and 2024). The shared color scale allows direct cross-year comparison. The summer-midday low-intensity band and winter-evening peak are consistent across all six years.

6.8 Lifecycle Break-Even Analysis

Figure 6.18 shows for each model-dataset combination the number of predictions at which cumulative inference CO_2 surpasses the one-time training footprint.

Training CO_2 is a fixed, one-time cost. Inference CO_2 accumulates with every prediction served. Whether training or inference dominates the total lifecycle footprint depends on deployment volume, and that relationship differs markedly across the model-dataset combinations in this study. Break-even points are computed from the hardware-corrected training footprint and measured per-sample inference latency, scaled by the grid carbon intensity derived from the total training energy (CPU, GPU, and RAM combined). CPU power during inference was measured directly; GPU idle draw during inference is not tracked, so inference costs for GPU-trained models represent a lower bound.

The lowest break-even point is Random Forest on Wine, at 105 predictions. Random Forest inference is slow relative to other classifiers: each prediction requires sequential traversal of the learned tree ensemble, producing a per-prediction cost of approximately 275 µg CO_2. Against a training footprint of only 28.8 mg, the inference cost accumulates to parity within a few minutes of active serving. For any realistic Random Forest deployment on tabular data at this scale, inference is the dominant ecological cost by a wide margin.

The opposite extreme is MLP on HIGGS, which does not reach break-even until approximately 23 million predictions. MLP inference is fast: the forward pass through a fixed-depth network requires no tree traversal, producing a per-prediction cost of only 6.3 µg CO_2. Against a training footprint of 145.9 g, that rate of accumulation reaches parity only after

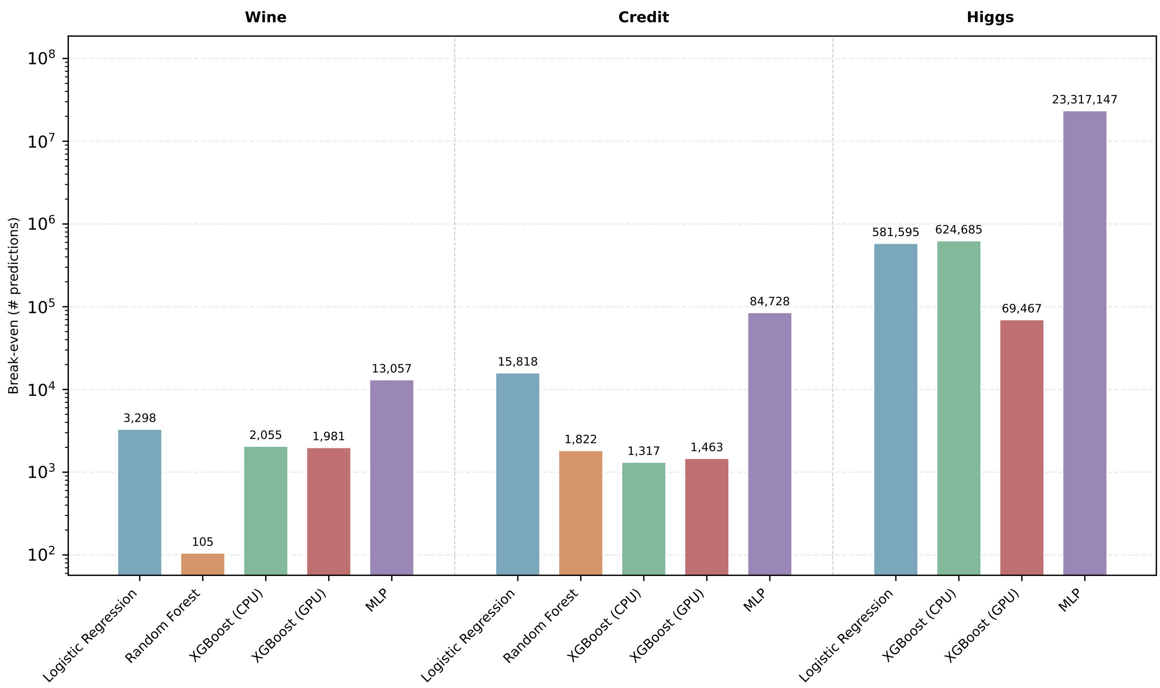


Figure 6.18: Break-even number of predictions at which cumulative inference CO_2 emissions surpass the one-time training footprint (log scale).

twenty-three million queries. A system serving this model on a low-traffic application would find that training represents the dominant ecological cost throughout its entire operational lifetime.

Between these extremes, break-even points organise around two drivers: training scale and inference latency. Logistic Regression on Credit breaks even at 15,818 predictions because both costs are modest in absolute terms. XGBoost GPU on HIGGS breaks even at 69,467 predictions: its training footprint is contained at 2.10 g, but inference at 30.3 µg per prediction accumulates faster than MLP's 6.3 µg, so the budget is recovered at a far lower deployment volume. The contrast between XGBoost GPU (69,467) and MLP (23 million) on the same dataset illustrates a deeper trade-off in model design. MLP's fast inference pushes the break-even point to a scale that most low-traffic applications will never reach, concentrating the ecological cost at training time. XGBoost GPU's slower inference distributes the lifecycle cost more evenly across the deployment period. For practitioners concerned with total deployment footprint rather than single-phase training emissions, the break-even point is as informative as any figure in the benchmark table.

The roughly five orders of magnitude separating these extremes reflect a structural divide rather than an accident of scale. Models with cheap training and slow inference front-load their ecological cost at the point of training, then spend that cost again with every subsequent prediction. Models with expensive training and fast inference invert this: the initial investment is large, but it is amortised across a large number of cheap queries. Which profile is ecologically preferable depends entirely on how many predictions the deployed system will serve, a figure that has nothing to do with accuracy or training efficiency [48].

This creates a direct inversion of the training-only rankings for high-volume deployments

Table 6.3: Lifecycle break-even analysis. Training CO_2 is the one-time hardware-corrected footprint. CO_2 per prediction is derived from measured inference latency and CPU power, scaled by the same grid intensity as training. Break-even is the number of predictions at which cumulative inference CO_2 equals training CO_2. Random Forest was not run on HIGGS.

Dataset	Model	Train CO_2 (μg)	CO_2/pred. (ng)	Break-even
Wine	Logistic Regression	13.7	4.2	3,298
	Random Forest	28.8	274.6	105
	XGBoost (CPU)	20.5	10.0	2,055
	XGBoost (GPU)	51.2	25.9	1,981
	MLP	58.7	4.5	13,057
Credit	Logistic Regression	60.5	3.8	15,818
	Random Forest	831.6	456.3	1,822
	XGBoost (CPU)	43.3	32.9	1,317
	XGBoost (GPU)	51.7	35.3	1,463
	MLP	633.0	7.5	84,728
HIGGS	Logistic Regression	5,148	8.9	581,595
	XGBoost (CPU)	14,650	23.5	624,685
	XGBoost (GPU)	2,104	30.3	69,467
	MLP	145,891	6.3	23,317,147

[39]. Random Forest on Wine ranks favourably on EWF1 precisely because it is cheap to train. Yet at ten thousand predictions it has already accumulated roughly 95 times its training budget in inference emissions, while MLP at the same query count has barely touched its much larger training investment. For a practitioner deploying at scale, the EWF1 ranking is not merely incomplete at this point. It points in the wrong direction. Ecological model selection therefore requires a deployment volume estimate as an explicit input. Below the break-even threshold, optimising training emissions is the right lever. Above it, inference latency per prediction governs the total lifecycle cost, and no amount of training efficiency compensates for a slow prediction function at scale.

6.9 Chapter Summary

Across the eight analyses in this chapter, the empirical answer to the core research question is conditional rather than absolute. On the EWF1 metric, Logistic Regression leads on Wine (0.989), XGBoost CPU on Credit (0.800), and XGBoost GPU on HIGGS (0.760). No single model occupies all three positions because ecological efficiency depends entirely on the fit between model capacity, hardware utilization, and task complexity. Logistic Regression is efficient where a linear decision boundary is sufficient. XGBoost GPU is efficient where the massive parallelism of GPU-accelerated tree construction finally outweighs its fixed overhead. The rankings are dataset-contingent by design, demonstrating empirically that any claim that one classifier is categorically more ecological than another must strictly specify the scale and hardware context in which that claim is meant to hold.

More profoundly, the full accounting of the machine learning pipeline shifts this picture further in every dimension examined, exposing the severe ecological limitations of standard, training-only benchmarking. Hyperparameter search adds a one-time overhead that can exceed the training benchmark cost by more than an order of magnitude, as the MLP tuning run on HIGGS cost roughly 15 times the benchmark run it enabled. Grid carbon intensity introduces a factor-of-two scaling on effective emissions that depends on temporal scheduling alone, entirely independent of any modelling decision. Finally, lifecycle analysis distributes the total cost across the deployment period in ways that vary by roughly five orders of magnitude across model-dataset combinations.

A practitioner targeting ecological efficiency in tabular classification therefore cannot optimize training emissions in isolation. Model choice, hardware, tuning strategy, scheduling, and deployment volume each contribute a distinct lever, and they interact dynamically. The ultimate takeaway of this chapter is that true Green AI requires abandoning the narrow evaluation of algorithms in a computational vacuum. Chapter 7 draws on this combined, systemic picture to formulate actionable recommendations for each of these levers and to definitively answer the research questions stated in Section 4.1.

7 Conclusion and Outlook

Bibliography

[1] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, "Green AI," *Commun. ACM*, vol. 63, no. 12, pp. 54–63, Nov. 2020, ISSN: 0001-0782. DOI: 10.1145/3381831 Accessed: Mar. 19, 2026.

[2] R. Verdecchia, L. Cruz, J. Sallou, M. Lin, J. Wickenden, and E. Hotellier, "Data-Centric Green AI An Exploratory Empirical Study," in *2022 International Conference on ICT for Sustainability (ICT4S)*, Plovdiv, Bulgaria: IEEE, Jun. 2022, pp. 35–45, ISBN: 978-1-6654-8286-8. DOI: 10.1109/ICT4S55073.2022.00015 Accessed: Mar. 26, 2026.

[3] E. Strubell, A. Ganesh, and A. McCallum, *Energy and Policy Considerations for Deep Learning in NLP*, Jun. 2019. DOI: 10.48550/arXiv.1906.02243 arXiv: 1906.02243 [cs]. Accessed: Mar. 19, 2026.

[4] *Green AI Position Paper*, https://greensoftware.foundation/policy/research/green-ai-position-paper/. Accessed: Mar. 30, 2026.

[5] T. Eilam, P. Bello-Maldonado, B. Bhattacharjee, C. Costa, E. K. Lee, and A. Tantawi, "Towards a Methodology and Framework for AI Sustainability Metrics," in *Proceedings of the 2nd Workshop on Sustainable Computer Systems*, ser. HotCarbon '23, New York, NY, USA: Association for Computing Machinery, Aug. 2023, pp. 1–7, ISBN: 979-8-4007-0242-6. DOI: 10.1145/3604930.3605715 Accessed: Apr. 10, 2026.

[6] P. Henderson, J. Hu, J. Romoff, E. Brunskill, D. Jurafsky, and J. Pineau, "Towards the Systematic Reporting of the Energy and Carbon Footprints of Machine Learning," *Journal of Machine Learning Research*, vol. 21, no. 248, pp. 1–43, 2020, ISSN: 1533-7928. Accessed: May 6, 2026.

[7] A. Lacoste, A. Luccioni, V. Schmidt, and T. Dandres, *Quantifying the Carbon Emissions of Machine Learning*, Nov. 2019. DOI: 10.48550/arXiv.1910.09700 arXiv: 1910.09700 [cs]. Accessed: Mar. 19, 2026.

[8] J. Dodge et al., "Measuring the Carbon Intensity of AI in Cloud Instances," in *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency*, ser. FAccT '22, New York, NY, USA: Association for Computing Machinery, Jun. 2022, pp. 1877–1894, ISBN: 978-1-4503-9352-2. DOI: 10.1145/3531146.3533234 Accessed: Mar. 11, 2026.

[9] C. Rodriguez, L. Degioanni, L. Kameni, R. Vidal, and G. Neglia, *Evaluating the Energy Consumption of Machine Learning: Systematic Literature Review and Experiments*, Aug. 2024. DOI: 10.48550/arXiv.2408.15128 arXiv: 2408.15128 [cs]. Accessed: Mar. 11, 2026.

[10] N. Bannour, S. Ghannay, A. Névéol, and A.-L. Ligozat, "Evaluating the carbon footprint of NLP methods: A survey and analysis of existing tools," in *Proceedings of the Second Workshop on Simple and Efficient Natural Language Processing*, Virtual: Association for Computational Linguistics, 2021, pp. 11–21. DOI: 10.18653/v1/2021.sustainlp-1.2 Accessed: Mar. 26, 2026.

[11] L. Lannelongue, J. Grealey, and M. Inouye, "Green Algorithms: Quantifying the Carbon Footprint of Computation," *Advanced Science*, vol. 8, no. 12, p. 2 100 707, 2021, ISSN: 2198-3844. DOI: 10.1002/advs.202100707 Accessed: May 4, 2026.

[12] L. F. W. Anthony, B. Kanding, and R. Selvan, *Carbontracker: Tracking and Predicting the Carbon Footprint of Training Deep Learning Models*, Jul. 2020. DOI: 10.48550/arXiv.2007.03051 arXiv: 2007.03051 [cs]. Accessed: May 4, 2026.

[13] B. Courty et al., *Mlco2/codecarbon: V2.4.1*, Zenodo, May 2024. DOI: 10.5281/zenodo.11171501 Accessed: Apr. 15, 2026.

[14] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*. Springer, 2017, ISBN: 978-0-387-84858-7. Accessed: May 4, 2026.

[15] Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko, "Revisiting Deep Learning Models for Tabular Data," in *Advances in Neural Information Processing Systems*, vol. 34, Curran Associates, Inc., 2021, pp. 18 932–18 943. Accessed: Mar. 26, 2026.

[16] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014, ISSN: 1533-7928. Accessed: Apr. 18, 2026.

[17] J. H. Friedman, "Greedy function approximation: A gradient boosting machine.," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, Oct. 2001, ISSN: 0090-5364, 2168-8966. DOI: 10.1214/aos/1013203451 Accessed: Apr. 19, 2026.

[18] D. R. Cox, "The Regression Analysis of Binary Sequences," *Journal of the Royal Statistical Society. Series B (Methodological)*, vol. 20, no. 2, pp. 215–242, 1958, ISSN: 0035-9246. JSTOR: 2983890. Accessed: Apr. 8, 2026.

[19] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, no. 85, pp. 2825–2830, 2011, ISSN: 1533-7928. Accessed: Mar. 19, 2026.

[20] J. R. Quinlan, "Induction of decision trees," *Machine Learning*, vol. 1, no. 1, pp. 81–106, Mar. 1986, ISSN: 1573-0565. DOI: 10.1007/BF00116251 Accessed: Apr. 29, 2026.

[21] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" *Advances in Neural Information Processing Systems*, vol. 35, pp. 507–520, Dec. 2022. Accessed: Apr. 29, 2026.

[22] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001, ISSN: 1573-0565. DOI: 10.1023/A:1010933404324 Accessed: Mar. 19, 2026.

[23] L. Breiman, "Bagging predictors," *Machine Learning*, vol. 24, no. 2, pp. 123–140, Aug. 1996, ISSN: 1573-0565. DOI: 10.1007/BF00058655 Accessed: Apr. 29, 2026.

[24] E. Scornet, G. Biau, and J.-P. Vert, "Consistency of Random Forests," *The Annals of Statistics*, vol. 43, Apr. 2014. DOI: 10.1214/15-AOS1321

[25] G. Biau and E. Scornet, "A random forest guided tour," *TEST*, vol. 25, no. 2, pp. 197–227, Jun. 2016, ISSN: 1863-8260. DOI: 10.1007/s11749-016-0481-7 Accessed: Apr. 29, 2026.

[26] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '16, New York, NY, USA: Association for Computing Machinery, Aug. 2016, pp. 785–794, ISBN: 978-1-4503-4232-2. DOI: 10.1145/2939672.2939785 Accessed: Mar. 19, 2026.

[27] R. Mitchell and E. Frank, "Accelerating the XGBoost algorithm using GPU computing," *PeerJ Computer Science*, vol. 3, e127, Jul. 2017, ISSN: 2376-5992. DOI: 10.7717/peerj-cs.127 Accessed: Apr. 29, 2026.

[28] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Networks*, vol. 2, no. 5, pp. 359–366, Jan. 1989, ISSN: 0893-6080. DOI: 10.1016/0893-6080(89)90020-8 Accessed: Apr. 29, 2026.

[29] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, Oct. 1986, ISSN: 1476-4687. DOI: 10.1038/323533a0 Accessed: Apr. 19, 2026.

[30] D. P. Kingma and L. J. Ba, "Adam: A Method for Stochastic Optimization," 2015. Accessed: Apr. 19, 2026.

[31] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Transactions on Neural Networks*, vol. 5, no. 2, pp. 157–166, Mar. 1994, ISSN: 1941-0093. DOI: 10.1109/72.279181 Accessed: Apr. 29, 2026.

[32] G. Hinton, V. Nair, and Y. Rachmad, "Rectified Linear Units Improve Restricted Boltzmann Machines Vinod Nair," vol. 27, pp. 807–814, Jun. 2010.

[33] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *Proceedings of the 32nd International Conference on Machine Learning*, PMLR, Jun. 2015, pp. 448–456. Accessed: Apr. 18, 2026.

[34] L. Prechelt, "Early Stopping — But When?" In *Neural Networks: Tricks of the Trade: Second Edition*, G. Montavon, G. B. Orr, and K.-R. Müller, Eds., Berlin, Heidelberg: Springer, 2012, pp. 53–67, ISBN: 978-3-642-35289-8. DOI: 10.1007/978-3-642-35289-8_5 Accessed: Apr. 29, 2026.

[35] J. Nickolls and W. J. Dally, "The GPU Computing Era," *IEEE Micro*, vol. 30, no. 2, pp. 56–69, Mar. 2010, ISSN: 1937-4143. DOI: 10.1109/MM.2010.41 Accessed: May 4, 2026.

[36] P. Koshute, J. Zook, and I. McCulloh, *Recommending Training Set Sizes for Classification*, https://arxiv.org/abs/2102.09382v1, Feb. 2021. Accessed: Mar. 25, 2026.

[37] A. Althnian et al., "Impact of Dataset Size on Classification Performance: An Empirical Evaluation in the Medical Domain," *Applied Sciences*, vol. 11, no. 2, p. 796, Jan. 2021, ISSN: 2076-3417. DOI: 10.3390/app11020796 Accessed: Apr. 16, 2026.

[38] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, Jul. 2009, ISSN: 0306-4573. DOI: 10.1016/j.ipm.2009.03.002 Accessed: May 4, 2026.

[39] R. Desislavov, F. Martínez-Plumed, and J. Hernández-Orallo, "Trends in AI inference energy consumption: Beyond the performance-vs-parameter laws of deep learning," *Sustainable Computing: Informatics and Systems*, vol. 38, p. 100 857, Apr. 2023, ISSN: 22105379. DOI: 10.1016/j.suscom.2023.100857 Accessed: Apr. 10, 2026.

[40] G. C. Cawley and N. L. C. Talbot, "On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation," *Journal of Machine Learning Research*, vol. 11, no. 70, pp. 2079–2107, 2010, ISSN: 1533-7928. Accessed: May 16, 2026.

[41] L. Yang and A. Shami, "On hyperparameter optimization of machine learning algorithms: Theory and practice," *Neurocomputing*, vol. 415, pp. 295–316, Nov. 2020, ISSN: 0925-2312. DOI: 10.1016/j.neucom.2020.07.061 Accessed: May 4, 2026.

[42] J. Bergstra and Y. Bengio, "Random Search for Hyper-Parameter Optimization," *Journal of Machine Learning Research*, vol. 13, no. 10, pp. 281–305, 2012, ISSN: 1533-7928. Accessed: Apr. 19, 2026.

[43] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for Hyper-Parameter Optimization," in *Advances in Neural Information Processing Systems*, vol. 24, Curran Associates, Inc., 2011. Accessed: Apr. 19, 2026.

[44] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, ser. KDD '19, New York, NY, USA: Association for Computing Machinery, Jul. 2019, pp. 2623–2631, ISBN: 978-1-4503-6201-6. DOI: 10.1145/3292500.3330701 Accessed: Apr. 1, 2026.

[45] R. Fischer, *Ground-Truthing AI Energy Consumption: Validating CodeCarbon Against External Measurements*, Sep. 2025. DOI: 10.48550/arXiv.2509.22092 arXiv: 2509.22092 [cs]. Accessed: May 6, 2026.

[46] D. Patterson et al., *Carbon Emissions and Large Neural Network Training*, Apr. 2021. DOI: 10.48550/arXiv.2104.10350 arXiv: 2104.10350 [cs]. Accessed: May 9, 2026.

[47] C.-J. Wu et al., "Sustainable AI: Environmental Implications, Challenges and Opportunities," *Proceedings of Machine Learning and Systems*, vol. 4, pp. 795–813, Apr. 2022. Accessed: May 9, 2026.

[48] S. Luccioni, Y. Jernite, and E. Strubell, "Power Hungry Processing: Watts Driving the Cost of AI Deployment?" In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency*, ser. FAccT '24, New York, NY, USA: Association for Computing Machinery, Jun. 2024, pp. 85–99, ISBN: 979-8-4007-0450-5. DOI: 10.1145/3630106.3658542 Accessed: May 9, 2026.

[49] D. Hernandez and T. B. Brown, *Measuring the Algorithmic Efficiency of Neural Networks*, May 2020. DOI: `10.48550/arXiv.2005.04305` arXiv: `2005.04305 [cs]`. Accessed: May 9, 2026.

[50] C. Coleman et al., "Analysis of DAWNBench, a Time-to-Accuracy Machine Learning Performance Benchmark," *SIGOPS Oper. Syst. Rev.*, vol. 53, no. 1, pp. 14–25, Jul. 2019, ISSN: 0163-5980. DOI: `10.1145/3352020.3352024` Accessed: May 9, 2026.

[51] P. Mattson et al., "MLPerf Training Benchmark," *Proceedings of Machine Learning and Systems*, vol. 2, pp. 336–349, Mar. 2020. Accessed: May 9, 2026.

[52] M. F. Stefan Aeberhard, *Wine*, 1992. DOI: `10.24432/C5PC7J` Accessed: Mar. 19, 2026.

[53] I-Cheng Yeh, *Default of Credit Card Clients*, 2009. DOI: `10.24432/C55S3H` Accessed: Mar. 19, 2026.

[54] P. Baldi, P. Sadowski, and D. Whiteson, "Searching for Exotic Particles in High-Energy Physics with Deep Learning," *Nature Communications*, vol. 5, no. 1, p. 4308, Jul. 2014, ISSN: 2041-1723. DOI: `10.1038/ncomms5308` arXiv: `1402.4735 [hep-ph]`. Accessed: Mar. 19, 2026.

[55] D. Whiteson, *HIGGS*, 2014. DOI: `10.24432/C5V312` Accessed: Mar. 19, 2026.

[56] A. Bagnall and G. C. Cawley, *On the Use of Default Parameter Settings in the Empirical Evaluation of Classification Algorithms*, Mar. 2017. DOI: `10.48550/arXiv.1703.06777` arXiv: `1703.06777 [cs]`. Accessed: Apr. 10, 2026.

[57] A. Paszke et al., *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, Dec. 2019. DOI: `10.48550/arXiv.1912.01703` arXiv: `1912.01703 [cs]`. Accessed: Mar. 19, 2026.

[58] B. Ghosh, *Skorch: The Bridge Between PyTorch and Scikit-Learn*, Sep. 2024. Accessed: Mar. 26, 2026.

[59] *Claude Code*, Anthropic, 2026.

[60] N. Feix, *Snip3rnick/PyHardwareMonitor*, Mar. 2026. Accessed: May 8, 2026.

[61] *Electricity Maps API: Carbon Intensity History*. Accessed: Apr. 20, 2026.

[62] European Environment Agency, *Monitoring of CO2 emissions from passenger cars, 2024 - Provisional data*, 2025. DOI: `10.2909/5814203D-F36D-4FEE-90D4-64BB348D15A6` Accessed: May 18, 2026.

List of Figures

List of Tables